

**THE UNIVERSITY OF DANANG
DANANG UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY**



GRADUATION PROJECT THESIS

**MAJOR: INFORMATION TECHNOLOGY
SPECIALTY: DATA SCIENCE AND ARTIFICIAL
INTELLIGENCE**

PROJECT TITLE:

BUILDING A SYSTEM TO IDENTIFY THE TONIC CHORD OF SONGS

Instructor: **MSc. DO THI TUYET HOA**
Student: **TRAN DUC TRI**
Student ID: **102210096**
Class: **21TCLC_KHDL**

Da Nang, 06/2025

**THE UNIVERSITY OF DANANG
DANANG UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY**



GRADUATION PROJECT THESIS

MAJOR: INFORMATION TECHNOLOGY

**SPECIALTY: DATA SCIENCE AND ARTIFICIAL
INTELLIGENCE**

PROJECT TITLE:

BUILDING A SYSTEM TO IDENTIFY THE TONIC CHORD OF SONGS

Instructor: MSc. DO THI TUYET HOA

Student: TRAN DUC TRI

Student ID: 102210096

Class: 21TCLC_KHDL

Da Nang, 06/2025

GRADUATION PROJECT COMMENT

1. General information:

1. Student name: Tran Duc Tri
2. Class: 21TCLC_KHDL Student ID: 102210096
3. Topic title: Building a system to identify the tonic chord of songs
4. Instructor: MSc. Do Thi Tuyet Hoa Academic title/ degree: Master of Science

2. Reviews of graduation project

1. About the urgency, novelty, usability of the topic: (2 points)

.....
.....

2. About the results of solving the tasks required by the project: (4 points)

.....
.....

3. About the form, structure and layout of the graduation project: (2 points)

.....
.....

4. The topic includes scientific value/article/problem solving of the enterprise or school: (1 point)

.....
.....

5. Existing shortcomings need to be supplemented or modified:

.....
.....

3. Spirit and attitude of the student (1 point):

.....
.....

4. Evaluation:

1. Evaluation point: /10
2. Suggest: Defense permitted/ Edit to defend/ Defense not permitted

Da Nang, date.....month..... 2025

Instructor

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the entire width of the page, providing a guide for handwriting or typing. There are no margins, text, or other markings on the page.

Instructor

[illegible]

Reviewer

SUMMARY

Project name: Building a system to identify the tonic chord of songs

Student name: Tran Duc Tri

Student ID: 102210096

Class: 21TCLC_KHDL

Summary: Nowadays, with the rapid advancement of Artificial Intelligence (AI), especially in the field of audio processing, it is now increasingly feasible to analyze and identify the tonic chords of a song. This progress has opened the door to various practical applications in music, such as accompaniment support, music theory analysis, and intelligent tools that assist musicians in identifying tonic chords more easily.

This project aims to build a smart and user-friendly system that supports users—especially music learners and enthusiasts—in identifying the tonic chord of a song. By leveraging AI techniques, the system delivers accurate chord recognition and provides helpful features to enhance the overall music experience.

Key features include:

- **Tonic Chord Identification:** Users can upload an audio file, and the system will analyze the signal to determine the song's main chord. *Example:* When uploading the audio "*Mưa Hồng - Lê Hiếu*", the system returns **C** as the tonic chord. When uploading the audio "*Mưa Hồng – Lê Quyên*", the system returns **G** as the tonic chord.
- **Voice Separator:** This feature separates vocals from the instrumental track, allowing users to download isolated audio files for practice, remixing, or analysis. This platform provides a powerful and intelligent solution for understanding the musical structure of songs, bridging the gap between AI technology and creative music applications.

GRADUATION PROJECT REQUIREMENTS

Student Name: Tran Duc Tri

Student ID: 102210096

Class: 21TCLC_KHDL Faculty: Information Technology

Major: Data Science and Artificial Intelligent

1. *Topic title:* Building a system to identify the tonic chord of songs
2. *Project topic:* ☐ *has signed intellectual property agreement for final result*
3. *Initial figure and data:* No data
4. *Content of the explanations and calculations:*

The content of the thesis includes:

INTRODUCTION - This chapter gives information about the context and purpose of the project as well as giving the scope of the problems which will be focused on the thesis.

Chapter 1: THEORIES AND TECHNOLOGIES - This chapter introduces the core theories and technologies used in the project, including fundamentals of music theory, digital audio processing, and artificial intelligence approaches.

Chapter 2: ANALYSIS AND DESIGN - This chapter describes the system's key features, functional requirements, and architectural design combining software components and AI-based processing.

Chapter 3: IMPLEMENTATION AND EVALUATION - This chapter details the implementation of the system and its AI components, along with experimental results and performance evaluation.

CONCLUSION - The concluding section of the project simultaneously emphasizes the problem solved, as well as presenting issues still unresolved and provides recommendations and suggestions.

REFERENCES - Presentation about detail of referenced information used in this thesis.

5. *Drawings, charts (specify the types and sizes of drawings):*

- Use case diagram
- Activity diagram
- Sequence diagram

6. *Name of instructors:* MSc. Do Thi Tuyet Hoa

7. *Date of assignment :* 04/04/2025

8. *Date of completion :*/...../2025

Da Nang, date 19 month 6 year 2025

Head of Division.....

Instructor

PREFACE

After an intensive period of over two months, today marks the culmination of my thesis project. This journey has been a time of profound learning and growth. As I look back, I would like to take this opportunity to sincerely thank those who have supported and accompanied me throughout this process.

First, I would like to express my gratitude to my instructor, MSc. Do Thi Tuyet Hoa, for her role in supervising and guiding this project. Despite her busy schedule, she regularly provided feedback, monitored my progress, and offered helpful suggestions that helped shape the direction of my work. Her involvement ensured that I stayed on track and maintained focus on the core objectives of the project.

I would also like to thank my family and friends for their steady support throughout the thesis period. Their encouragement, patience, and belief in me helped me stay motivated and overcome the challenges during both the learning and implementation phases.

Without your generous help and encouragement, my senior year would not have been as successful.

Thank you for being a part of this journey.

Sincerely,

Tran Duc Tri

ASSURANCE

I guarantee:

The contents of this senior project are performed by myself and my teammate following the guidance of instructors MSc. Do Thi Tuyet Hoa.

All references used in this senior project thesis, are quoted with the author's name, project name, time and location to publish clearly and faithfully.

All invalid copies, educated statute violation or cheating will be borne the full responsibility by myself.

Student Performed

Tran Duc Tri

WORK RESPONSIBILITY AND DISTRIBUTION OF TASKS

No.	Task	Description	Persion in charge
1	Define Requirements	Research the problem domain, define the system scope, document functional and non-functional requirements, and identify main features such as chord recognition and vocal separation.	Tran Duc Tri Pham Nguyen Anh Phat
2	Research Music Theory and AI Models	Study basic music theory to understand chords and song structure. Research audio signal processing, suitable datasets, and deep learning models CNN, CRNN	Tran Duc Tri
		Learn about the human singing voice, vocal enhancement techniques to avoid noise and musical influences, and explore the U-Net model for vocal separation and its improved version TDF_TFC_U-Net	Pham Nguyen Anh Phat
3	System Architecture Design	Design the overall architecture of the system, including the interaction between frontend, backend, AI services, and cloud infrastructure (AWS).	Tran Duc Tri Pham Nguyen Anh Phat
4	Implement AI Models	Train and evaluate CNN, CRNN for chord detection and optimize performance and integrate models into the system.	Tran Duc Tri

		Train and evaluate U-Net-based model for vocal separation. Optimize performance and integrate models into the system.	Pham Nguyen Anh Phat
5	Backend Implementation	Develop backend APIs for handling audio uploads, model inference, and file processing. Integrate with AI models and cloud storage.	Tran Duc Tri
6	Frontend Implementation	Build the web interface, implement user interaction features, visualize chord output and playback functionality. Ensure smooth integration with backend services.	Pham Nguyen Anh Phat
7	Cloud Deployment	Deploy the system using AWS services such as EC2 to Run AI server and backend server, using S3 to hosting web and set up CDN for delivery	Tran Duc Tri
		Send EC2 error logs to CloudWatch, trigger a Lambda on errors, and notify a Google Chat webhook for real-time bug alerts.	Pham Nguyen Anh Phat
8	Testing and Evaluation	Perform testing with real audio data, evaluate system accuracy and usability, and compare model CNN, CRNN performances by confusion matrix	Tran Duc Tri
		Perform testing with real audio data, evaluate system accuracy and usability, and compare	Pham Nguyen Anh Pha

		model U-Net, TDF_TFC U-Net performances by confusion matrix	
9	Documentation	Compile project documentation, including methodology, system architecture, implementation details, and evaluation results.	Tran Duc Tri Pham Nguyen Anh Phat

TABLE OF CONTENT

PREFACE	I
ASSURANCE	II
WORK RESPONSIBILITY AND DISTRIBUTION OF TASKS	III
TABLE OF CONTENT	VI
LIST OF PICTURES	VIII
LIST OF TABLES.....	XI
LIST OF SYMBOLS, ACRONYM.....	XII
INTRODUCTION	1
CHAPTER 1: THEORIES AND TECHNOLOGIES	4
1.1 Javascript	4
1.2 VueJS	4
1.3 FastAPI.....	5
1.4 HTTP API	5
1.5 Domain Name System (DNS).....	6
1.6 Convolutional Neural Network (CNN).....	7
1.7 Convolutional Recurrent Neural Network (CRNN).....	8
1.8 U-shaped Convolutional Network Architecture (Unet)	9
1.8 Digital Audio Signal Processing	10
1.9 Overview of Music Theory and Tonic Chords.....	12
1.10. Overview of the problem of identifying the tonic chord	15
1.11. Overview of the vocal separation solution.....	18
1.9. AWS	21
1.10. Conclusion	23
CHAPTER 2: SYSTEM ANALYSIS AND DESIGN.....	24
2.1 Requirement analysis	24
2.1.1 Functional requirements	24
2.1.2 Non-functional requirements	24
2.2 System design	25
2.2.1 Use case diagram	25
2.2.2 Use case specification.....	26
2.2.3 System architecture.....	31
2.2.4 Activity Diagram	33
2.2.5 Sequence diagrams	35

2.2.6 API specifications.....	36
2.3 Method to Identify the Tonic Chord & Chord Progression	38
2.3.1 Data Solutions.....	38
2.3.2 Evaluation method and loss function.....	43
2.3.3 CNN Model Architecture for Identifying the Tonic Chord.....	45
2.3.4 CRNN Model Architecture for Identifying the Tonic Chord	51
2.3.5 Tonic Key Inference from Frame-wise Chord Predictions	57
2.4 Method to separate vocals using UNet & TFD_TFC_UNet	60
2.4.1 Data Solutions.....	60
2.4.2 Evaluation method and loss function.....	63
2.4.3 UNet model architecture for vocal separator.....	64
2.4.4 TFD_TFC_UNet for vocal separator.....	68
2.5 Conclusion	74
CHAPTER 3: SYSTEM IMPLEMENTATION AND EVALUATION	75
3.1 Experimental Results	75
3.1.1 Overview of the Tonic Chord Identification Task.....	75
3.1.2 Overview of the Vocal Separation Approach.....	113
3.2 Implementing the system	127
3.2.1 Technology stack.....	127
3.2.2 Deployment Enviroment.....	128
3.2.3 Domain Registration.....	129
3.2.4 System Access	130
3.4 Results.....	130
3.4.1 Home screen	130
3.4.2 Key Chord Detection Screen	131
3.4.3 Vocal Separator Screen	136
3.5 Conclusion	141
CONCLUSION	143
REFERENCES	145

LIST OF PICTURES

Figure 1.1: Rest API Architecture	5
Figure 1.2: CNN architectures	7
Figure 1.3: CRNN architectures	8
Figure 1.4: Unet Architectures	9
Figure 1.5: Digital audio signal example.....	10
Figure 1.6: Chord Progressions	14
Figure 1.7: Chroma of chords.....	16
Figure 1.8: STFT spectrogram of audio signal.....	19
Figure 1.9: Process of separating vocals from song	20
Figure 1.10: Amazon Web Services	21
Figure 1.11: EC2.....	21
Figure 1.12: Lambda.....	21
Figure 1.13: S3.....	22
Figure 1.14: Cloudfront	22
Figure 1.15: ACM.....	22
Figure 1.16: CloudWatch Logs.....	23
Figure 2.1: Overview use case diagram.....	26
Figure 2.2: System architecture	31
Figure 2.3: Activity diagram for recognize tonic chord	33
Figure 2.4: Activity diagram for extract vocal track	34
Figure 2.5: Recognize tonic chord sequence diagram	35
Figure 2.6: Extract vocal track sequence diagram.....	35
Figure 2.7: API specifications of detecting tonic chord of audio	37
Figure 2.8: API specifications of Separating voice from audio.....	38
Figure 2.9: Constant Q-Transform spectrogram.....	40
Figure 2.10: CNN Model Architecture	46
Figure 2.11: CRNN Model Architecture Overview	52
Figure 2.12: Musdb18 dataset.....	60
Figure 2.13: Waveform of one frame in dataset	61
Figure 2.14: Unet architecture	65
Figure 2.15: vocal separation pipeline.....	67
Figure 2.16: Time-Frequency Convolutions architecture.....	69
Figure 2.17: Time-Distributed Fully-connected networks architecture	71

Figure 2.18: TFC with TDF architecture.....	73
Figure 3.1: CQT-based spectrograms chords C, F, and A:min	77
Figure 3.2: Chord Distribution Analysis in Training and Test Sets	80
Figure 3.3: CNN model architecture summary.....	82
Figure 3.4: CNN training accuracy vs. validation accuracy over	83
Figure 3.5: CRNN model architecture summary	85
Figure 3.6: CRNN training vs. validation accuracy curves.	87
Figure 3.7: CNN Original Audio – Strict – No Mode-Aware	92
Figure 3.8: CNN Original Audio – Strict – Mode-Aware	93
Figure 3.9: Original Audio – Semi-Strict – No Mode-Aware	94
Figure 3.10: CNN Original Audio – Semi-Strict – Mode-Aware	95
Figure 3.11: Separated Audio – Strict – No Mode-Aware	96
Figure 3.12: CNN Separated Audio – Strict – Mode-Aware.....	97
Figure 3.13: CNN Separated Audio – Semi-Strict – No Mode-Aware	98
Figure 3.14: Separated Audio – Semi-Strict – Mode-Aware	99
Figure 3.15: Original Audio – Strict – No Mode-Aware.....	100
Figure 3.16: CRNN Original Audio – Strict – Mode-Aware	101
Figure 3.17: CRNN Original Audio – Semi-Strict – No Mode-Aware	102
Figure 3.18: CRNN Original Audio – Semi-Strict – Mode-Aware.....	103
Figure 3.19: CRNN Separated Audio – Strict – No Mode-Aware	104
Figure 3.20: CRNN Separated Audio – Strict – Mode-Aware.....	105
Figure 3.21: CRNN Separated Audio – Semi-Strict – No Mode-Aware	106
Figure 3.22: CRNN Separated Audio – Semi-Strict – Mode-Aware	107
Figure 3.23: Accuracy heatmap across 16 evaluation configurations	108
Figure 3.24: Direct accuracy comparison between CNN and CRNN	112
Figure 3.25: Data structure of musdb18 dataset	114
Figure 3.26: Spectrogram stft after process for vocal separation	114
Figure 3.27: UNet model summary	118
Figure 3.28: UNet training process.....	119
Figure 3.29: Loss and metric UNet training	119
Figure 3.30: Statistics spectrogram of test data UNet model.....	120
Figure 3.31: Statistics waveform of test data.....	120
Figure 3.32: Compare predict and true data test UNet model	121
Figure 3.33: TFD_TFC_UNet model summary	123
Figure 3.34: TFD_TFC_UNet training process.....	124
Figure 3.35: Loss and metric TFD_TFC_UNet training	125

Figure 3.36: Statistics spectrogram of test data metric TFD_TFC_UNet.....	126
Figure 3.37: Statistics waveform of test data metric TFD_TFC_UNet model.	126
Figure 3.38: Compare predict and true data test TFD_TFC_UNet model	126
Figure 3.39: Home page of HarmonySeeker	130
Figure 3.40: Main layout of the Key Detection page	131
Figure 3.41: Successful file upload and system ready for analysis	132
Figure 3.42: Visualization of the predicted key and suggested progressions..	133
Figure 3.43: Frame-wise chord progression timeline with playback support .	134
Figure 3.44: Error message for oversized file in song chord detection	135
Figure 3.45: Vocal separator screen before processing	136
Figure 3.46: File uploaded successfully and ready for separation.....	137
Figure 3.47: Display of original audio, separated vocals, and instrumentals..	138
Figure 3.48: File size exceeds limit error in voice separator	139
Figure 3.49: Error message for audio files exceeding 60 seconds	140

LIST OF TABLES

Table 1.1: Diatonic Chord Functions in the Major Scale	13
Table 2.1: Actor in the system.....	25
Table 2.2: Use case specification for recognize tonic chord	26
Table 2.3: Use case specification for extract vocal track	27
Table 2.4: Use case specification for viewing chord segmentation.....	28
Table 2.5: Use case specification for download separated audio	29
Table 2.6: API specifications of detecting tonic chord of audio	36
Table 2.7: API specifications of Separating voice from audio	37
Table 2.8: CQT parameters.....	40
Table 2.9: Parameter Rationale.....	42
Table 2.10: Distribution of chord predictions.....	59
Table 2.11: STFT parameters	62
Table 3.1: The top 5 most common chords	76
Table 3.2: Most Common Tonic Chords in Vietnamese Songs	79
Table 3.3: Side-by-Side Chord Type Comparison	81
Table 3.4: Evaluation Configurations for Tonic Chord Recognition	88
Table 3.5: Mapping of Major Keys to Their Relative Minor Keys	90
Table 3.6: Mapping of Minor Keys to Their Relative Major Keys	91
Table 3.7: Compare UNet with TFC_TDF_UNet	127
Table 3.8: System Specifications (EC2 Instances)	128
Table 3.9: DNS Mapping and Subdomains	129

LIST OF SYMBOLS, ACRONYM

No.	Item	Description
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	AWS	Amazon Web Services
4	CQT	Constant-Q Transform
5	CRNN	Convolutional Recurrent Neural Network
6	CNN	Convolutional Neural Network
7	GRU	Gated Recurrent Unit
8	HTML	Hypertext Markup Language
9	HTTP	Hypertext Transfer Protocol
10	HTTPS	Hypertext Transfer Protocol Secure
11	JSON	JavaScript Object Notation
12	UI	User Interface
13	UX	User Experience
14	UNet	U-shaped Convolutional Network
15	STFT	Short-Time Fourier Transform
16	SDR	Signal-to-Distortion Ratio
17	DOM	Document Object Model
18	API GW	API Gateway (AWS)
19	S3	Simple Storage Service (AWS)
20	EC2	Elastic Compute Cloud (AWS)
21	Lambda	AWS Lambda – Serverless Compute
22	CDN	Content Delivery Network
23	ACM	Amazon Certificate Manager
24	ECS	Amazon Elastic Compute Cloud
25	CW	CloudWatch
26	RESTful	Representational State Transfer
27	ReLU	Rectified Linear Unit
28	TFD_TFC	Time-Frequency Discriminator / Time-Frequency Channeling
29	BPM	Beats Per Minute

30	MIREX	Music Information Retrieval Evaluation eXchange
31	WCRA	Weighted Chord Root Accuracy
32	GT	Ground Truth
35	RNN	Recurrent Neural Network
36	FFT	Fast Fourier Transform
37	SR	Sampling Rate
38	MAE	Mean Absolute Error

INTRODUCTION

1. Problem Statement

In recent years, the integration of Artificial Intelligence (AI) into music has opened up new possibilities for analysis, composition, and creative expression. Among these advancements, the ability to automatically identify musical chords from audio has gained significant attention, particularly due to its relevance in music education, practice, and production.

However, for many music learners and enthusiasts—especially those without formal training—identifying the tonic chord of a song remains a challenging task. Existing tools often focus on full chord transcriptions or require advanced knowledge of music theory, making them less accessible for beginners. Furthermore, manual chord recognition is time-consuming and error-prone, especially when dealing with complex audio mixes.

Recognizing this gap, our project aims to build a simple yet effective web-based platform that allows users to upload an audio file and receive an accurate identification of the song's tonic chord. The system also offers an additional vocal remover feature, helping users isolate instruments for more focused analysis or practice.

This solution combines recent developments in audio signal processing and AI, delivering a practical and accessible tool for users to engage with music in a deeper and more intuitive way.

2. Purposes

The purpose of this project is to develop an intelligent and user-friendly system that assists users—particularly music learners, creators, and enthusiasts—in identifying the tonic chord of a song. By simplifying chord recognition through automation, the platform enables users to gain better musical insight, supporting activities such as learning to play by ear, remixing, and understanding song structure.

At the same time, the platform promotes accessibility by offering a lightweight, web-based interface that can be used from any device, anywhere, without the need for specialized software or music theory knowledge.

3. Objectives

Build a Functional Web Platform: Develop a simple and intuitive interface that allows users to upload songs and receive analysis results quickly.

Tonic Chord Recognition: Apply AI and audio signal processing techniques to identify the tonic chord of a song from an uploaded audio file.

Vocal Remover Feature: Provide an additional utility to separate vocals from instrumentals, enabling further analysis or usage.

Deploy the System to the Cloud: Ensure that the platform is always available and scalable by deploying backend services using AWS Lambda and other cloud technologies.

4. Implementation process

Step 1: Idea Development and Requirements Definition

We started by defining the scope of the system, outlining key features such as tonic chord identification and vocal separation, and specifying both functional and non-functional requirements.

Step 2: Music Theory Research and AI Model Exploration

To lay a solid foundation, we first studied basic music theory to understand song structure, chords, and harmonic relationships. Following this, we researched audio signal processing techniques and relevant AI models. We focused on extracting meaningful features from audio and explored suitable datasets. A CRNN model was trained for chord detection, while a U-Net-based model was developed for vocal separation.

Step 3: System Architecture Design

We designed the architecture of the entire system, including the frontend, backend, AI services, and cloud infrastructure. This design ensures modularity, scalability, and maintainability across all components.

Step 4: Feature Implementation

Core functionalities such as audio upload, chord detection, and vocal removal were implemented. The user interface was also built to ensure ease of use and smooth interaction.

Step 5: Cloud Deployment

The system was deployed using a combination of AWS services such as EC2, S3, Lambda, CDN, and ECS. Logs are tracked through CloudWatch and notifications are integrated with Google Chat to monitor the system in real time.

Step 6: Testing and Evaluation

We tested the system with various audio files and evaluated the performance of our models to ensure accurate output and responsive interaction.

Step 7: Documentation

The final stage involved compiling a comprehensive report that details the project goals, technical approach, implementation, and results.

5. Structure of the thesis

INTRODUCTION - This chapter gives information about the context and purpose of the project as well as giving the scope of the problems which will be focused on the thesis.

Chapter 1: THEORIES AND TECHNOLOGIES - This chapter introduces the core theories and technologies used in the project, including fundamentals of music theory, digital audio processing, and artificial intelligence approaches.

Chapter 2: ANALYSIS AND DESIGN - This chapter describes the system's key features, functional requirements, and architectural design combining software components and AI-based processing.

Chapter 3: IMPLEMENTATION AND EVALUATION - This chapter details the implementation of the system and its AI components, along with experimental results and performance evaluation.

CONCLUSION - The concluding section of the project simultaneously emphasizes the problem solved, as well as presenting issues still unresolved and provides recommendations and suggestions.

REFERENCES - Presentation about detail of referenced information used in this thesis.

CHAPTER 1: THEORIES AND TECHNOLOGIES

1.1 Javascript

JavaScript is a high-level programming language widely used in web development. It allows developers to create dynamic and interactive user interfaces on the browser. JavaScript can also be used on the server side through environments like Node.js.

In this project, JavaScript was mainly used to:

- Handle user interactions and form submissions.
- Make API calls to the backend (using Fetch or Axios).
- Dynamically update the user interface.

JavaScript works closely with HTML and CSS and plays a vital role in building modern web applications.

1.2 VueJS

Vue.js is a progressive JavaScript framework used for building user interfaces and single-page applications (SPAs). Created by Evan You, Vue is designed to be incrementally adoptable. Its core library focuses on the view layer only, making it easy to integrate with other libraries or existing projects. However, Vue is also capable of powering advanced applications when used in combination with modern tooling and supporting libraries.

Key Features

- **Declarative Rendering:** Vue allows developers to bind the data model to the DOM declaratively using template syntax.
- **Component-Based Architecture:** The UI is built from isolated and reusable components, each with its own HTML, CSS, and JavaScript logic.
- **Reactivity System:** Vue has a powerful and efficient reactivity system that automatically updates the DOM when the underlying data changes.
- **Virtual DOM:** Vue uses a virtual DOM for efficient rendering and better performance.
- **Single File Components (.vue):** Components can be written in .vue files that encapsulate template, logic, and style in a single file.
- **Advantages of Using Vue.js**
- **Ease of Use:** Vue has a gentle learning curve and simple syntax, making it ideal for both beginners and experienced developers.

- Flexible and Lightweight: Vue can be used for both small components and large-scale SPAs.
- High Performance: The virtual DOM and optimized reactivity system ensure smooth performance.
- Strong Community and Documentation: Vue has a vibrant community, a wide ecosystem, and well-maintained documentation.

1.3 FastAPI

FastAPI is a modern, high-performance web framework for building APIs with Python 3.7+ based on standard Python type hints. It is designed for building fast and scalable web APIs with automatic documentation, strong typing, and asynchronous support.

FastAPI is widely recognized for its simplicity, efficiency, and speed, and is often used for backend development in projects involving RESTful APIs, microservices, and machine learning.

1.4 HTTP API

An HTTP API (HyperText Transfer Protocol Application Programming Interface) is a standardized interface that allows different software systems to communicate with each other over the web using the HTTP protocol. It is one of the most common methods used in modern web development to enable the interaction between the client-side (frontend) and server-side (backend) of an application.

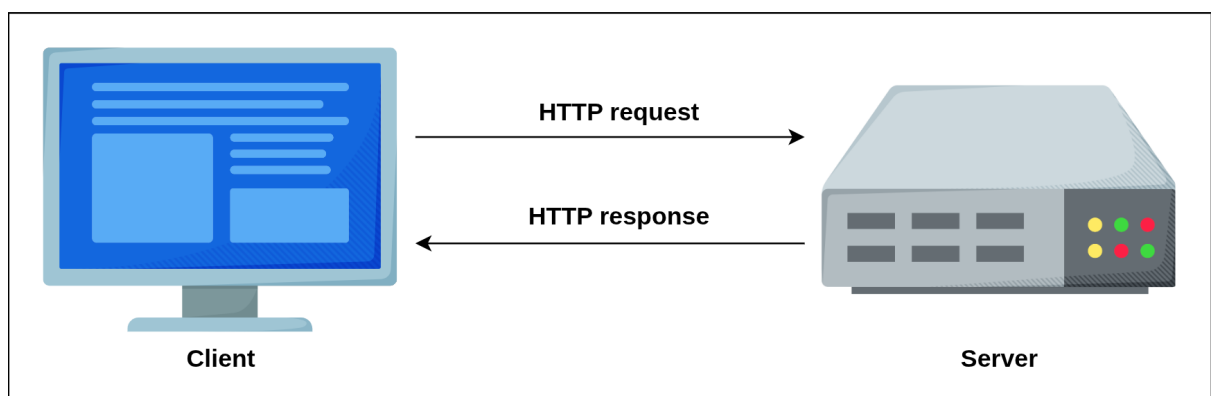


Figure 1.1: Rest API Architecture

Structure of an HTTP API

HTTP APIs are typically organized around RESTful principles (Representational State Transfer), where each resource (such as a user, order, or file) is

accessible through a specific URL (endpoint), and actions on those resources are performed using standard HTTP methods:

- GET: Retrieve data from the server
- POST: Send new data to the server
- PUT / PATCH: Update existing data
- DELETE: Remove data

API Request:

- URL: Identifies the resource
- Method: Specifies the operation (GET, POST, etc.)
- Headers: Additional metadata (e.g., authentication tokens, content type)
- Body: Contains data being sent to the server (usually in JSON format)

API Response

- Status Code: Indicates the result of the request (e.g., 200 OK, 404 Not Found, 500 Internal Server Error)
- Headers: Metadata about the response
 - Body: The actual data (usually in JSON format)

Benefits of Using HTTP APIs

- Platform Independent: Any client (browser, mobile app, or another server) can communicate with the API over HTTP.
- Separation of Concerns: Frontend and backend development can proceed independently.
- Scalability: Easily extendable as the application grows.
- Security: Can be secured using authentication methods such as API keys, JWT (JSON Web Tokens), or OAuth.

1.5 Domain Name System (DNS)

The Domain Name System (DNS) is a hierarchical and decentralized naming system used to translate human-readable domain names (e.g., `www.example.com`) into IP addresses (e.g., `192.0.2.1`) that computers use to identify each other on the Internet. DNS is a fundamental component of the modern web, allowing users to access websites and services without remembering complex numerical addresses.

DNS operates through several components:

- **Domain Registrars:** Entities that allow users to register and manage domain names (e.g., Namecheap, GoDaddy).
- **DNS Records:** Configuration entries such as A, CNAME, TXT, and MX records that determine how domain traffic is routed.
- **DNS Servers:**
 - o Authoritative DNS Servers hold actual domain records.
 - o Recursive Resolvers query DNS records on behalf of clients.

In this project, DNS is configured to map custom domain names such as api.harmonyseeker.site or ai.harmonyseeker.site to cloud services hosted on AWS. Proper DNS setup is crucial for routing requests securely to the backend server, enabling HTTPS support via certificate management (e.g., with ACM), and ensuring frontend/backend integration using CloudFront and API Gateway.

1.6 Convolutional Neural Network (CNN)

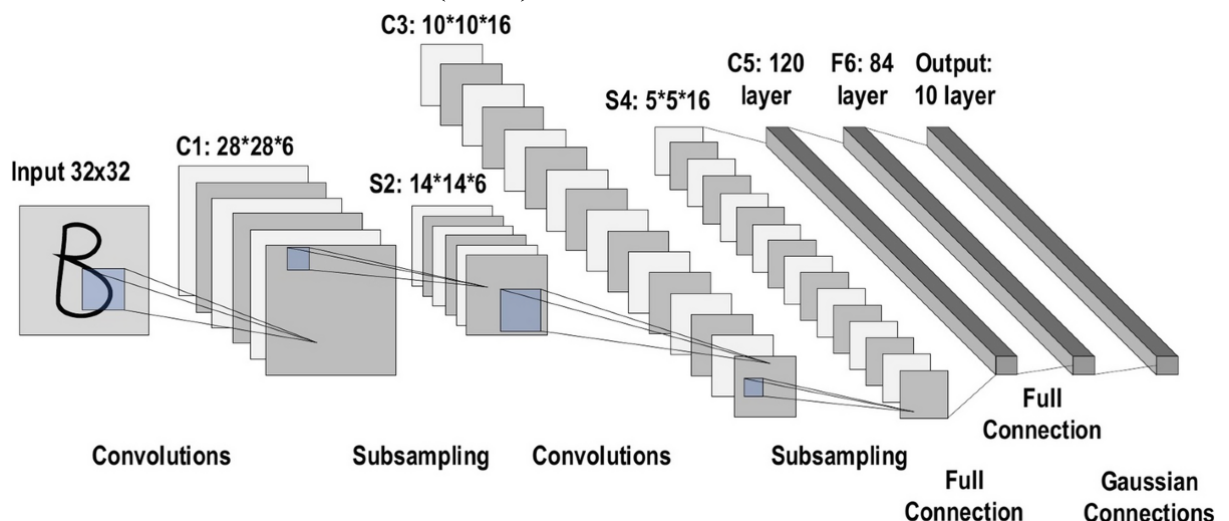


Figure 1.2: CNN architectures

Convolutional Neural Network (CNN) is a class of deep learning architectures particularly well-suited for processing grid-like data structures such as images, audio spectrograms, and time-frequency representations. Originally developed for image classification, CNNs have become a cornerstone in various domains, including computer vision, speech recognition, and music information retrieval.

A typical CNN architecture is composed of a series of layers including:

- **Convolutional layers**, which apply a set of learnable filters to input data to extract local features (e.g., harmonic textures, spectral energy).
- **Activation functions** like ReLU (Rectified Linear Unit), introducing non-linearity.

- **Pooling layers** (e.g., max pooling or average pooling), which reduce spatial dimensions while preserving the most prominent features.
- **Fully connected (dense) layers**, often used at the end for classification purposes.

The key advantage of CNNs lies in **local receptive fields**, **parameter sharing**, and **hierarchical feature learning**. These characteristics make CNNs especially efficient and effective in capturing localized patterns and spatial hierarchies, which are crucial when analyzing complex structures such as spectrograms of musical signals.

In the context of this project, CNNs are applied to **Constant-Q Transform (CQT)** representations of music to learn frequency-time patterns associated with chords. Although CNNs perform well in detecting local features, their limited temporal modeling ability restricts their performance in tasks that depend on sequential context, such as tracking chord progressions over time. This shortcoming leads to the use of hybrid architectures like CRNN.

1.7 Convolutional Recurrent Neural Network (CRNN)

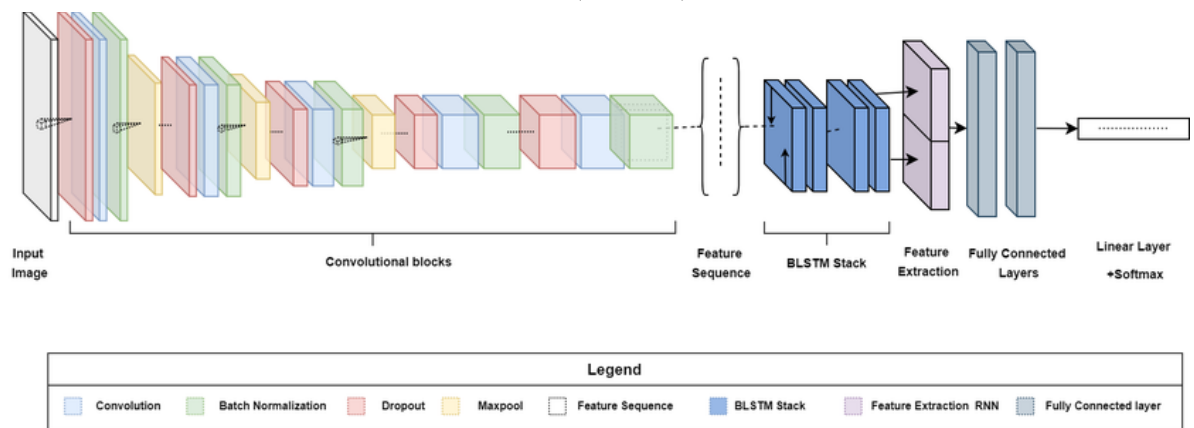


Figure 1.3: CRNN architectures

The Convolutional Recurrent Neural Network (CRNN) is a hybrid deep learning architecture that combines the strengths of CNNs and Recurrent Neural Networks (RNNs). While CNNs excel at extracting spatial features from input data, RNNs—particularly Gated Recurrent Units (GRUs) and Long Short-Term Memory (LSTM) cells—are adept at modeling temporal dependencies in sequential data. CRNNs integrate both capabilities to handle tasks involving spatiotemporal patterns.

A typical CRNN model includes:

- **Convolutional blocks**, which extract local time-frequency features from the input (e.g., CQT spectrogram).
- **Recurrent layers**, typically bidirectional GRU or LSTM, that capture temporal patterns and sequence dynamics by processing features across the time dimension.

- **Dense output layers** for final classification or prediction.

The combination of convolution and recurrence enables CRNNs to learn both local harmonic textures and global chord transitions—a critical requirement in music chord recognition, where chord labels not only depend on the current spectral content but also on musical context and temporal progression.

In this project, CRNN serves as the primary model for frame-wise chord recognition. It takes CQT spectrograms as input and outputs a probability distribution over chord classes at each time frame. Compared to CNNs, CRNNs have demonstrated significantly higher accuracy, particularly in identifying the tonic chord, by leveraging both short-term and long-term musical structures.

1.8 U-shaped Convolutional Network Architecture (Unet)

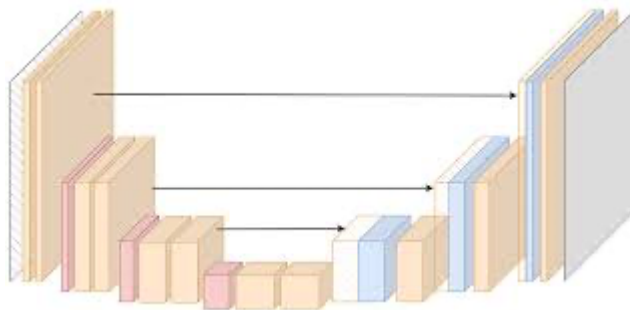


Figure 1.4: Unet Architectures

U-Net is a type of convolutional neural network designed for semantic segmentation (Olaf Ronneberger, 18 May 2015), originally introduced in the biomedical imaging field by Olaf Ronneberger et al. in 2015. Its unique U-shaped architecture and effective skip connections have made it a foundational model for tasks that require precise localization and detailed reconstruction.

The architecture consists of two main parts:

- Encoder (contracting path): This part captures context through a series of convolutional and pooling layers, progressively reducing the spatial resolution.
- Decoder (expanding path): This path upsamples the compressed features using transposed convolutions and merges them with corresponding feature maps from the encoder using skip connections. These connections help recover spatial details lost during downsampling.

U-Net is particularly useful in cases where training data is limited but spatial precision is essential. Its design ensures that both high-level contextual features and low-level spatial features are preserved and leveraged during prediction.

Beyond biomedical applications, U-Net has been widely adopted in:

- Remote sensing and satellite image segmentation

- Self-driving cars (road, lane, and obstacle segmentation)
- Speech enhancement and music source separation

In this project, a U-Net variant is used to separate vocals from accompaniment in music audio. By treating the time-frequency spectrogram as an image, U-Net learns to isolate vocal and instrumental components, enabling more accurate chord recognition by removing the influence of vocals. This pre-processing step improves the robustness of the chord recognition pipeline and enhances user experience by enabling karaoke-style outputs.

1.8 Digital Audio Signal Processing

Digital Audio Signal Processing is the process of converting and analyzing audio signals using computers after they have been digitized. This process includes sampling, quantization, and applying algorithms such as digital filtering and Fourier transform to process audio in the time or frequency domain (Choi, 2021). Common applications are in audio compression, audio effects, speech recognition, and the development of intelligent audio systems.

Digital Audio Signal

An audio signal is represented as a sequence of numbers in modern computer systems. As illustrated in Figure 2.1, each item in a digital audio signal is sampled from the value of a sound wave at a specific moment. A digital audio signal is commonly expressed as a discrete function $x[n]$, where n is an integer index and $x[n]$ is the sampled value at time step n (Choi, 2021).

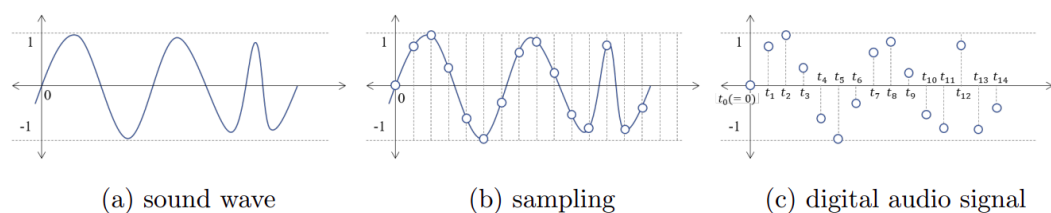


Figure 1.5: Digital audio signal example

The number of samples taken per second is called the sampling rate. One of the most common sampling rates is 44,100 Hz, which is slightly more than twice the maximum frequency that humans can hear, ensuring accurate sound reproduction according to the Nyquist theorem. In contrast, some systems may use lower sampling rates for specific purposes—for instance, in Automatic Speech Recognition (ASR), 16 kHz is often sufficient due to the limited frequency range of human speech. In such

cases, lower sampling rates may offer a good trade-off between performance and computational efficiency.

Time-Frequency Analysis

Many real-world applications require the analysis of audio signals in both the time and frequency domains. For example, music production software (DAW) developers may want to visualize the variation of the frequency spectrum over time to gain a better understanding of how the audio energy is distributed.

In such cases, Time–Frequency analysis methods, such as the Short-Time Fourier Transform (STFT) (Nuclear Engineering and Technology, 2024), are often used. The STFT performs the Fourier transform on fixed-length signal windows, resulting in a series of frequency spectra over time.

The result of the STFT is a complex coefficient, which is often decomposed into two parts: the magnitude spectrum and the phase spectrum. The magnitude spectrum provides information about the distribution of signal energy over frequency at each point in time, which is useful in many audio processing tasks such as analysis, source separation, or pattern recognition.

Short-time Fourier transform

The Short-Time Fourier Transform (STFT) is a fundamental technique in audio signal processing that enables the analysis of how the frequency content of a signal evolves over time. Unlike the standard Fourier Transform, which assumes a signal is stationary throughout its duration, STFT divides the audio signal into short, overlapping segments and computes the Fourier Transform for each segment individually. This allows for the examination of time-localized frequency content, which is crucial for non-stationary signals like speech and music.

Each segment, or window, captures a small slice of the audio, and the resulting series of spectra creates a time-frequency representation typically visualized as a spectrogram. The spectrogram encodes the intensity of frequencies over time, enabling researchers and engineers to observe harmonic patterns, transitions, and rhythmic structures. The choice of window length is critical, as it balances time and frequency resolution—short windows provide better time resolution while long windows enhance frequency resolution.

In the context of this project, STFT plays a critical role in the feature extraction pipeline. The raw waveform of the input audio is first converted into a spectrogram, from which further features such as the Constant-Q Transform (CQT) are derived.

These representations serve as the basis for machine learning models like CNN and CRNN to recognize chords and identify the tonic. STFT thus acts as a bridge between raw audio and meaningful, structured input for deep learning systems.

1.9 Overview of Music Theory and Tonic Chords

Music theory encompasses a set of principles that describe how musical elements such as pitch, harmony, and rhythm are organized. For computational systems dealing with audio content, particularly tasks like chord recognition, key estimation, harmonic analysis, and music generation, an understanding of tonal theory is essential. This section introduces foundational music theory concepts, including musical keys, tonic chords, scales, and chord progressions, with an emphasis on their structural and functional roles in Western tonal music.

Musical Key (Tonality)

The musical key, or tonality, is the harmonic framework that establishes the tonal center of a musical piece. It defines the basic scale and the relationships between pitches, chords, and harmonic progressions.

A musical key is determined by:

- Tonic: The central or root note, which serves as the tonal anchor.
- Mode: Typically Major or Minor, indicating the general character or mood of the music.

Examples:

- C Major: Notes are C–D–E–F–G–A–B. Often sounds bright and cheerful.
- A Minor: Notes are A–B–C–D–E–F–G. Often sounds melancholic or somber.

There are 24 standard keys in Western music:

- 12 Major keys: C, C#, D, D#, E, F, F#, G, G#, A, A#, B
- 12 Minor keys: Am, A#m, Bm, Cm, C#m, Dm, D#m, Em, Fm, F#m, Gm, G#m

Identifying the key of a piece is foundational for understanding its harmonic progression, guiding improvisation, supporting transposition, and improving the performance of automatic music processing systems.

Tonic Chord (Key Chord):

The tonic chord, also referred to as the key chord, is the triad or seventh chord built on the tonic note of the key. It serves as the harmonic center of gravity and is perceived as the point of rest or resolution within a musical phrase.

In C major, the tonic chord is C major (C–E–G)

In A minor, the tonic chord is A minor (A–C–E)

The tonic chord fulfills several critical roles:

- **Establishes Tonality:** It defines the home key and is often used to confirm the harmonic center.
- **Provides Resolution:** It acts as the final destination in cadences and progressions.
- **Functions as Reference:** Other chords derive functional identity (dominant, subdominant, etc.) relative to the tonic.

In harmonic analysis and chord prediction algorithms, the tonic chord serves as a reference point for building probabilistic or rule-based models of chord transitions.

Musical Scales

A scale is an ordered sequence of musical notes used to construct melodies and harmonies. It provides the sonic framework for harmonic and melodic structure.

- Major Scale formula: Whole – Whole – Half – Whole – Whole – Whole – Half
→ Example: C Major = C–D–E–F–G–A–B
- Natural Minor Scale formula: Whole – Half – Whole – Whole – Half – Whole – Whole
→ Example: A Minor = A–B–C–D–E–F–G

Other variants include harmonic minor, melodic minor, and modal scales such as Dorian, Phrygian, etc.

Scales are essential for:

- Building chords
- Developing melodies
- Forming harmonic progressions

Chord Progressions

Chord progressions are sequences of chords played in succession to form the harmonic structure of a piece or section of music.

Roman numeral notation for scale degrees:

Table 1.1: Diatonic Chord Functions in the Major Scale

Degree	Function	Example in C Major
I	Tonic	C
ii	Supertonic	Dm
iii	Mediant	Em

IV	Subdominant	F
V	Dominant	G
vi	Submediant	Am
vii°	Leading Tone	Bdim

Common chord progressions include:

- I–V–vi–IV (e.g., C–G–Am–F): Popular in pop music
- vi–IV–I–V (e.g., Am–F–C–G)
- ii–V–I (e.g., Dm–G–C): Common in jazz

Chord progressions shape the emotional direction of a song and create tension and resolution.

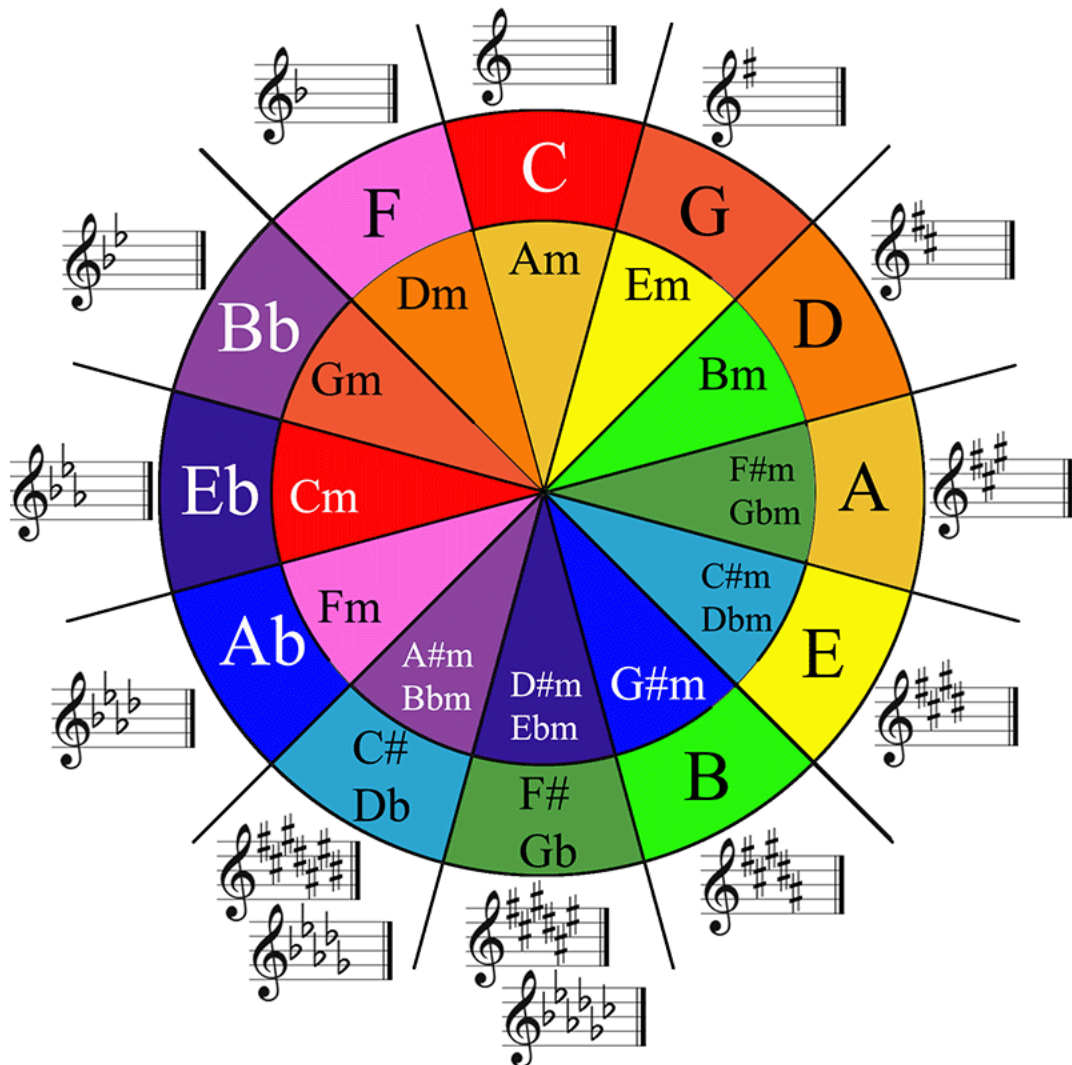


Figure 1.6: Chord Progressions

Importance of Key and Tonic Chord Identification

Identifying the key and tonic chord of a musical excerpt is critical for both theoretical analysis and practical applications in music technology. These tasks serve as the basis for a wide range of downstream processing techniques.

Applications in Music Technology:

- **Digital Signal Processing:** Enables harmonic segmentation and pitch normalization.
- **Chord Recognition Algorithms:** Narrows the set of likely chord candidates based on key context.
- **Transposition Systems:** Supports pitch shifting while maintaining harmonic integrity.
- **Automatic Music Generation:** Enhances coherence in generated content via key-based constraints.
- **Music Recommendation Systems:** Allows for mood classification and harmonic matching.
- **Performance and Arrangement:** Assists musicians in adapting songs to different vocal ranges or instruments.

The tonic chord, in particular, provides a harmonic anchor that simplifies the modeling of transitions and cadences, often serving as the primary target in resolution-oriented musical passages.

1.10. Overview of the problem of identifying the tonic chord

❖ Introduction and Challenges

In the field of music information retrieval and computational music analysis, identifying the tonic chord (also referred to as the key chord or root chord) of a song is a foundational problem. The tonic chord serves as the central harmonic reference around which the harmonic structure of a piece is organized. Accurate identification of this chord is crucial for tasks such as chord progression analysis, automatic music transcription, music recommendation, and emotion recognition (al M. e., 2014).

Despite its importance, key chord identification remains a challenging task due to several factors. Firstly, musical compositions often exhibit modulation (key changes), ambiguous harmonic structures, and non-diatonic chord usage, which complicate the analysis. Secondly, audio recordings contain overlapping harmonics, timbral variations, and noise, making it difficult to isolate the tonal center from raw signals. Additionally, performances can vary in style and interpretation, further obscuring the harmonic cues that indicate the key chord.

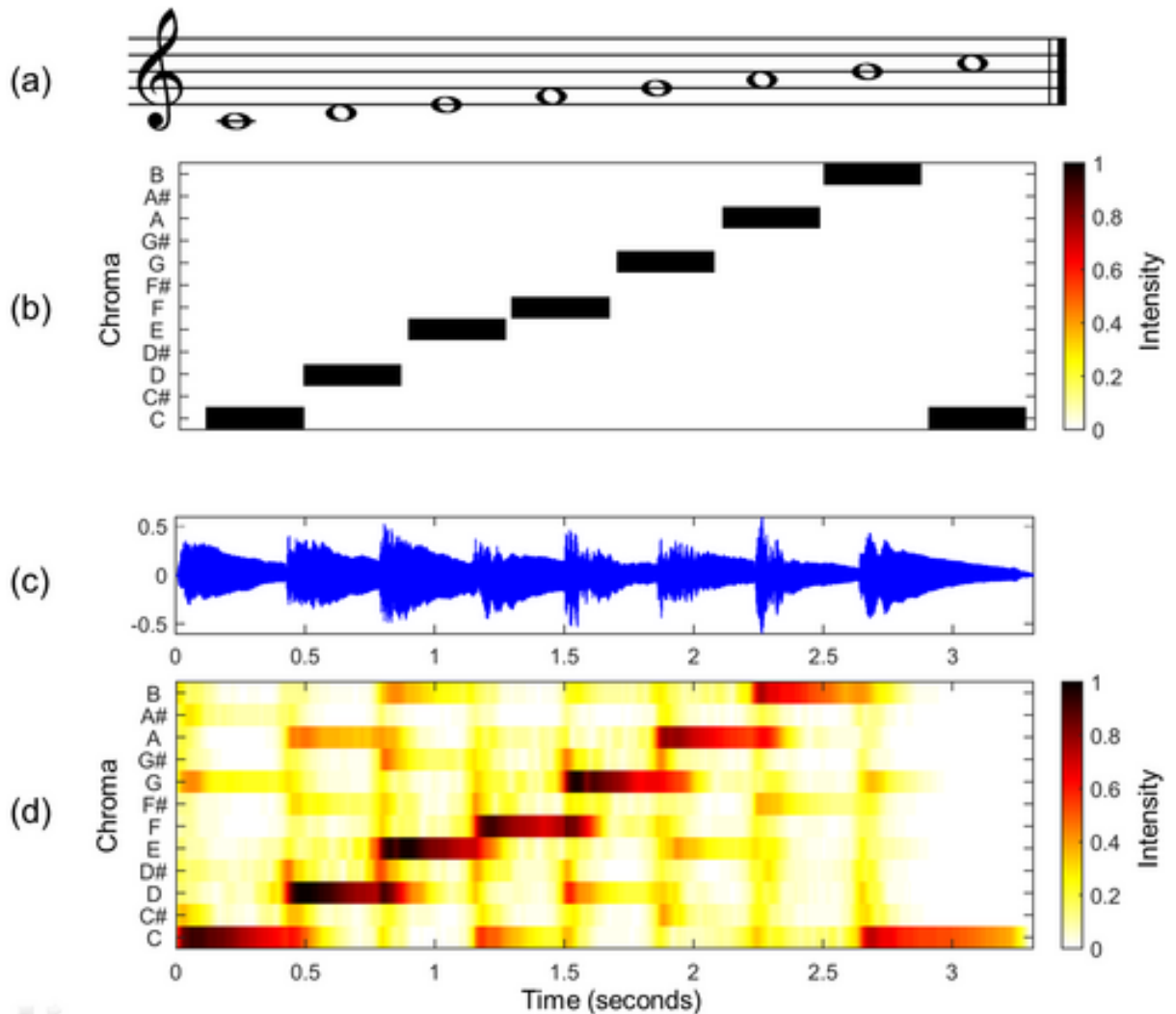


Figure 1.7: Chroma of chords

(a) Musical score of a C-major scale. (b) Chromagram obtained from the score. (c) Audio recording of the C-major scale played on a piano. (d) Chromagram obtained from the audio recording.

❖ Approaches to the Problem

Traditional approaches to key chord identification typically begin with symbolic or audio preprocessing. For symbolic data (e.g., MIDI), key detection often involves counting pitch class occurrences and comparing them to predefined key profiles such as those proposed by Krumhansl and Kessler. For audio signals, methods often rely on chroma feature extraction—representing the intensity of each of the 12 semitones over time (Dixon, 2010).

In early signal-based systems, chroma features were extracted from the spectrogram using techniques such as Short-Time Fourier Transform (STFT) and Constant-Q Transform (CQT) (al., 2016). These features are then aggregated and analyzed using template matching, statistical models, or rule-based systems to infer the most likely key and tonic chord.

More recent approaches apply deep learning models to automatically learn representations of harmonic content. Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs) (al G. e., 2016), and Transformer-based models are commonly used to model temporal harmonic dependencies. These models are typically trained on large datasets annotated with chord labels and key information. Some systems combine chord estimation and key detection as a joint task, leveraging the interdependence between chord functions and tonality.

Another approach involves framing the task as a classification problem, where the model predicts one of the 24 standard keys (12 major and 12 minor), or directly identifies the tonic chord based on the harmonic context derived from chroma-like input representations.

❖ **Proposed Solution for Key Chord Identification**

The core objective of the key chord identification task is to determine the tonic chord of a musical piece from an audio signal. Given an input audio waveform $x(t)$, the goal is to extract features that capture harmonic information and accurately identify the tonic chord C_{tonic} such that it best explains the global or local harmonic structure of the track.

This task can be addressed using a supervised learning framework where a model is trained to classify segments of audio into tonic chord classes. A common pipeline includes:

- **Preprocessing:** Convert audio into time-frequency representations such as CQT or chromagrams.
- **Feature Extraction:** Extract harmonic features that are invariant to timbre and instrumentation.
- **Model Architecture:** Use a CRNN or Transformer-based model that can learn spatial and temporal harmonic patterns.
- **Output:** Predict the tonic chord label or probability distribution over possible tonic chords.

Evaluation metrics for key chord identification include accuracy, precision, and recall against ground truth chord annotations. In practice, hierarchical evaluation may

also be employed, where near-miss predictions (e.g., predicting the dominant instead of the tonic) are penalized less severely than unrelated predictions.

1.11. Overview of the vocal separation solution

❖ Introduction and Challenges

In the field of music information retrieval, separating vocals from instrumental components in a mixed audio track is a fundamental yet challenging task. This problem, commonly referred to as vocal separation or source separation, has numerous applications in karaoke systems, music remixing, music analysis, and audio indexing. The main challenge lies in the complex overlap of frequencies between vocals and instruments, as well as the diverse range of vocal timbres and musical styles. Moreover, real-world audio often contains noise and reverberations, making the task even more difficult.

❖ Approaches to the Problem

Over the years, various approaches have been developed to tackle the vocal separation problem. Early methods relied on signal processing techniques such as Non-negative Matrix Factorization (NMF), Independent Component Analysis (ICA), and spectrogram masking. While these traditional methods offered some success, they often struggled with generalization and high-quality separation.

One of the most common and effective approaches to vocal separation involves transforming the audio signal into the time-frequency domain using the Short-Time Fourier Transform (STFT). This transformation represents the audio as a spectrogram, which makes it easier to analyze and distinguish overlapping sources like vocals and instruments based on their spectral patterns.

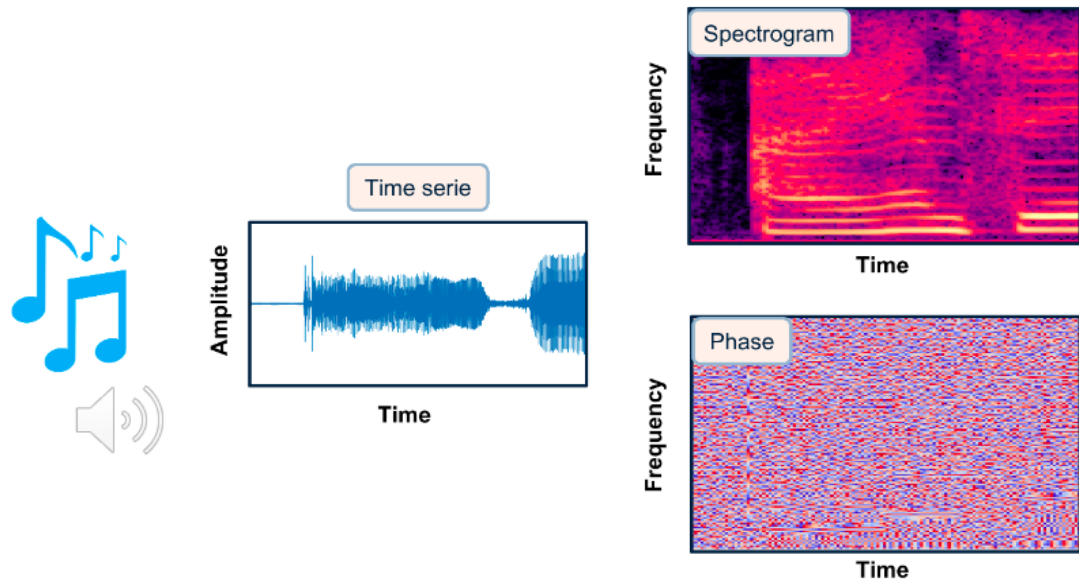


Figure 1.8: STFT spectrogram of audio signal

Once in the spectrogram domain, deep learning models are employed to learn the mapping from the mixed spectrogram to separate vocal and instrumental spectrograms. Typical models include Convolutional Neural Networks (CNNs), U-Net architectures (Tal Remez, 6 Jan 2017), and more recently, Transformer-based and attention-based models. These models are trained on large datasets containing pairs of mixed and clean source audio, allowing them to learn complex features and patterns that distinguish vocals from instruments.

A common technique is to predict a time-frequency mask, which is then applied to the input spectrogram to filter out either the vocal or instrumental components. After masking, the Inverse STFT (ISTFT) is applied to convert the separated spectrograms back into time-domain audio signals.

This STFT + deep learning pipeline has become the backbone of many state-of-the-art vocal separation systems due to its ability to preserve both temporal and frequency characteristics of audio signals while leveraging the power of data-driven learning.

❖ Proposed Solution for Vocals from Instruments

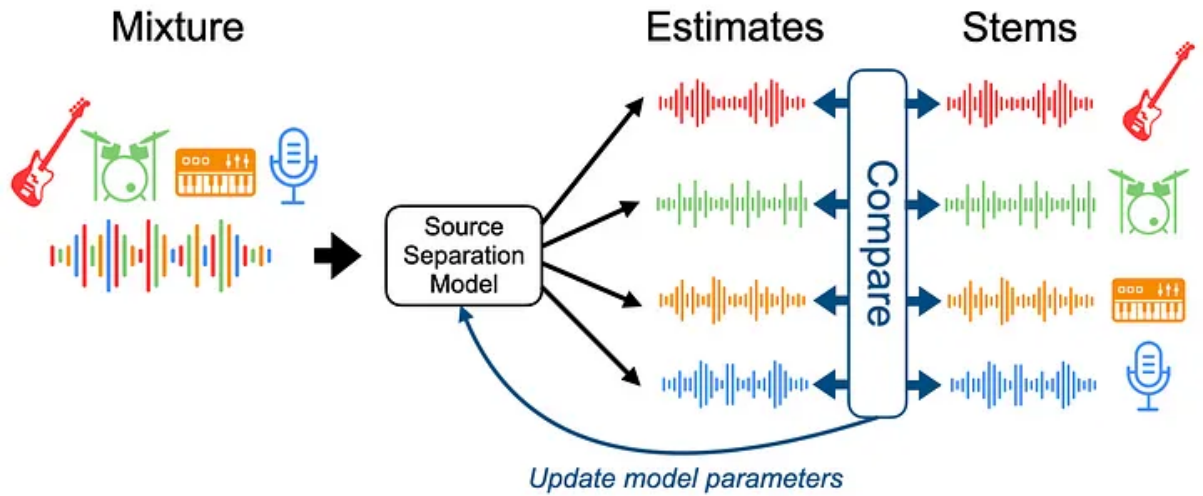


Figure 1.9: Process of separating vocals from song

The core problem addressed in vocal separation is to isolate the vocal signal from a music mixture, which typically consists of vocals and one or more instrumental accompaniments. Formally, given an input audio signal $x(t) \approx \hat{v}(t) + \hat{i}(t)$, where $v(t)$ is the vocal component and $i(t)$ is the instrumental component, the goal is to accurately estimate $\hat{v}(t)$. This problem can be framed as a supervised learning task where a model learns to predict a time-frequency mask or the clean sources directly from the mixed spectrogram. The performance of such systems is often evaluated using metrics such as Signal-to-Distortion Ratio (SDR), Signal-to-Interference Ratio (SIR), and Signal-to-Artifacts Ratio (SAR) (Choi, 2021).

Our chosen solution is based on a U-Net architecture, a type of convolutional neural network that is well-suited for image-like data such as spectrograms. The U-Net is designed with an encoder-decoder structure that captures both high-level context and fine-grained details. It is capable of predicting a spectral mask that highlights the vocal or instrumental components within the input spectrogram.

1.9. AWS



Figure 1.10: Amazon Web Services

Amazon Web Services (AWS) is the most comprehensive and widely adopted cloud platform in the world, offering over 200 fully featured services from global data centers. AWS allows individuals and organizations to build infrastructure, store and process data, distribute content, and manage applications without having to handle physical hardware.

❖ EC2 (Elastic Compute Cloud)

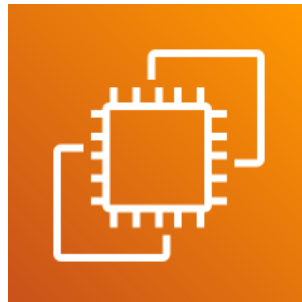


Figure 1.11: EC2

Amazon EC2 is a service that provides scalable virtual servers (instances) in the cloud. Users can launch and configure virtual machines with specific operating systems, CPU, RAM, and storage to deploy web applications, backend services, or data processing systems.

❖ Lambda (Serverless Computing)



Figure 1.12: Lambda

AWS Lambda is a serverless compute service that allows users to run code without provisioning or managing servers. Developers simply upload their code and define an event trigger (e.g., a file uploaded to S3 or an HTTP request), and Lambda automatically executes the code in response to those events.

❖ **S3 (Simple Storage Service)**



Figure 1.13: S3

Amazon S3 is an object storage service that enables users to store and retrieve data such as images, videos, documents, and system backups. Data is stored as “objects” within “buckets” and can be managed easily via the AWS Console, APIs, or SDKs.

❖ **CloudFront**



Figure 1.14: Cloudfront

Amazon CloudFront is a Content Delivery Network (CDN) service that speeds up the delivery of static and dynamic content by serving it through a global network of edge locations. It is commonly used to deliver images, videos, and web content hosted on services like S3.

❖ **ACM (AWS Certificate Manager)**



Figure 1.15: ACM

AWS Certificate Manager (ACM) provides free SSL/TLS certificates for securing HTTPS traffic. In this project, ACM is used to enable secure connections for custom domains such as <https://api.harmonyseeker.site>, ensuring encrypted communication and trust.

❖ **CloudWatch Logs (Monitoring and Logging)**



Figure 1.16: CloudWatch Logs

Amazon CloudWatch Logs collects and stores logs from EC2, Lambda, and other AWS resources. It is used in this project to:

- Monitor backend inference activity
- Track error rates or performance bottlenecks
- Send alerts or messages (e.g., to Google Chat) in case of anomalies

❖ **Elastic IP**

An Elastic IP is a static, public IPv4 address assigned to an EC2 instance. It ensures that the backend API remains reachable via a fixed address even after instance restarts. This is especially useful for configuring DNS records and domain bindings with services like api.harmonyseeker.site.

1.10. Conclusion

This chapter has presented the foundational knowledge and technologies used in the project, including JavaScript, VueJS, FastAPI, HTTP API, digital audio signal processing, music theory and tonic chords, the tonic chord identification problem, vocal separation, and AWS services. Understanding these concepts plays an important role in selecting the right tools and frameworks for building and deploying the system.

CHAPTER 2: SYSTEM ANALYSIS AND DESIGN

2.1 Requirement analysis

2.1.1 Functional requirements

1. Chord Recognition

- The system accepts input in the form of audio files (MP3, WAV).
- It analyzes the audio signal to detect the tonic chord and chord progression of the song.
- The recognized chords are displayed in a clear format, suitable for music analysis or instrumental accompaniment.

2. Vocal Removal

- Users can remove vocals from the original audio track.
- The system outputs separate files for vocals and instrumental backing.
- This function improves the accuracy of chord recognition from the accompaniment.

3. User Interface and Upload Features

- A web-based interface allows users to:
- Upload audio files.
- View chord analysis results.
- Supports common audio formats (MP3, WAV).

4. Monitoring and Alert Integration

- Integrates CloudWatchlog and Google chat for real-time system performance monitoring.
- Sends alerts in case of errors or performance degradation.

2.1.2 Non-functional requirements

❖ Performance

- The system should respond to chord recognition requests within an average of 5–10 seconds for typical audio inputs (under 1 minute).
- Audio processing time must remain consistent and scalable with longer audio files, ideally maintaining linear scalability.

❖ Scalability

- The architecture must support concurrent requests from multiple users without performance degradation.

- The system should allow future integration with other music-related features or APIs without major redesign.

❖ **Availability & Reliability**

- The system must maintain at least 99% uptime during deployment and usage.
- Fault-tolerant mechanisms should be implemented to recover from unexpected crashes or failures.
- Background services (e.g., vocal remover, chord analysis) should queue and retry on failure automatically.

❖ **Maintainability**

- The codebase must follow clean architecture and modular design principles to ensure ease of maintenance and future development.
- Developers should be able to update models, APIs, or UI independently through version control and CI/CD pipelines.

❖ **Usability**

- The user interface should be intuitive, with clear instructions for uploading files.
- Results must be presented in a user-friendly and interpretable format, especially for users with limited music theory background.

❖ **Portability**

- The system should be deployable on various environments including local servers, cloud platforms (e.g., AWS), or containerized systems using Docker.
- All services must run independently using container orchestration where applicable.

❖ **Monitoring and Logging**

The system must log key actions (e.g., uploads, predictions, errors) for analysis and debugging.

2.2 System design

2.2.1 Use case diagram

❖ **Actors in the System**

Table 2.1: Actor in the system

Actor	Description
User	Extract Vocal Track, Recognize Tonic Chord

❖ **Overview use case diagram**

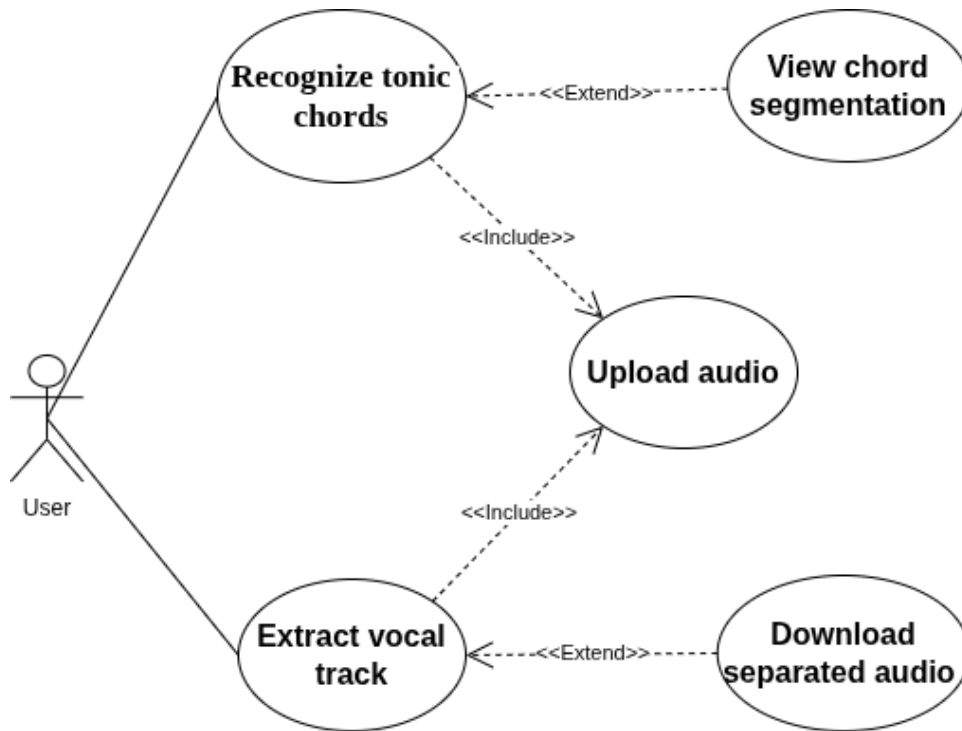


Figure 2.1: Overview use case diagram

2.2.2 Use case specification

❖ Use case specification for recognizing the tonic chord

Table 2.2: Use case specification for recognize tonic chord

Use Case Name	Recognize Tonic Chord
Use Case ID	UC-01
Extended Use Cases	UC-03 viewing chord segmentation
Actor	User (singer, musician, music teacher)
Description	The system analyzes the processed audio signal (after vocal extraction) to detect the tonic chord of the song.
Trigger	The user uploads an audio file. Click the button “Detect key chord”
Pre-condition	The audio file has been successfully uploaded and the vocal has been separated from the background music.
Post-condition(s)	The tonic chord of the song is returned and displayed to the user
Basic Flow	1. User uploads or selects an audio/video file. 2. System extracts the vocal from the music using a vocal remover.

	3. AI model processes the audio signal. 4. System detects the tonic chord from the input. 5. The tonic chord result is returned and shown on the user interface.
Alternative Flow	If the AI model is uncertain about the tonic chord: → The system displays a list of possible chords and asks the user to confirm.
Exception Flow	File Upload Failure: The system fails to upload the audio/video file due to unsupported format or file corruption. → The system displays an error message: "Unsupported file format or file corrupted. Please upload a valid MP3/WAV."

❖ **Use case specification for extracting the vocal track**

Table 2.3: Use case specification for extract vocal track

Use Case Name	Extract Vocal Track
Use Case ID	UC-02
Extended Use Cases	UC-04 downloading separated vocals and accompaniment
Actor	User (singer, musician, music producer)
Description	The system extracts the vocal component from an input audio or video file, separating it from the background music. This allows for further processing such as chord recognition, pitch correction, or vocal analysis.
Trigger	The user uploads an audio/video file and selects the "Separate" button.
Pre-condition	The audio or video file must be in a supported format (e.g., MP3, WAV, MP4) and be successfully uploaded.
Post-condition(s)	
Basic Flow	1. User uploads or selects an audio file. 2. System verifies the format and integrity of the file. 3. The vocal remover module processes the file to separate vocals.

	4. User can listen separator audio
Alternative Flow	User can play audio vocal after exact vocal and watch wave
Exception Flow	File Upload Failure: If the file format is not supported or the file is corrupted: → The system displays an error message: “Unsupported file format or corrupted file. Please upload a valid MP3/WAV/MP4.” Extraction Failure: If the AI fails to separate vocals: → The system shows: “Unable to extract vocals from this file. Try another or contact support.”

❖ **Use case specification for viewing chord segmentation**

Table 2.4: Use case specification for viewing chord segmentation

Use Case Name	Viewing Chord Segmentation Music
Use Case ID	UC-03
Extended Use Cases	UC-01 recognizing the tonic chord
Actor	User (singer, musician, music learner, music teacher)
Description	After the tonic chord has been identified, the system plays back the audio and displays the corresponding chord for each lyric line or music segment in real time. Users can visually follow along with the chord progression as the song plays.
Trigger	The user uploads an audio file. Click the button Detect key chord
Pre-condition	The tonic chord of the song has been successfully recognized (UC-01 completed).
Post-condition(s)	The audio is played along with segment-wise chords displayed in sync with playback. The user can clearly follow the chord progression visually as the song progresses.
Basic Flow	1. User uploads or selects an audio/video file.

	2. System extracts the vocal from the music using a vocal remover. 3. AI model processes the audio signal. 4. System detects the tonic chord from the input. 5. The tonic chord result of segment music is returned and shown on the user interface.
Alternative Flow	If the user wants to preview all chord segments before playback: → The system provides a timeline-based chord list for user reference
Exception Flow	File Upload Failure: If the file format is not supported or the file is corrupted: → The system displays an error message: “Unsupported file format or corrupted file. Please upload a valid MP3/WAV/MP4.” Extraction Failure: If the AI fails to separate vocals: → The system shows: “Unable to extract vocals from this file. Try another or contact support.”

❖ **Use case specification for downloading separated vocals and accompaniment**

Table 2.5: Use case specification for download separated audio

Use Case Name	Download Separated Audio
Use Case ID	UC-04
Extended Use Cases	UC-02 extracting the vocal track
Actor	User (singer, musician, music producer, teacher)
Description	After the vocal extraction process, the system provides download options for three types of audio files: the original mixed audio, the isolated vocal track, and the instrumental (background music) track.
Trigger	The user uploads an audio/video file and selects the “Separate” button.
Pre-condition	The audio or video file must be in a supported format (e.g., MP3, WAV, MP4) and be successfully uploaded.

Post-condition(s)	The selected audio file(s) are downloaded to the user's device. The system may log the download action or remove temporary files after completion.
Basic Flow	1. The user completes vocal extraction (UC-02). 2. The system displays download options for: - Mixed audio (original) - Vocal track - Instrumental track 3. The user selects one or more files to download. 4. The system prepares and initiates the download process. 5. The selected files are downloaded to the user's device.
Alternative Flow	If the user wants to preview files before downloading: → The system offers playback buttons for each file.
Exception Flow	File Not Available: If one or more files are missing or failed to generate: → The system displays an error: <i>"Requested file is unavailable. Please re-run vocal extraction or contact support."</i> Download Failure: If a network error occurs: → The system displays a retry option with a message: <i>"Download failed. Please check your connection and try again."</i>

2.2.3 System architecture

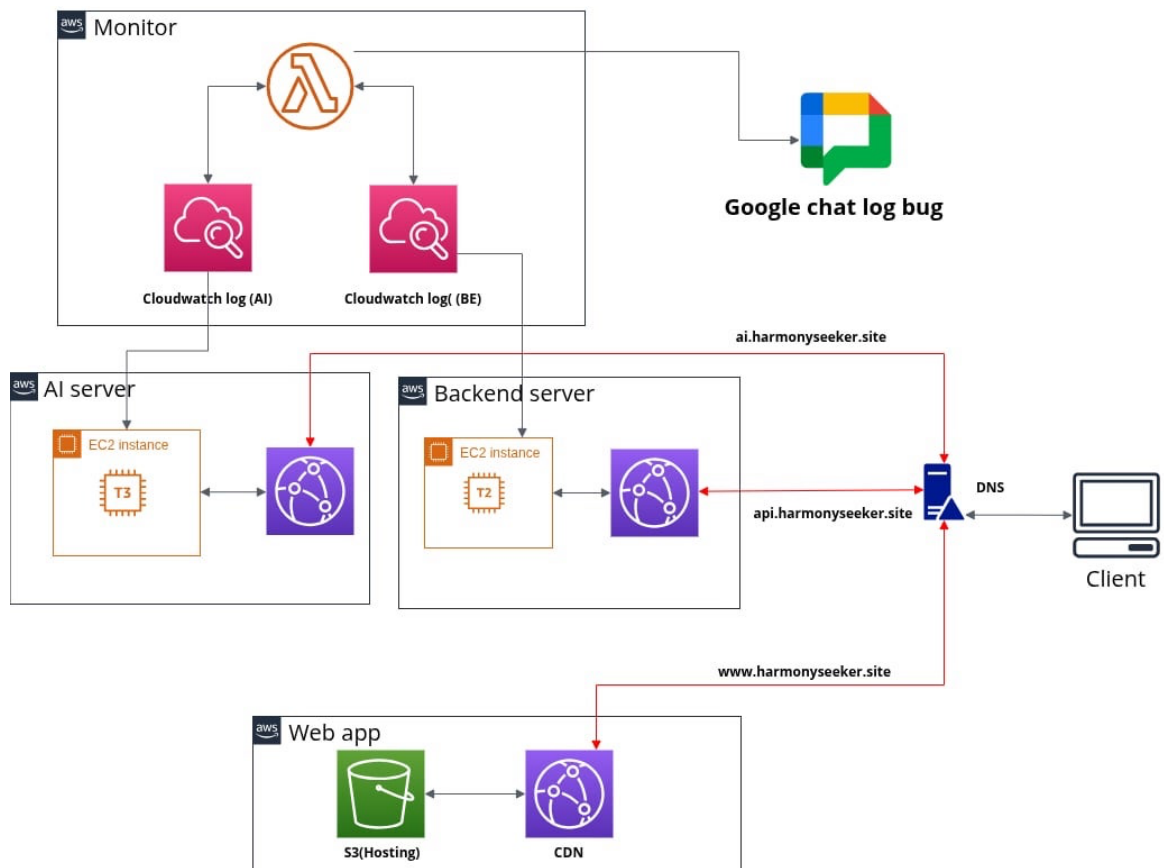


Figure 2.2: System architecture

The HarmonySeeker system is designed using a modular cloud-based architecture deployed on Amazon Web Services (AWS), integrating microservices, serverless components, and monitoring. The architecture ensures high availability, scalability, and maintainability, while supporting a responsive web frontend and AI-powered backend services. The overall infrastructure is illustrated in Figure 3.3.

A. Web Frontend (Static Application)

- Amazon S3 is used to host the static frontend web application, built with VueJS.
- Amazon CloudFront (CDN) is integrated with S3 to accelerate content delivery globally, reducing latency and improving performance.
- The web is accessible via the custom domain: <https://www.harmonyseeker.site>

B. Backend API Service

- A lightweight RESTful API service is deployed on an Amazon EC2 t2.medium instance using FastAPI (Python).

- This server handles user uploads, request validation, inference orchestration, and serves data for the frontend via: <https://api.harmonyseeker.site>
- API documentation is available via auto-generated OpenAPI docs.

C. AI Inference Microservice

- The AI models (CRNN for chord recognition, UNet for vocal separation) are deployed on a dedicated EC2 t3.xlarge instance with 16 GiB RAM and 4 vCPUs to ensure smooth model execution and low-latency inference (Kojs, 2023).
- This microservice is isolated from the main backend to allow independent scaling and resource allocation.
- The AI service exposes its own endpoint for internal communication, as well as public documentation at: <https://ai.harmonyseeker.site>

D. Monitoring and Logging

- AWS CloudWatch Logs is used to collect logs from both:
 - The backend server
 - The AI inference service
- A custom AWS Lambda function is configured to monitor these logs in near real-time.

On detecting predefined anomalies (e.g., crashes, timeouts, inference errors), the Lambda triggers alerts.

E. Alerting via Google Chat

- The monitoring Lambda integrates with Google Chat via Webhooks.
- Upon detecting issues (e.g., high error rate, server failures), logs and messages are forwarded to a dedicated Google Chat space for the development team.
- This enables real-time alerting and helps maintain system reliability.

F. Domain and DNS Configuration

- The custom domain harmonyseeker.site and its subdomains are managed via [Tenten.vn](https://tenten.vn).
- DNS records point to:
 - CloudFront for the frontend
 - EC2 public IPs for backend and AI servers
- HTTPS is enabled via **AWS Certificate Manager (ACM)**, ensuring secure communication across all endpoints.

2.2.4 Activity Diagram

2.2.4.1 Activity diagram for recognize tonic chord

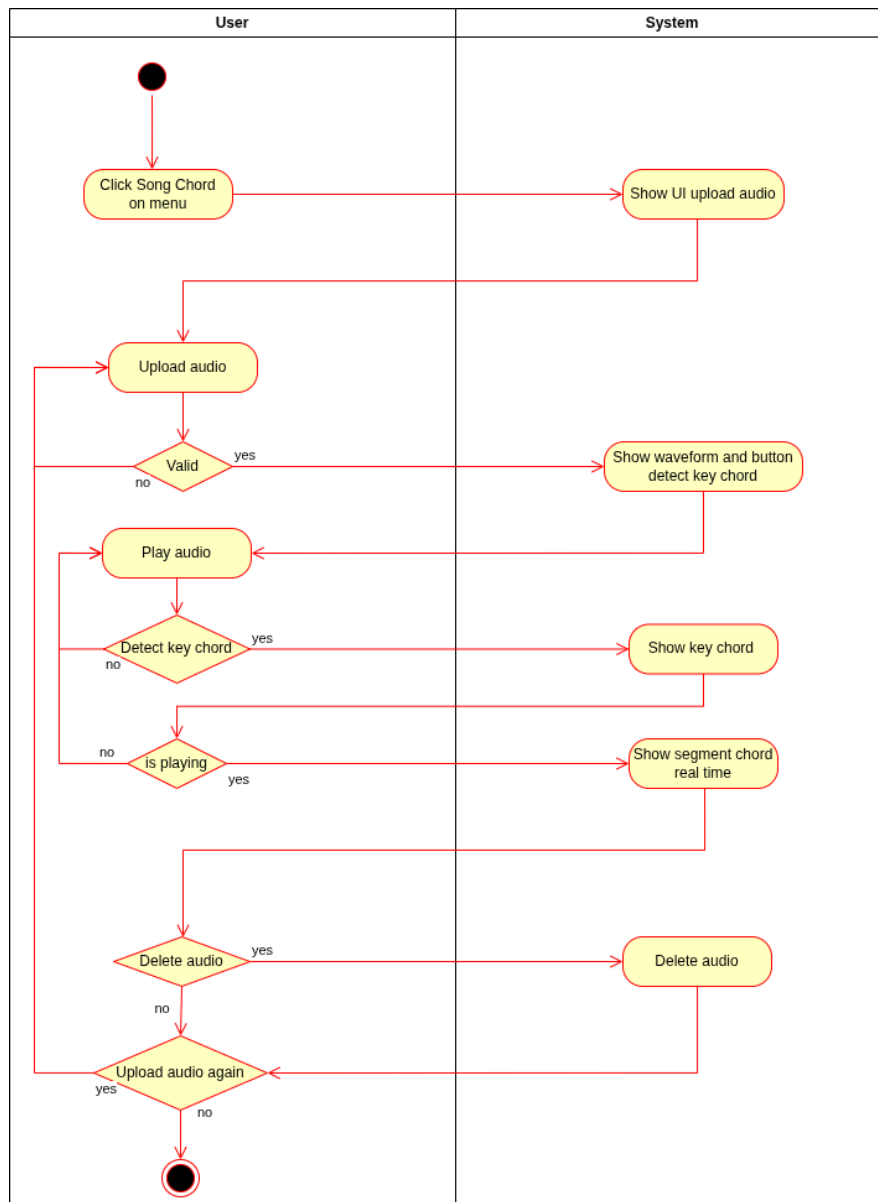


Figure 2.3: Activity diagram for recognize tonic chord

2.2.4.2 Activity diagram for extract vocal track

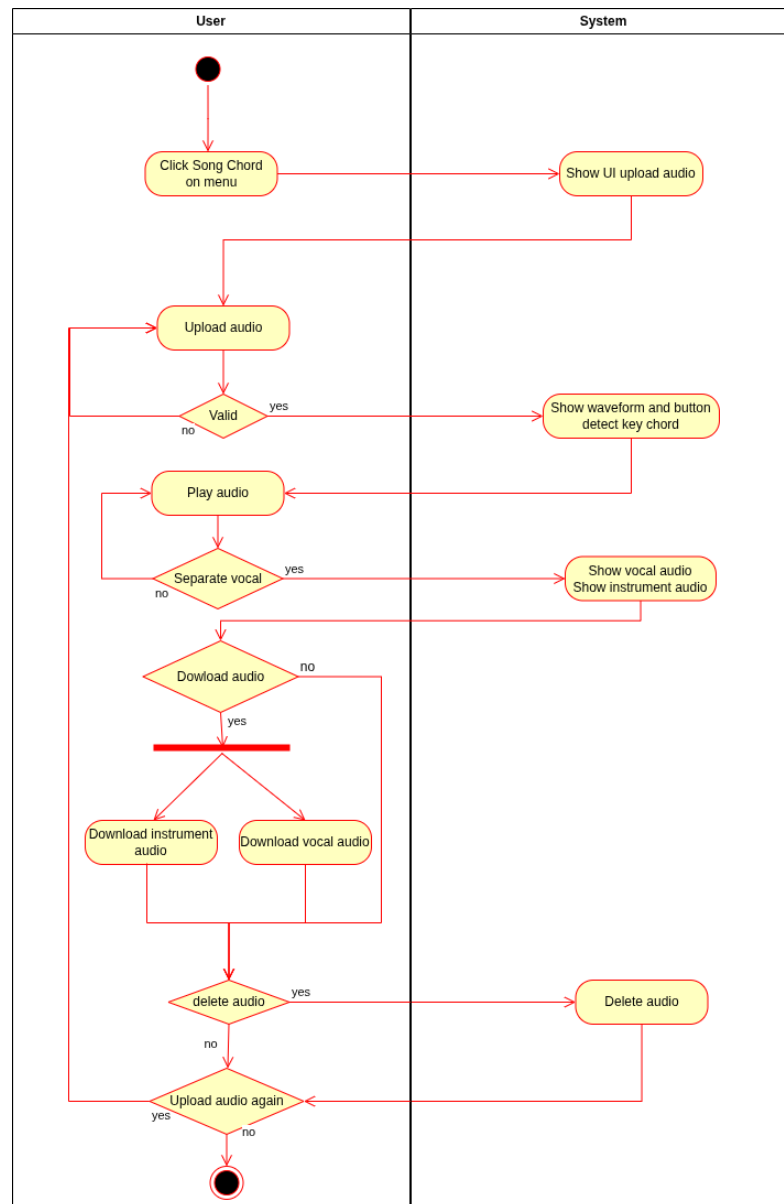


Figure 2.4: Activity diagram for extract vocal track

2.2.5 Sequence diagrams

2.2.5.1 Recognize tonic chord sequence diagram

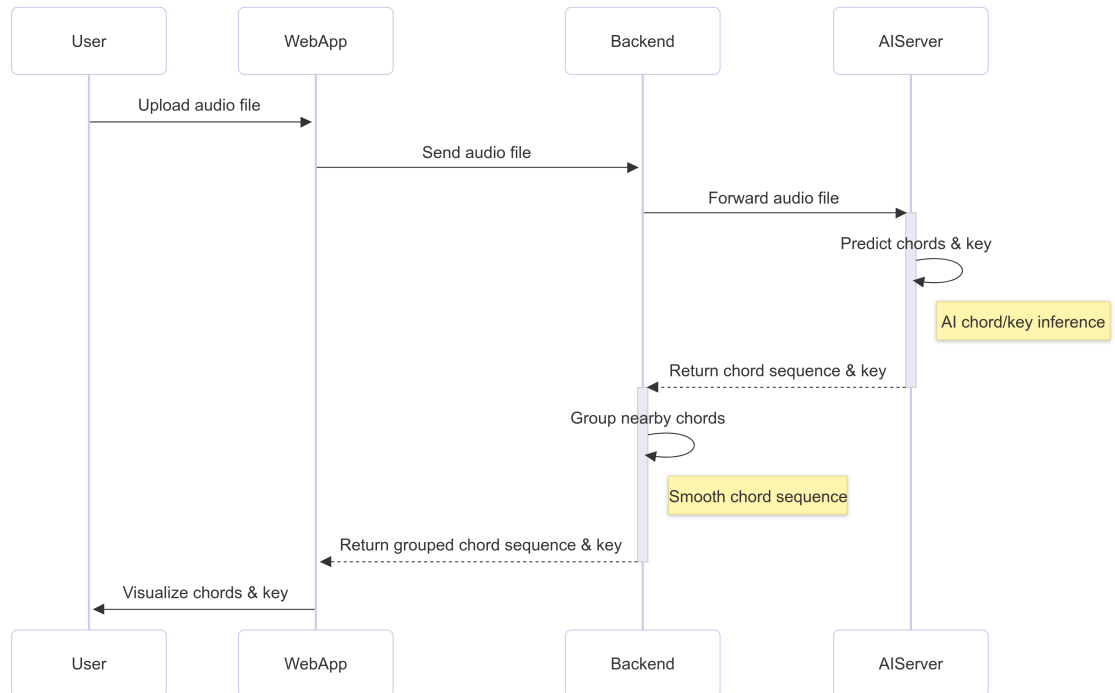


Figure 2.5: Recognize tonic chord sequence diagram

2.2.5.2 Extract vocal track sequence diagram

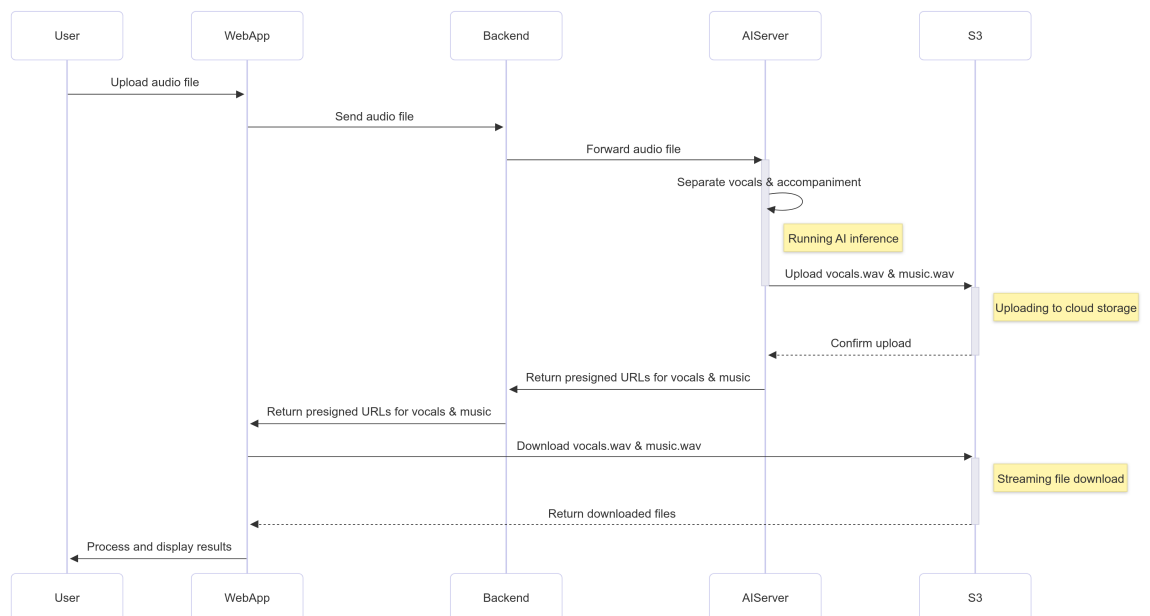


Figure 2.6: Extract vocal track sequence diagram

2.2.6 API specifications

❖ Detect tonic chord of audio API

Table 2.6: API specifications of detecting tonic chord of audio

Description	This API takes in an audio file and analyzes it to detect the tonic chord, main chord, and chord progression segments.
Endpoint	/api/predict-chord
HTTP Method	POST
Payload	multipart/form-data
Response	<pre>{ "key": "C", "main_chord": "C", "chord_segments": [{ "chord": "C", "start_time": 0.000, "end_time": 3.500 }, { "chord": "G", "start_time": 3.500, "end_time": 7.200 }, ...] }</pre>

POST /api/predict-chord Predict Chord

Receive an audio file, forward it to the AI server, then group the returned chord sequence into segments with the same consecutive chords, each with start_time and end_time.

Parameters

No parameters

Request body required multipart/form-data

file * required
string(binary) Choose File MushHong-LeHieu_8e.mp3

Execute **Clear**

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8001/api/predict-chord' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'file=@MushHong-LeHieu_8e.mp3;type=audio/mpeg'
```

Request URL

http://localhost:8001/api/predict-chord

Server response

Code **Details**

200

Response body

```
{
  "key": "C",
  "main_chord": "C",
  "chord_segments": [
    {
      "chord": "C",
      "start_time": 0,
      "end_time": 0.951
    },
    {
      "chord": "A:min",
      "start_time": 0.951,
      "end_time": 4.127
    },
    {
      "chord": "F",
      "start_time": 4.127,
      "end_time": 5.078
    },
    {
      "chord": "E:min",
      "start_time": 5.078,
      "end_time": 7.493
    }
  ]
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: http://localhost:8001
content-length: 1185
content-type: application/json
date: Wed, 18 Jun 2025 13:15:07 GMT
server: uvicorn
vary: Origin
```

Responses

Code	Description	Links
200	Successful Response	No links

Figure 2.7: API specifications of detecting tonic chord of audio

❖ Separate voice from audio API

Table 2.7: API specifications of Separating voice from audio

Description	This API separates the vocals and instrumental tracks from the instrumental/background music in an input audio file.
Endpoint	/api/ separate-voice
HTTP Method	POST
Payload	multipart/form-data
Response	{ "vocals_url": "https://example.com/vocals.wav",

	<pre>"music_url": "https://example.com/instrumental.wav", "expiration": "2029-07-01T12:00:00Z" }</pre>
--	--

POST /api/separate-voice Separate Voice

Receive an audio file, forward it to the AI server for voice separation, and return the S3 URLs for both the vocals and instrumental tracks.

Parameters

No parameters

Request body required

multipart/form-data

file * required

string(binary)

Choose File

file_min - Dấu tình sầu - Ngọc (mp3cut.net).wav

Execute

Clear

Responses

Curl

curl -X 'POST' \
'http://localhost:8001/api/separate-voice' \
-H 'accept: application/json' \
-H 'Content-Type: multipart/form-data' \
-F 'file=@file_min - Dấu tình sầu - Ngọc (mp3cut.net).wav;type=audio/wav'

Request URL

http://localhost:8001/api/separate-voice

Server response

Code

Details

200

Response body

{
 "vocals_url": "https://harmonyseeker-ai.s3.ap-southeast-1.amazonaws.com/vocals_2025-06-18T20-17-46.wav?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIASWC7WN2E2G7FKZK2F20250618K2Fap-southeast-1K2F3K2Faws4_request&X-Amz-Date=20250618T131749Z&X-Amz-Expires=3600&X-Amz-SignedHeaders=host&X-Amz-Signature=2f33192423202f0211af09051eac5d467dfe045f5a759f0f6e30a4042b092",
 "music_url": "https://harmonyseeker-ai.s3.ap-southeast-1.amazonaws.com/music_2025-06-18T20-17-46.wav?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIASWC7WN2E2G7FKZK2F20250618K2Fap-southeast-1K2F3K2Faws4_request&X-Amz-Date=20250618T131749Z&X-Amz-Expires=3600&X-Amz-SignedHeaders=host&X-Amz-Signature=fc46b1b881a94269c2bde5d8f129d354b51b925d49b-c5f18d283a1729e33727",
 "expiration": 3600
}

Response headers

access-control-allow-credentials: true
access-control-allow-origin: http://localhost:8001
content-length: 767
content-type: application/json
date: Wed, 18 Jun 2025 13:17:37 GMT
server: uvicorn
vary: Origin

Responses

Code

Description

Links

200

Successful Response

No links

Figure 2.8: API specifications of Separating voice from audio

2.3 Method to Identify the Tonic Chord & Chord Progression

2.3.1 Data Solutions

Data collection

In this study, we adopt the Isophonics dataset—an extensively utilized benchmark for chord recognition tasks—as our primary training source. The dataset contains 225 professionally annotated songs from renowned artists such as The Beatles, Queen, Carole King, and Zweieck. Each track is accompanied by time-aligned chord annotations provided in .lab format, specifying the onset and offset of every chord segment. Key information is also included, allowing for tasks such as tonic chord detection and key normalization.

Perfomed Student: Tran Duc Tri

Instructor: MSc. Do Thi Tuyet Hoa

38

The dataset's diversity in genre, harmonic structure, and compositional complexity makes it an ideal resource for training supervised deep learning models. Its widespread adoption in music information retrieval research further supports its reliability and comparability for benchmarking.

For model evaluation, we construct a custom Vietnamese chord dataset by crawling and processing 125 popular songs from publicly available sources such as hopamviet.vn and hopamchuan.com. Each song is aligned with its corresponding chord progression to generate frame-level annotations, following a similar structure to the Isophonics dataset. This test set provides an independent evaluation scenario that emphasizes real-world diversity and linguistic-cultural contrast in musical content, ensuring robust assessment of the model's generalization capacity.

In addition to the original version of these 125 test songs, we further process them using a vocal separation model (described in Section 2.4) to isolate the vocal tracks from the instrumental accompaniment. This yields a second test set containing vocal-only audio, which allows us to systematically examine the impact of source separation on chord recognition performance. By comparing model outputs on the full mix versus the vocal-isolated tracks, we aim to evaluate how the presence or absence of instrumental context affects frame-wise chord classification and tonic inference accuracy. This dual-evaluation strategy highlights the model's robustness under varying audio conditions and offers insight into the harmonic contribution of vocal versus instrumental components.

Data preprocessing

To facilitate effective learning, we implement a structured preprocessing pipeline that converts raw audio into harmonically meaningful time-frequency representations, aligned with corresponding chord labels. The pipeline includes the following stages:

Feature Extraction

The primary representation used in our system is the Constant-Q Transform (CQT) spectrogram, which offers logarithmic frequency spacing aligned with the pitch scale of Western music. Unlike linear spectrograms, CQT more accurately reflects the harmonic content of music—especially in the lower frequency regions where chord roots often reside—making it particularly suitable for chord recognition tasks (Brown, 1991).

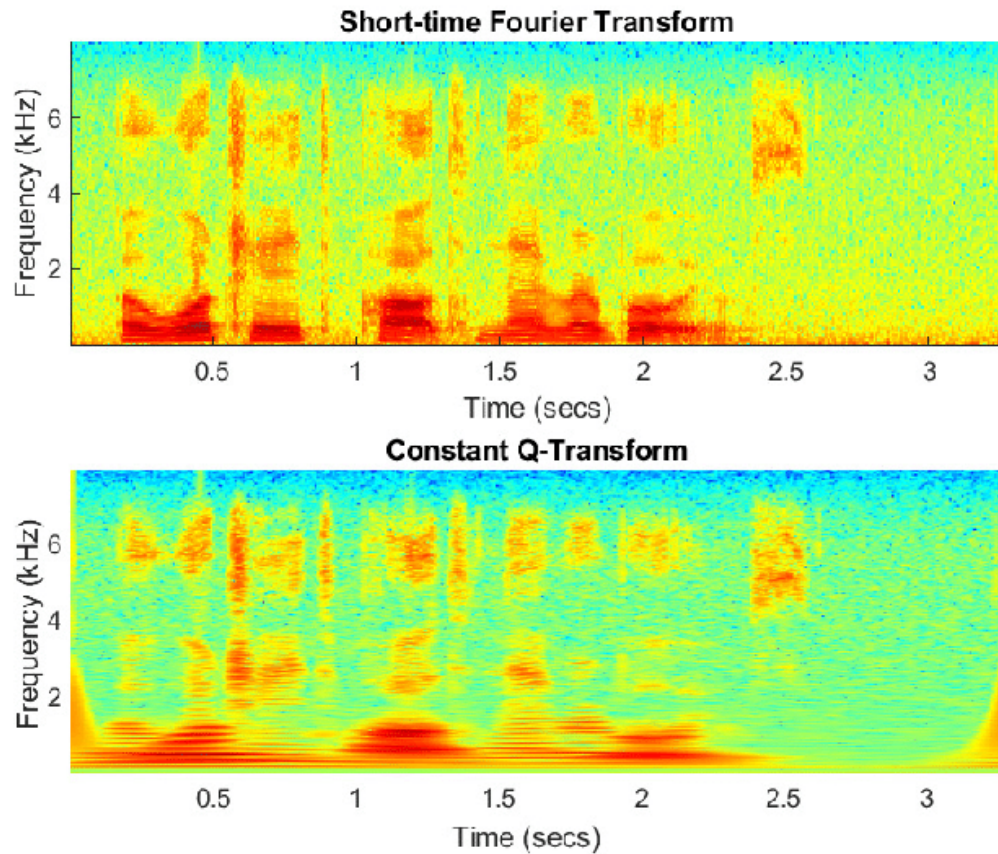


Figure 2.9: Constant Q-Transform spectrogram

Figure 25: Visual comparison between Short-Time Fourier Transform (STFT) and Constant-Q Transform (CQT) representations of the same audio excerpt. The CQT provides improved resolution in lower frequency regions, making it more suitable for harmonic content analysis such as chord recognition.

The following table summarizes the key parameters used in CQT feature extraction:

Table 2.8: CQT parameters

Parameter	Value
Sample rate	22,050 Hz (CRNN configuration)
FFT size	16,384 (2^{14})
Hop length	512 samples (~23.2 ms)
Number of CQT bins	252
Bins per octave	36
Starting note	C1

This configuration yields a high-resolution, pitch-aligned time-frequency representation that preserves tonal structure while maintaining computational feasibility.

Sequence Construction

The CQT spectrogram is segmented into overlapping sequences, each consisting of 1,000 consecutive frames, corresponding to approximately 23 seconds of audio at the specified hop length. This window size allows the model to capture short-term transitions as well as longer harmonic progressions across musical phrases. Zero-padding is applied to trailing segments as needed to ensure consistent input dimensionality across all sequences.

Label Alignment

Each CQT frame is annotated with a chord label derived from the timestamp-aligned annotation files. Labels are mapped to one of 24 chord classes (12 major and 12 minor), and encoded as integer indices to facilitate supervised learning. This frame-level alignment allows the system to support both chord sequence modeling and tonic chord classification.

Data Augmentation

To improve model robustness and reduce overfitting, several data augmentation strategies are employed:

- Time shifting: Randomizing the starting frame of each sequence introduces variability in harmonic context.
- Key normalization (optional): Transposing all chord labels and corresponding features to a common root (e.g., C major) enhances key invariance.
- Contextual windowing: Additional surrounding frames may be concatenated to reinforce temporal continuity in chord progression patterns.

Storage Optimization

All processed features and aligned chord labels are serialized and stored in binary .ds files, ensuring fast disk access and minimal I/O latency during training. This format is optimized for batch loading on GPU-based training pipelines.

Table 2.9: Parameter Rationale

Design Choice	Justification
Sample Rate (22,050 Hz)	Provides adequate frequency resolution for harmonic analysis while significantly reducing computational cost compared to 44.1 kHz.
Hop Length (512 samples)	Offers ~23 ms temporal resolution, balancing time granularity and processing efficiency—ideal for capturing chord transitions.
FFT Size (16,384 samples)	Supports high frequency resolution in CQT computation, critical for resolving closely spaced harmonic components.
CQT with 252 Bins and 36 Bins/Octave	Aligns with the equal-tempered musical scale, ensuring that pitch-related structures such as chords and keys are well represented.
Starting Frequency (C1)	Captures low-range harmonic content effectively, important for modeling chord roots and bass-driven harmony.
Sequence Length (1,000 frames)	Encompasses a wide temporal context (~23 seconds), enabling the model to learn complex chord progression dynamics and long-range dependencies.

This preprocessing pipeline produces harmonically rich and temporally structured inputs suitable for deep learning models. It ensures that the CNN and CRNN architectures receive data that aligns with the musical logic of chord progression, ultimately supporting accurate identification of tonic chords and detailed harmonic analysis.

2.3.2 Evaluation method and loss function

Loss Function: Sparse Categorical Crossentropy

During training, the chord recognition model operates on short audio frames of approximately 23 milliseconds, extracted from continuous music signals. Each frame is treated as an independent classification instance and is labeled with its corresponding chord class from a predefined vocabulary (e.g., 24 classes: 12 major and 12 minor chords).

The Sparse Categorical Crossentropy loss function is used to guide the model's learning process. This loss is appropriate for multi-class classification problems with integer labels, and it measures the discrepancy between the predicted probability distribution and the true class label for each frame.

The loss is formally defined as:

$$L_{CE} = - \sum_{i=1}^n t_i \log(p_i)$$

Where:

- t_i : Ground-truth indicator (1 for correct class, 0 otherwise)
- p_i : Predicted probability assigned to class i by softmax output
- n : Number of classes or total outcomes in the classification task

By minimizing this loss across all frames, the model is trained to assign high probability to the correct chord class at each time step. This frame-level learning approach enables the model to capture both local harmonic features and long-range chord transitions across the duration of the musical piece.

Evaluation Metrics

To assess the model's performance, a combination of frame-level accuracy and musically informed evaluation metrics is used. These metrics help quantify not just raw classification accuracy, but also how well the model understands chord structure and progression within the context of a full song.

a. Frame-wise Accuracy

This metric computes the percentage of audio frames for which the model correctly predicts the chord label. It is a simple yet effective measure to monitor model performance during training and validation, though it treats all frames equally regardless of chord duration or musical context.

b. mir_eval-based Chord Metrics.

These metrics provide deeper insight into the harmonic correctness and segmentation quality of the predicted chord sequences:

- **Weighted Chord Recognition Accuracy (WCRA):**
Measures the duration-weighted proportion of correctly recognized chords, assigning greater importance to longer chord segments.
- **Triads Accuracy:**
Evaluates whether the predicted chord captures the correct root, third, and fifth, while ignoring inversions and extensions.
- **MIREX Score:**
Based on the Music Information Retrieval Evaluation eXchange framework, this metric assesses the correctness of predicted chord root and quality, tolerating minor label variations.
- **Undersegmentation:**
Penalizes predictions that fail to capture all chord transitions, resulting in overly long segments.
- **Oversegmentation:**
Penalizes predictions that introduce unnecessary chord changes by splitting single chords into multiple segments.
- **Segmentation Accuracy (Seg):**
Measures the alignment between predicted and ground-truth chord boundaries, incorporating both under- and oversegmentation errors.

c. Weighted Chord Symbol Recall (WCSR)

This custom metric, inspired by MIREX 2013, calculates the ratio of correctly predicted chord segments to the total number of chord segments, regardless of duration. It emphasizes the sequence correctness of chord predictions and complements the frame-wise metrics.

Tonic Chord Recognition Evaluation

In addition to evaluating detailed frame-wise chord sequences, an important aspect of harmonic analysis is the model's ability to identify the tonic chord or key of an entire musical piece.

For each song in the test set, a ground-truth tonic chord is defined in advance. After the model performs frame-level chord recognition, a post-processing algorithm is applied to infer the tonic chord of the song by analyzing the distribution and progression of predicted chords.

The tonic chord recognition performance is then evaluated by comparing the inferred tonic chord to the ground-truth label. Evaluation metrics include:

- **Accuracy:** The proportion of songs for which the inferred tonic chord matches the true tonic.
- **Confusion Matrix:** A detailed visualization of how tonic chords are misclassified, helping identify consistent tonal ambiguities (e.g., confusing C major with A minor).

This evaluation component provides insight into the model's ability to capture the global harmonic structure of a song, beyond localized chord changes. It is especially valuable for applications such as music retrieval, tonal analysis, and key detection.

By combining Sparse Categorical Crossentropy for frame-level training with a diverse set of harmonic evaluation metrics and tonic chord analysis, the system provides a holistic assessment of model performance. This enables both granular chord classification and broader harmonic understanding across complete musical works.

2.3.3 CNN Model Architecture for Identifying the Tonic Chord

Convolutional Neural Networks (CNNs) are effective architectures for tasks involving time–frequency representations like spectrograms. In this system, a CNN is used to classify each audio frame into a specific chord class based on localized harmonic features present in Constant-Q Transform (CQT) spectrograms. Although limited in capturing long-term temporal dependencies, CNNs provide an efficient and interpretable baseline model for frame-wise chord classification (al C. e., 2016).

Suitability of CNNs for Chord Recognition

CNNs apply shared convolutional kernels over time–frequency representations, making them ideal for spectrogram inputs where harmonically structured patterns exist. In music, chords manifest as vertical energy clusters across frequency bands over short temporal spans, allowing CNNs to detect:

- Chord roots and harmonic overtones
- Triadic structures (root–third–fifth)
- Voice-leading consistency

Advantages

- Efficiently captures localized spectral patterns in CQT.
- Fast inference with relatively few parameters.
- Easily parallelized on GPU hardware.

Limitations

- Lacks recurrent memory — predictions are independent across time.
- Cannot inherently model long-term musical context or harmonic progression.
- Suitable as a baseline, but superseded by CRNN in modeling full musical structure.

CNN Model Architecture Overview

The CNN model consists of a shallow yet powerful architecture composed of three convolutional layers followed by fully connected layers for classification. The input to the model is a time–frequency slice of the song represented as a 2D CQT spectrogram.

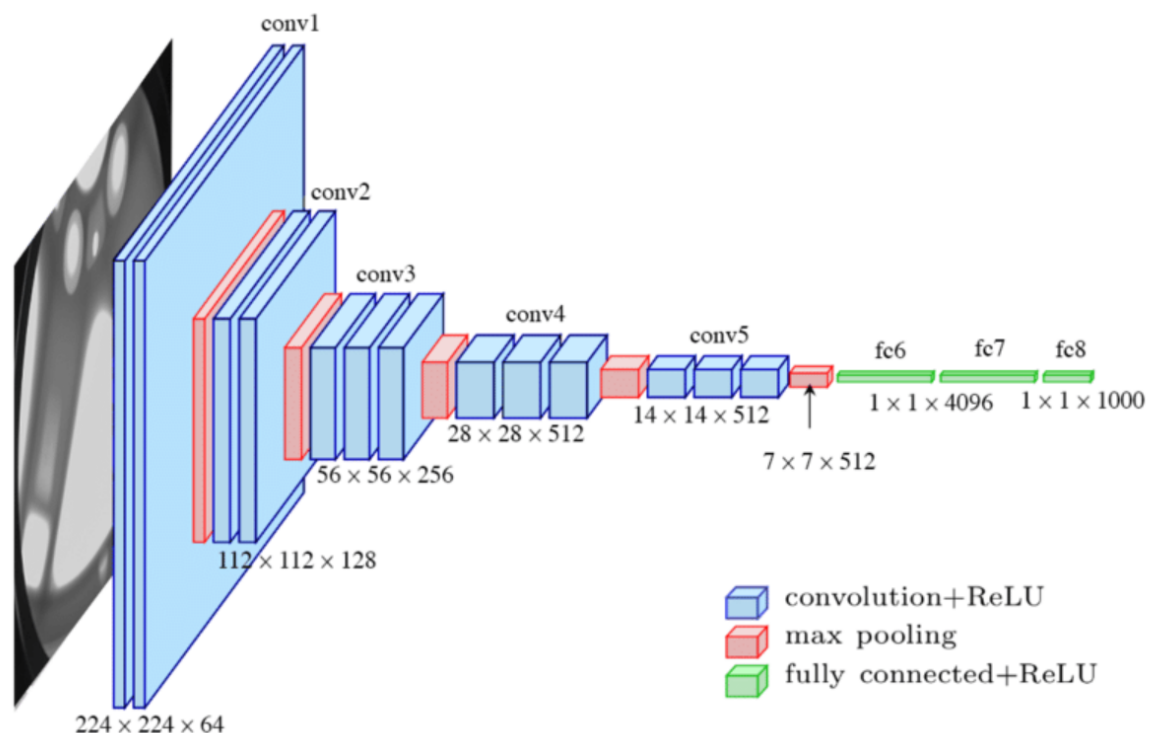


Figure 2.10: CNN Model Architecture

Input Shape:

$$X \in R^{1000 \times 252 \times 1}$$

where:

- 1000 = number of time frames (~23.2 seconds),
- 252 = number of frequency bins (CQT resolution),
- 1 = single-channel input (magnitude spectrogram).

This shape represents a segment of musical audio in time–frequency space, suitable for convolutional processing.

Feature Extraction Layers

CNN learns increasingly abstract harmonic patterns through stacked convolutional layers. Each layer applies local filters that detect characteristic spectral patterns associated with chords.

- **Conv2D-1:**

$$F^{(1)} = \text{ReLU}(X * K^{(1)} + b^{(1)})$$

where:

- $*$ is 2D convolution,
- $K^{(1)}$: 3×3 filter bank (32 filters),
- $b^{(1)}$: bias,
- $F^{(1)}$: output feature map.

This layer captures basic harmonic edges and time–frequency contours.

- **MaxPooling-1:**

Down samples the feature maps by a factor of 2 in each dimension to reduce spatial size:

$$P^{(1)} = \max_{(2 \times 2)} F^{(1)}$$

reserving dominant features while reducing computation.

- **Conv2D-2** and **Conv2D-3:** These deeper layers capture increasingly complex patterns such as chordal textures and harmonic stacks across multiple frequency bins. The outputs are similarly activated with ReLU and pooled to form a compressed representation.

Classifier

After convolutional processing, the feature maps are flattened into a dense vector for classification.

- **Flatten:**

$$x_{flat} = \text{Flatten}(F^{(3)})$$

where $F^{(3)}$ is the final convolutional output tensor.

- **Dense Layer:**

$$h = \text{ReLU}(W_1 \cdot x_{flat} + b_1)$$

where $W_1 \in R^{d \times \mathbb{Q}}$, d is the input size, and h is a compressed high-level harmonic descriptor.

- **Output Layer (Softmax):**

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}, \quad i = 1, \dots, C$$

where:

- $C = 25$ (24 chord classes + 1 for "No Chord"),
- z_i : logit for class i ,
- $\hat{y}_i$: predicted probability for class i

Loss Function and Optimization

- **Loss Function:**

The model is trained using Sparse Categorical Crossentropy, suitable for integer-labeled chord classes.

$$L_{CE} = -\log(\hat{y}_c), \quad c = \text{ground-truth class index}$$

This loss encourages the model to assign a high probability to the correct chord class at each frame.

- **Optimizer:**

Adam optimizer is used with learning rate:

$$\alpha = 0.001$$

to balance fast convergence with numerical stability.

Preprocessing

Raw audio waveforms are transformed into CQT spectrograms using the Constant-Q Transform, providing logarithmic frequency scaling aligned with musical perception.

Each spectrogram is then segmented into overlapping slices:

$$\text{Segment length} = 1000 \text{ frames} \approx 23.2 \text{ seconds}$$

Model Inference

Each CQT segment is passed through the trained CNN model, which outputs a probability distribution over chord classes for each time frame:

$$\hat{Y} \in R^{1000 \times 25}$$

The predicted probabilities are converted to final chord labels using:

$$y_t = \arg \max_i \hat{y}_{t,i}, \quad t = 1, \dots, 1000$$

producing a frame-wise chord sequence that spans the entire audio.

The sequence of predicted chord labels is analyzed by a separate tonic detection module (KeyRecognizer). This module estimates the tonic chord by evaluating chord frequency, position, and co-occurrence using voting or rule-based heuristics:

$$\text{Tonic} = \arg \max_{c \in C} \text{Score}(c)$$

Evaluation

The final outputs are evaluated using:

- Frame-level accuracy,
- Chord metrics (e.g., WCRA, Triads, MIREX),
- Tonic chord accuracy (compared to ground truth key label).

This pipeline ensures both localized chord detection and global harmonic understanding.

Summary

The CNN model provides a fast and effective method for extracting localized chord information from time–frequency spectrograms. Its architectural simplicity allows rapid training and inference, making it ideal as a baseline system. Despite its

limitations in modeling temporal dependencies, it delivers strong performance in frame-wise chord classification, while downstream modules (e.g., KeyRecognizer) compensate for tonic inference using aggregated chord information.

2.3.4 CRNN Model Architecture for Identifying the Tonic Chord

Convolutional Recurrent Neural Networks (CRNNs) effectively combine the local pattern recognition strength of CNNs with the sequential modeling capacity of RNNs. This hybrid architecture is particularly suited for chord recognition tasks, where harmonic content is structured both vertically in the frequency domain and horizontally across time.

In music, chords generate localized spectral features—captured by CNNs—while their temporal evolution follows theory-driven progressions—captured by RNNs. CRNNs address both aspects in a unified framework: convolutional layers detect harmonic textures and chordal components, while recurrent layers model temporal context, improving coherence across frames (al S. e., 2015).

This dual capacity makes CRNNs ideal for frame-wise chord classification as well as for inferring broader tonal structures like the tonic chord of a musical piece.

Suitability of CRNNs for Chord Recognition

CRNNs are especially effective in modeling musical audio for the following reasons:

- Convolutional Layers extract localized harmonic patterns from spectrograms. These include chord root positions, harmonic stacks, and triadic structures (root–third–fifth) in the frequency domain.
- Recurrent Layers model temporal relationships across time, capturing how chords evolve in sequence and preserving the continuity of musical progression.

This dual capability enables the CRNN to handle both the vertical structure of individual chords and the horizontal structure of chord transitions.

Advantages

- Captures both local harmonic content and long-range temporal dependencies.
- More expressive than CNNs for modeling musical phrases or cadences.
- Suited for sequence labeling tasks like frame-wise chord classification.

Limitations

- Computationally heavier than CNNs due to sequential RNN processing.
- Requires careful regularization (e.g., dropout) to avoid overfitting.
- Tuning recurrent layers can be more complex.

CRNN Model Architecture Overview

The CRNN used in this system is inspired by the architecture proposed by Junyan Jiang et al. (2019). It integrates convolutional blocks with bidirectional gated recurrent units (Bi-GRU) to model time–frequency dynamics in musical audio. The model is

designed to first extract harmonic features locally from spectrograms, then capture chord continuity and transitions over time.

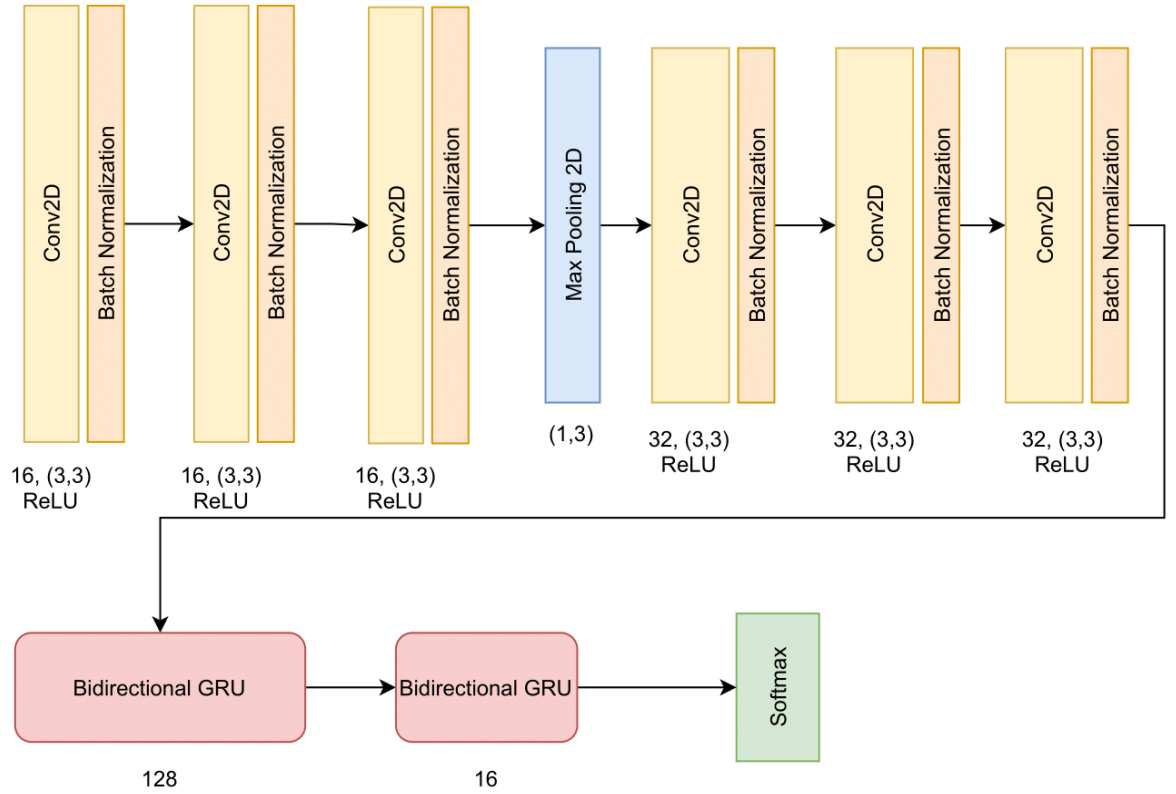


Figure 2.11: CRNN Model Architecture Overview

Input Layer

Let the input spectrogram be:

$$X \in R^{1000 \times 252 \times 1}$$

Where:

- 1000: number of time frames (~23.2 seconds),
- 252: frequency bins from the CQT representation,
- 1: single-channel (magnitude-only).

This input encapsulates both the short-term and mid-term harmonic structure of a song fragment. The spectrogram acts as a rich, compact representation of pitch and time, serving as the basis for feature extraction.

Convolutional Feature Extraction

The CRNN begins with stacked convolutional layers with batch normalization and ReLU activations:

- **Conv Block 1–3:** 16 filters, 3×3 kernel, ReLU + BatchNorm.
- **Conv Block 4–6:** 32 filters, 3×3 kernel, ReLU + BatchNorm.
- **Pooling Layer:** MaxPooling2D with size 1×3 to down sample frequency while preserving time.

Each layer transforms the input as:

$$F^{(l)} = \text{ReLU} \left(\text{BN} \left(X * K^{(l)} + b^{(l)} \right) \right)$$

Where $K^{(l)}$ is the convolutional kernel and $*$ denotes 2D convolution.

The convolutional blocks act as hierarchical detectors for harmonic features such as pitch clusters, partials, and overtones. BatchNorm helps stabilize learning and improves convergence, while asymmetric pooling ensures temporal information is retained.

Reshaping for Sequence Modeling

After the last convolutional layer, the tensor is reshaped into a 2D sequence of feature vectors:

$$X' = \text{Reshape}(F^{(6)}) \in R^{T \times D}$$

Where:

- T : number of time steps (frames),
- D : flattened feature dimension per time step.

This transformation converts 2D spatial features into sequential data that can be processed by RNN layers. Each frame becomes a high-dimensional feature vector representing its harmonic content.

Recurrent Sequence Modeling

Two stacked Bidirectional GRU (BiGRU) layers are used to capture both short-term and long-term harmonic transitions in the temporal domain. This recurrent block enables the model to not only consider the current frame but also integrate contextual information from both the past and future, which is critical for music where harmony often depends on what came before and what follows.

Let the reshaped output from the CNN be:

$$X' \in R^{T \times D}$$

Where:

- T : number of time steps (frames),
- D : number of extracted spectral features per frame.

The processing flow is as follows:

$$H^{(1)} = \text{BiGRU}_{128}(X', \text{dropout} = 0.5)$$
$$H^{(2)} = \text{BiGRU}_{16}(H^{(1)}, \text{dropout} = 0.5)$$

Each BiGRU layer consists of a forward GRU (left-to-right) and a backward GRU (right-to-left), and their outputs are concatenated. Here's the role of each layer:

- **First BiGRU Layer (128 units):**
Learns mid-level temporal features and captures prominent sequential harmonic relationships. It models basic progression patterns like tonic–subdominant–dominant cycles, repeated rhythmic chord motifs, or cadences.
- **Second BiGRU Layer (16 units):**
Acts as a refinement layer, condensing the information passed from the first layer into a compact representation suitable for frame-level classification. This layer is smaller in size to control overfitting and serves to emphasize fine-grained temporal consistency, smoothing noisy transitions in complex musical phrases.

By stacking these two layers, the model is able to hierarchically encode temporal information: first broadly and then precisely.

These RNN blocks enhance harmonic continuity, enable modeling of progressive musical movement, and improve classification stability across time — a crucial aspect of music understanding that static models like CNNs alone cannot offer. Together, these recurrent layers enhance the model's ability to recognize and maintain consistent harmonic interpretation over long musical sequences — essential for accurate progression modeling.

Output Layer (Softmax Classification)

The final predictions are made through a dense layer with softmax activation:

$$\widehat{y}_{t,i} = \frac{e^{z_{t,i}}}{\sum_{j=1}^C e^{z_{t,j}}}, \quad i = 1, \dots, C$$

Where:

- $C = 25$: number of chord classes (24 chords + *NoChord*),
- $\widehat{y}_{t,i}$: predicted probability for class i at time t ,
- $z_{t,i}$: unnormalized logit for class i at time t .

This layer provides a probability distribution over all possible chord classes per frame, enabling frame-wise classification and probabilistic interpretation of predictions.

Loss Function and Optimization

- **Loss Function:** Sparse Categorical Crossentropy (suitable for integer-labeled multi-class tasks):
- $L_{CE} = -\log(\widehat{y}_{t,c})$, c = ground-truth class at time t
- **Optimizer:** Adam optimizer with default parameters.
- **Metric:** Frame-wise accuracy to evaluate classification performance over time.

This objective encourages the model to assign high probability to the correct chord class for each time frame while penalizing incorrect ones proportionally.

Processing Steps for CRNN-Based Chord Recognition

Step 1: Preprocessing

- Raw audio is converted into CQT spectrograms.
- StandardScaler is applied to normalize features to zero mean and unit variance:

$$X_{\text{norm}} = \frac{X - \mu}{\sigma}$$

This ensures consistency and numerical stability for both training and inference.

Step 2: Model Inference

- Each normalized spectrogram segment is fed into the CRNN model.
- Outputs a frame-level probability distribution over chord classes:

$$\hat{Y} \in R^{1000 \times 25}$$

Each row of $\hat{Y}$ corresponds to the predicted chord class probabilities at a given frame.

Step 3: Prediction Aggregation

- Predicted chord labels are determined by:

$$y_t = \arg \max_i \widehat{y}_{t,i}, \quad t = 1, \dots, 1000$$

This results in a complete chord sequence over time for the input audio.

Step 4: Tonic Inference (Post-processing)

- The predicted chord sequence is analyzed using the KeyRecognizer module.
- A voting or statistical model selects the most likely tonic:

$$\text{Tonic} = \arg \max_{c \in \mathcal{C}} \text{Score}(c)$$

This final step links the chord-level predictions into a global key estimation, yielding musical insight at the scale of the full piece.

Evaluation

- Performance is evaluated using:
- Frame-wise Accuracy: Proportion of correctly labeled frames.
- Chord Structure Metrics: WCRA, Triads, MIREX score, segmentation metrics.
- Tonic Accuracy: Agreement between inferred tonic and ground-truth key.

Summary

The CRNN model offers a robust and musically informed architecture for chord recognition. It outperforms static models by leveraging convolutional layers for harmonic structure extraction and recurrent layers for capturing sequential patterns. By modeling both the "vertical" (chord content) and "horizontal" (chord progression) aspects of music, it delivers highly accurate and context-aware chord predictions — ideal for both frame-wise classification and tonic inference in musical audio.

2.3.5 Tonic Key Inference from Frame-wise Chord Predictions

Introduction

While the chord recognition model outputs a sequence of chord predictions at the frame level, musical analysis often requires a higher-level understanding — namely, identifying the **tonic key** of an entire piece. The tonic represents the harmonic “home” and plays a central role in defining the tonal structure of Western music. This section outlines a statistical approach to inferring the tonic key from the recognized chord sequence.

Motivation and Principle

In tonal music, each key is associated with a characteristic group of chords derived from its scale — typically seven diatonic chords built on each degree of the scale. For example, in C major, these chords are:

- I: C major
- ii: D minor
- iii: E minor
- IV: F major
- V: G major
- vi: A minor
- vii^o: B diminished

If a song contains a high frequency of chords drawn from one of these diatonic sets, it is reasonable to assume that this key governs the overall harmonic structure.

Inference Mechanism

The tonic key is inferred using a frequency-based voting mechanism. This process involves several steps:

1. Chord Distribution Collection

From the model’s predictions, we compute how often each chord class appears throughout the song:

$$\text{chord_counts} = \{c_i: n_i\}$$

where ci is the chord index (e.g., 1 = C major, 10 = E minor) and ni is the number of frames assigned to that chord.

2. Key Profiles

Each key has a predefined set of seven chords commonly found within it (based on diatonic harmony). For example:

C major $\rightarrow$ [C, Dm, Em, F, G, Am, Bm]

A minor $\rightarrow$ [Am, Bdim, C, Dm, Em, F, G]

These chord sets serve as templates representing the harmonic vocabulary of each key.

3. Scoring Candidate Keys

Each key is scored based on how many of its typical chords appear in the chord prediction sequence, and how frequently. For each key k , the score is computed as:

$$\text{Score}_k = \sum_{c_i \in \text{chords of } k} n_i$$

This score measures the statistical fit between the predicted chords and the harmonic profile of each key. Also, this scoring approach is robust to minor prediction noise, as it relies on aggregate frequency rather than exact chord sequences.

4. Key Selection

The key with the highest total score is selected as the most likely tonic:

$$\text{Tonic} = \arg \max_{k \in \mathcal{K}} \text{Score}_k$$

where $\mathcal{K}$ is the set of all 12 major keys (or optionally all major and minor keys, depending on design).

5. Optional: Relative Key Conversion

In cases where the analysis requires output in a specific mode (e.g., minor), a relative mapping may be applied. For instance:

- If the inferred key is C major and a minor mode is desired, the result is converted to A minor.
- This relative mapping follows standard music theory relationships between major and minor keys (e.g., $C \leftrightarrow A:\text{min}$, $G \leftrightarrow E:\text{min}$, etc.).

Example

Suppose the following distribution is observed from the chord predictions:

Table 2.10: Distribution of chord predictions

Chord	Count
C major	45
D minor	30
E minor	28
F major	40
G major	35
A minor	25
B minor	5

These chords align closely with the diatonic chords of C major, which would receive the highest score in the key inference step. Therefore, the tonic is inferred to be C.

If the output is required in minor mode, the system can optionally convert C major to A minor using relative key mapping. This is useful in contexts where the perceived emotional tone, stylistic interpretation, or analytical preference favors a minor representation, or when the tonal center of the music is ambiguous between a major and its relative minor.

Summary

This tonic key inference process provides a musically interpretable summary of the song's harmonic center by analyzing the frequency and distribution of frame-wise chord predictions. Rather than relying on symbolic or structural analysis, it adopts a statistical matching approach based on established music theory. This enables the system to identify the key with high accuracy even in the presence of noisy chord estimates, serving as a crucial post-processing step in the overall chord recognition pipeline. This makes tonic inference a vital bridge between low-level audio prediction and high-level harmonic understanding in the full chord recognition system.

2.4 Method to separate vocals using UNet & TFD_TFC_UNet

2.4.1 Data Solutions

Data collection

In this study, we utilize the MUSDB18 dataset, one of the most widely used and standardized datasets for music source separation tasks. MUSDB18 contains 150 professionally mixed stereo music tracks across various genres such as pop, rock, electronic, and hip-hop. Each track is approximately 3 to 6 minutes long and is provided in a multitrack format with isolated stems for the main sources: vocals, drums, bass, and other (including accompaniment like guitars, synths, etc.).

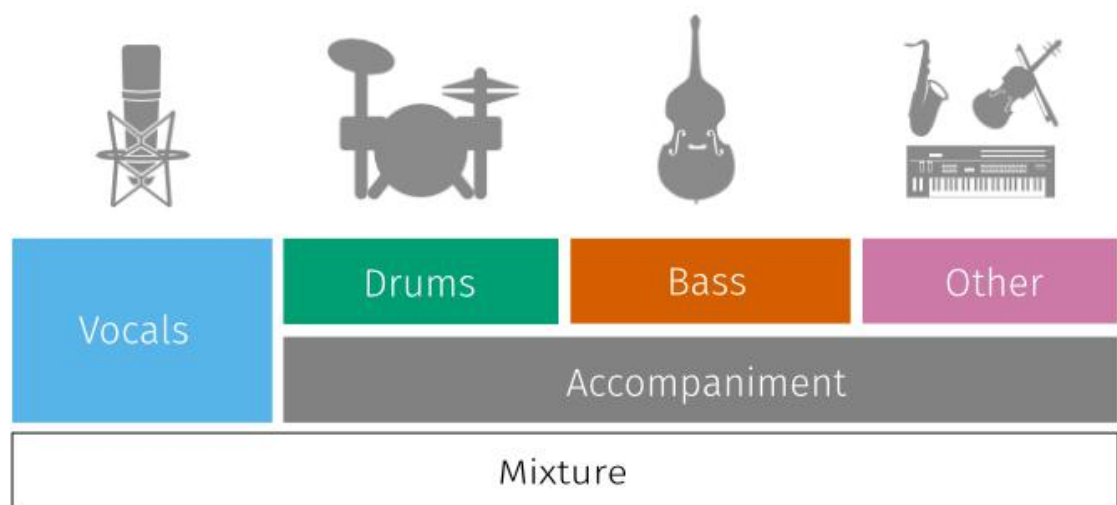


Figure 2.12: Musdb18 dataset

This dataset is particularly useful for supervised learning approaches because it offers both the mixture audio and the corresponding ground-truth source signals, which allows models to learn mappings from mixtures to specific target sources. In our case, we use MUSDB18 to train and evaluate models that aim to separate vocals from the background music (sigsep, 2022).

For the experiments, we follow the standard train/test split suggested by the dataset: 100 tracks for training and 50 tracks for testing. The training set is used to optimize the model parameters, while the test set is held out for final performance evaluation to ensure generalization.

Data preprocessing

To enhance the diversity and generalization capability of the model, we implemented a custom dataset class capable of dynamically generating audio chunks and corresponding training labels on the fly.

Each data sample is created through the following steps:

- Random selection of song and instrument: From the list of training songs, the system randomly selects one song and then randomly picks an instrument (e.g., vocals, drums, bass, etc.).
- Random audio chunk extraction: A fixed-length audio segment (chunk_size, e.g., ~4 seconds for TFD_TFC_UNet and ~1 second for UNet) is extracted from a random position in the selected song. If the original segment is shorter than the required size, zero-padding is applied to match the desired length.

Data augmentation

- Vocals may undergo transformations such as pitch shifting, frequency filtering using equalizers, or distortion effects.
- Other instruments are augmented by adding Gaussian noise or pitch shifting.
- Additionally, augmentations like signal reversal, polarity inversion, and gain adjustment are randomly applied.
- Mixture synthesis: After generating individual audio chunks for each instrument, we sum them together to create a mixed audio signal. This serves as the input for the model during training.

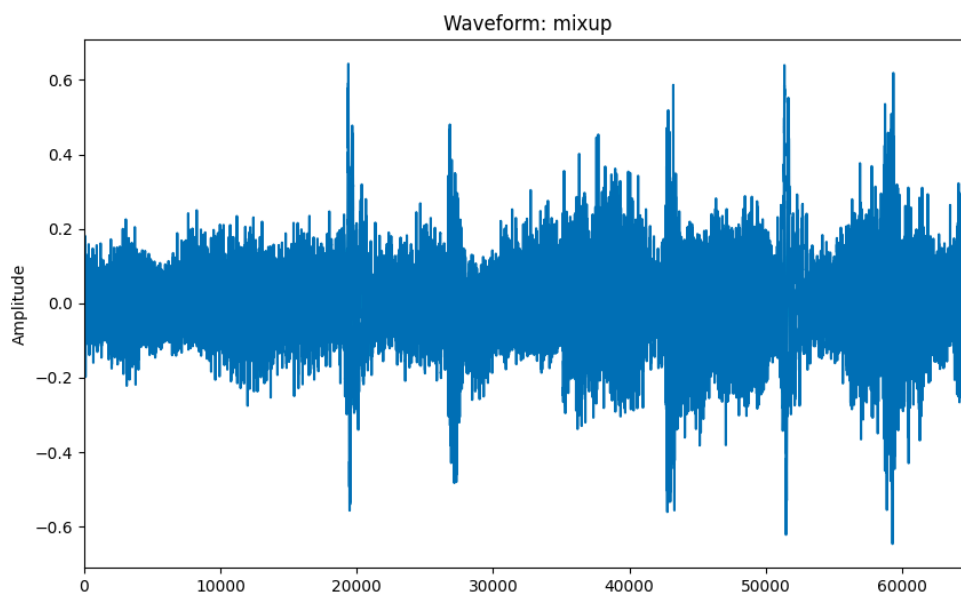
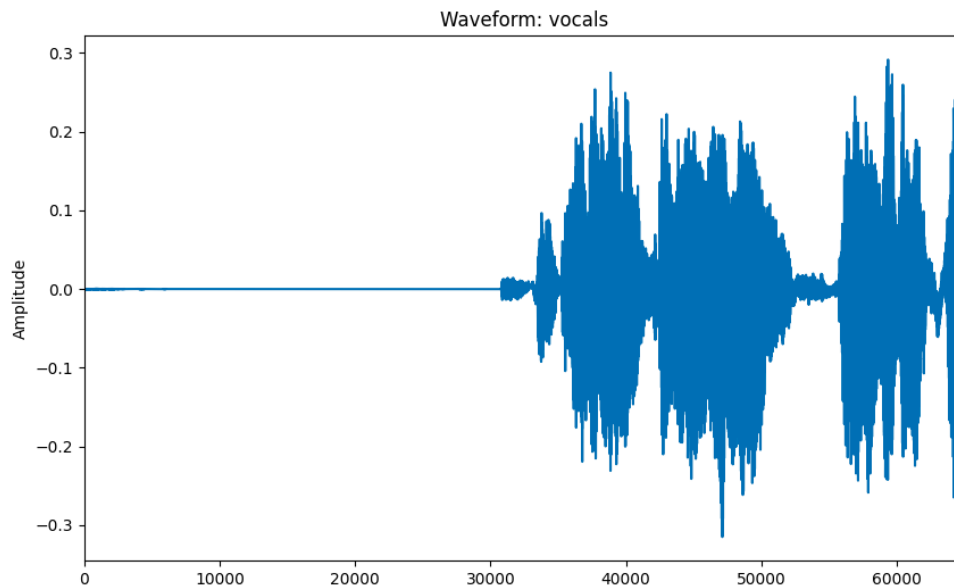


Figure 2.13: Waveform of one frame in dataset



Instead of using raw audio signals, the model operates on STFT (Short-Time Fourier Transform) representations to better capture frequency features.

During preprocessing, we use the `stft()` function to convert the input waveform into the frequency domain, and only the real part of the STFT is used as input to the model. The `inverse_stft()` function is applied to convert the model's output back to the time domain (Jimenez, n.d.).

Table 2.11: STFT parameters

Parameters	Value
Sample rate	16kHz
N_FFT	2048
Hop_length	512

Reasons for choosing these parameters

- **Sample Rate (16 kHz):** A sampling rate of 16,000 Hz is commonly used in speech processing tasks as it provides a good balance between audio quality and computational efficiency. It is sufficient to capture most of the frequency content of human speech, which typically ranges up to around 8 kHz.
- **N_FFT (2048):** A larger FFT window size allows for higher frequency resolution, which is helpful for distinguishing fine-grained spectral features. With an N_FFT of 2048 and a sampling rate of 16 kHz, the frequency resolution is approximately 7.8 Hz ($16,000 / 2048$), which is adequate for many audio analysis tasks.
- **Hop Length (512):** The hop length determines the time resolution of the STFT. A value of 512 (i.e., 32 ms at 16 kHz) provides a good compromise between time

and frequency resolution. It ensures that the STFT captures temporal dynamics without introducing excessive overlap, which would increase computational cost.

Together, these parameters enable the model to effectively analyze the audio signal in both time and frequency domains, ensuring that important features are preserved while maintaining a manageable computational load.

2.4.2 Evaluation method and loss function

MAE

Spectral loss is computed as the Mean Absolute Error (MAE) between the predicted and ground truth STFTs. It encourages the model to reconstruct accurate frequency representations of the sources.

$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

Where:

- $\hat{y}_i$: Predicted value for the i^{th} data point
- y_i : Actual value for the i^{th} data point
- n : Number of observations

Since vocal separation is sensitive to frequency characteristics (e.g., harmonics, pitch), working in the frequency domain allows the model to focus on those patterns. MAE is used instead of MSE to reduce the impact of outliers and produce smoother outputs.

Signal-to-Distortion Ratio (SDR)

SDR is a common metric for evaluating the quality of source separation. It measures how close the predicted waveform is to the target signal, penalizing both interference and artifacts.

$$SDR = 10 \cdot \log_{10} \left(\frac{|y_{true}|^2}{|y_{true} - y_{pred}|^2 + \epsilon} \right)$$

Where:

- y_{true} : Ground truth signal (original reference signal)
- y_{pred} : Predicted signal output from the model
- $|\cdot|^2$: Squared L2 norm (total energy of a signal)

- ϵ : Small constant (e.g., $1e-8$) to avoid division by zero
- SDR: Signal-to-Distortion Ratio, measured in decibels (dB)
- $SDR = \infty$: Indicates perfect prediction ($y_{pred} = y_{true}$)

$SDR < 0$: Indicates prediction worse than the signal itself

SDR directly reflects the perceptual quality of the separated audio. Higher SDR indicates less distortion in the reconstructed signal. It is widely used in audio source separation benchmarks.

By combining spectral loss (for training in the STFT domain) and SDR (as an evaluation metric in the time domain), we ensure that the model learns both fine-grained frequency details and produces high-quality waveforms that are perceptually accurate.

2.4.3 UNet model architecture for vocal separator

U-Net is a type of deep neural network originally designed for image segmentation – like identifying tumors in medical scans by dividing an image into meaningful parts. However, it can also be used to separate different sounds from a mixed audio signal. In vocal separation, the goal is to isolate the singing voice from background music.

Instead of using images, this version of U-Net works with spectrograms, which are 2D visual representations of audio signals (time vs. frequency) (Walid Ehab, 22 Sep 2023). These spectrograms are generated using a technique called Short-Time Fourier Transform (STFT), which turns the audio waveform into frequency-based information.

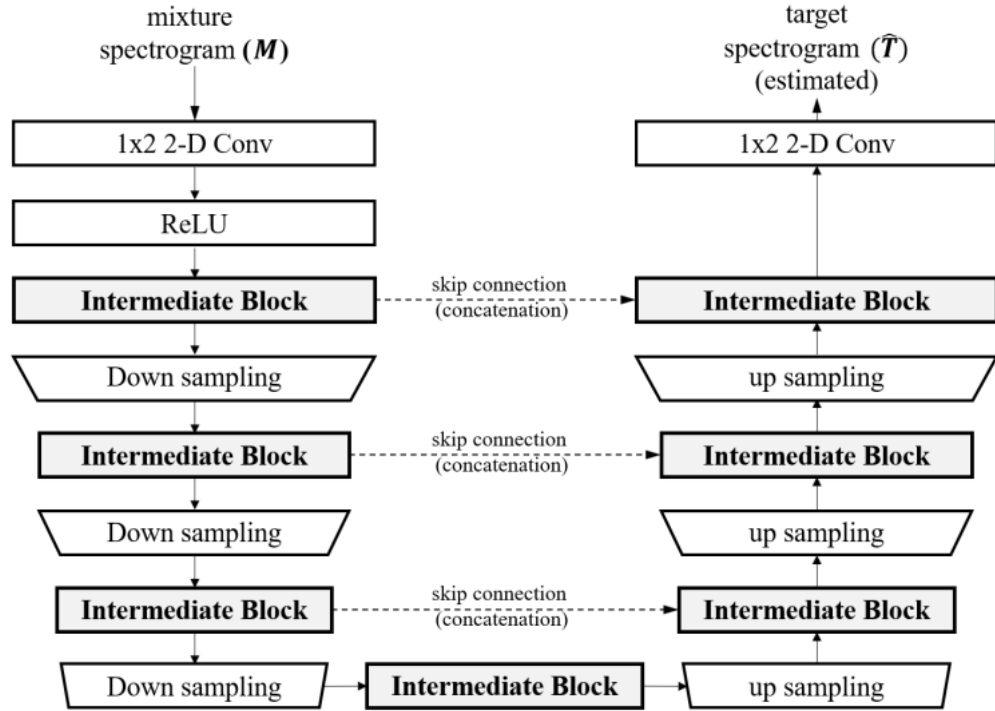


Figure 2.14: Unet architecture

U-Net Architecture (for Spectrogram Input)

The U-Net model has a U-shaped structure with three main parts:

1. Contracting Path (Encoder)

- **Input Preprocessing:**

The input to the U-Net is the magnitude spectrogram ($M_{\text{mag}} = |M_{\text{complex}}|$), derived from the complex spectrogram (M_{complex}) by applying the modulus operation (Choi, 2021):

$$M_{\text{mag}} = |M_{\text{complex}}|$$

This effectively removes the phase component, allowing the network to focus on magnitude features.

- **Convolutional Layers:**

A series of 2D convolutional layers with 3×3 filters is applied to extract local time-frequency patterns. These patterns capture harmonic and temporal features of the audio signal.

- **Activation Function:**

Each convolutional layer is followed by a ReLU activation function to introduce non-linearity and enhance the model's capacity to learn complex mappings.

- **Downsampling:**

Max-pooling layers (typically 2×2) reduce the resolution of the feature maps while preserving high-level abstract features. This compression helps the network focus on broader contextual patterns such as note transitions, harmonic progression, and rhythm structures.

- **Intermediate Blocks:**

Between pooling operations, intermediate blocks are inserted. Each block contains one or more convolutional layers and nonlinearities, processing the feature maps without changing their spatial resolution but allowing the network to learn richer representations.

2. Bottleneck

The bottleneck sits at the deepest point in the U-Net and has the smallest spatial dimensions but the richest channel-wise information. It serves as the core abstraction of the input spectrogram, where both frequency and temporal dependencies are compressed into a high-level latent representation. This representation encodes key vocal cues or harmonic signatures depending on the task.

An *intermediate block* is also used here to further refine this abstracted information before upsampling begins.

3. Expansive Path (Decoder)

- **Upsampling Layers:**

To recover the original input resolution, the decoder applies upsampling operations (typically using transposed convolutions or bilinear upsampling followed by convolution). This enlarges the spatial size of the feature maps step by step.

- **Skip Connections:**

A key component of U-Net is the use of **skip connections**. At each resolution level, feature maps from the encoder are concatenated with the corresponding upsampled maps from the decoder. These connections help restore spatial details lost during downsampling and improve gradient flow during training.

- **Intermediate Blocks (Decoder Side):**

After each concatenation, an intermediate block further processes the merged features to filter noise and refine the output representation.

- **Final Convolution:**

A concluding 1×2 convolutional layer projects the decoder output into the target spectrogram dimensions $\hat{T}$, which matches the input in time-frequency resolution but possibly with fewer channels (e.g., a single-channel vocal mask or a harmonic estimation map).

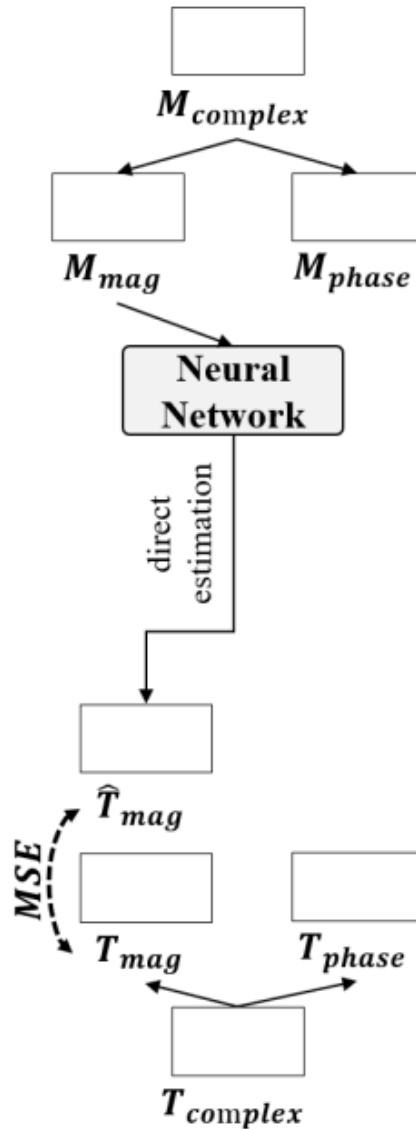


Figure 2.15: vocal separation pipeline

Here is a step-by-step explanation of the U-Net process for vocal separation (Belz, Jan 19, 2020):

Step 1: STFT Preprocessing

- The raw waveform of the mixed audio is transformed using STFT into a complex spectrogram $M_{complex}$.

- From this, two components are extracted:
 - Magnitude M_{mag} : represents the energy at each frequency and time.
 - Phase M_{phase} : captures the phase information needed to reconstruct the waveform later.

Step 2: Neural Network Processing

- The magnitude spectrogram M_{mag} is passed through the U-Net model.
- The model predicts a soft mask $f(M_{mag})$ which highlights the parts of the spectrogram that belong to the singing voice.

Step 3: Spectrogram Estimation

- The estimated magnitude of the vocal spectrogram is obtained by element-wise multiplying the soft mask with the original magnitude:

$$\widehat{T_{mag}} = f(M_{mag}) \odot M_{mag}$$

- This step effectively filters out the background music and retains only the vocals.

Step 4: Recombining Phase

- To convert the spectrogram back into an audio waveform, we need both magnitude and phase.
- The original phase M_{phase} from the input is reused:

$$\widehat{T_{complex}} = \widehat{T_{mag}} \cdot e^{jM_{phase}}$$

- This complex spectrogram is then passed through the inverse STFT (iSTFT) to generate the time-domain audio signal of the vocals.

2.4.4 TFD_TFC_UNet for vocal separator

❖ Time-Frequency Convolutions

In this work, we adopt Time-Frequency Convolution (TFC) as the baseline intermediate block in the U-Net architecture for spectrogram-based source separation. TFC is a densely connected block composed of multiple 2D convolutional layers, each designed to jointly capture local patterns across both time and frequency dimensions of the input spectrogram (Choi, 2021).

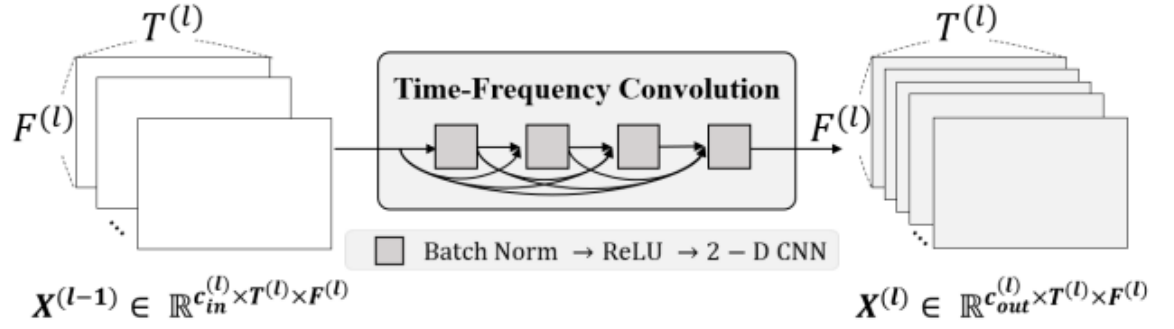


Figure 2.16: Time-Frequency Convolutions architecture

Architecture and Operations

Each TFC block is composed of a sequence of densely connected 2D convolutional layers. These layers are designed to process the input tensor:

$$X^{(l-1)} \in \mathbb{R}^{c_{in}^{(l)} \times T^{(l)} \times F^{(l)}}$$

where:

$c_{in}^{(l)}$: Number of input channels,

$T^{(l)}$: Time dimension

$F^{(l)}$: Frequency dimension

Each layer inside the block performs the following sequence of operations:

- 2D Convolution: Learns joint time-frequency patterns,
- Group Normalization (GN): Normalizes across groups of channels to stabilize training,
- GELU Activation: Introduces smooth, non-linear transformation.

The output of the TFC block is:

$$X^{(l)} \in \mathbb{R}^{c_{out}^{(l)} \times T^{(l)} \times F^{(l)}}$$

Dense Connectivity

A key characteristic of TFC blocks is dense connectivity, inspired by DenseNet. Suppose the block contains N layers. The input to the i -th layer H_i is the concatenation of all previous outputs:

$$H_i = \text{GELU} \left(\text{GN} \left(\text{Conv2D}([! [H_0, H_1, \dots, H_{i-1}]!]) \right) \right)$$

with:

- $H_0 = X^{(l-1)}$
- $[! [\cdot]!]$: denotes concatenation along the channel dimension.

This dense concatenation enables:

- Feature reuse from earlier layers,
- Improved gradient flow during backpropagation,
- Multi-scale representation learning across different receptive fields.

The final output is:

$$X^{(l)} = [! [H_1, H_2, \dots, H_N]!]$$

Time-Frequency Feature Extraction

TFC layers view spectrograms as images, where:

- The horizontal axis corresponds to time,
- The vertical axis corresponds to frequency.

Therefore, each 2D filter spans a patch across both axes. This allows the model to:

- Capture local harmonic structures (e.g., chords or instrument resonances),
- Detect short-term rhythmic patterns (e.g., vocal onsets or syllable changes),
- Learn context-aware audio features, essential for distinguishing overlapping sources in polyphonic audio mixtures.

Each layer within the TFC block consists of three consecutive operations: 2D convolution, followed by Group Normalization, and then a GELU activation function. These layers are densely connected, meaning that the output of each layer is concatenated with all previous layers and passed to the next one. This structure improves the flow of information and gradients, while allowing the model to extract hierarchical and complex time-frequency features.

TFC blocks treat spectrograms as 2D inputs, similar to images, where the horizontal axis represents time and the vertical axis represents frequency. The 2D filters used in this block learn to capture simultaneous temporal and spectral patterns, which is crucial for distinguishing overlapping sources in a mixture — such as separating vocals from background instruments.

This design serves as the core of the U-Net’s intermediate representation, providing a strong foundation for learning meaningful audio features and improving separation performance.

❖ Time-Distributed Fully-connected networks

To effectively model long-range spectral dependencies in audio signals, we integrate Time-Distributed Fully-connected (TDF) blocks into the U-Net architecture. Unlike convolutional layers that operate locally across time and frequency, TDF blocks apply fully connected layers to each frame independently, allowing the model to capture global relationships along the frequency axis at each time step (Choi, 2021).

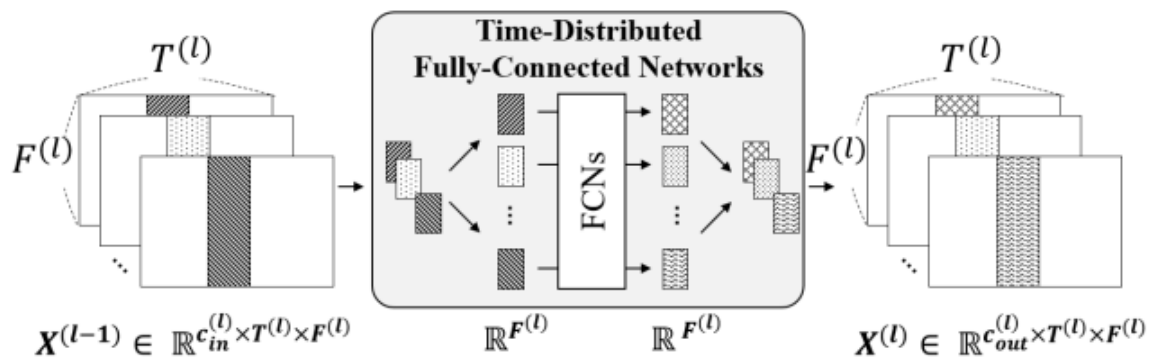


Figure 2.17: Time-Distributed Fully-connected networks architecture

1. Input and Reshaping

The same input shape with TFC.

2. Layer Structure

Each TDF block consists of one or more fully-connected layers, where each layer includes:

1. **Linear Transformation** (Dense/FC),
2. **Batch Normalization** (BN),
3. **ReLU Activation**.

If the block is multi-layered, internal layers map the input to a latent space of reduced dimensionality (bottleneck), followed by a final projection back to the output dimension.

Let:

- $x_t \in R^{F^{(l)}}$: frequency vector at time frame t
- bf : bottleneck factor,
- $h = \left\lfloor \frac{F^{(l)}}{bf} \right\rfloor$ number of hidden units in each dense layer.

Then, the transformations inside a 2-layer TDF block are:

$$h_t^{(1)} = \text{ReLU}(\text{BN}(W_1 x_t + b_1)) \in R^h$$

$$h_t^{(2)} = \text{ReLU}(\text{BN}(W_2 h_t^{(1)} + b_2)) \in R^{F^{(l)}}$$

This process is repeated independently for each time frame. The resulting tensor $X^{(l)} \in$

3. Bottleneck Design

Using a bottleneck structure ($bf > 2$) reduces the number of parameters and computational cost significantly while still preserving the ability to learn meaningful frequency representations:

Parameter efficiency increases as bf grows,

Empirically, $bf = 4$ or $bf = 8$ often yields a good trade-off between performance and efficiency.

Each TDF block processes a spectrogram-like input by applying the same fully connected network to every time frame and channel. This network consists of one or

more fully connected layers, each followed by Batch Normalization and ReLU activation. When using multiple layers, a bottleneck architecture can be introduced to reduce the number of parameters, where the hidden size is determined by a predefined bottleneck factor.

The main advantage of using fully connected layers in TDF is their ability to link all frequency bins within a frame, making them particularly effective at learning harmonic patterns and resonant frequency structures that are often dispersed across the frequency axis. These patterns are essential for distinguishing between sound sources such as vocals and instruments.

By operating in a time-distributed manner—processing each frame independently—TDF blocks extract time-independent spectral features, which complement the spatiotemporal patterns learned by convolutional blocks like TFC. This enables the model to more effectively separate complex mixtures by combining both local and global spectral insights.

❖ TFC with TDF

To combine the strengths of both local and global spectral modeling, we adopt the Time-Frequency Convolution with Time-Distributed Fully-connected (TFC-TDF) block as an intermediate component in the U-Net architecture. This block integrates a TFC block, which captures local time–frequency patterns, with a TDF block, which learns global frequency dependencies at each time frame (Choi, 2021).

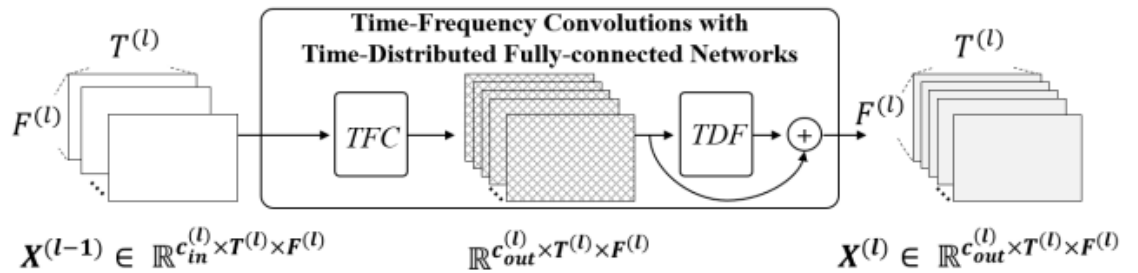


Figure 2.18: TFC with TDF architecture

• **TFC Block:**
First, the input is passed through a Time-Frequency Convolution block to produce an intermediate representation:

$$\widetilde{X}^{(l)} = \text{TFC}(X^{(l-1)}) \in R^{c_{\text{out}}^{(l)} \times T^{(l)} \times F^{(l)}}$$

- **TDF Block:**

The TDF block is then applied to the TFC output:

$$\widehat{X^{(l)}} = \text{TDF}(\widetilde{X^{(l)}})$$

- **Residual Connection:**

Finally, a residual connection adds the TDF output to the TFC output:

$$X^{(l)} = \widetilde{X^{(l)}} + \widehat{X^{(l)}}$$

The TFC-TDF block first applies a **TFC module** to the input feature map $X^{(l-1)}$ producing a spectrogram-like output of the same spatial dimensions but with updated channel depth $c_{out}^{(l)}$. The result is then passed through a TDF block, which applies a fully connected network to each frame independently to capture frequency correlations such as harmonic structures.

To improve training stability and preserve original information, a residual connection is added from the block input to its output. This design facilitates better gradient flow and enables the network to learn both low-level and high-level features more effectively.

Inspired by the Phasen model [63], which demonstrated the effectiveness of time-distributed operations in speech enhancement, the TFC-TDF block is evaluated in this work for music source separation. Its ability to model both time–frequency interactions and intra-frame frequency dependencies makes it particularly suited for separating vocals from complex music mixtures.

2.5 Conclusion

Chapter 2 presented a comprehensive analysis and design of the proposed system for vocal separation. It began by identifying both functional and non-functional requirements, followed by a detailed system design including use case diagrams, system architecture, and API specifications. The chapter then outlined methods for recognizing the tonic key and proposed a structured approach for vocal separation using deep learning.

CHAPTER 3: SYSTEM IMPLEMENTATION AND EVALUATION

3.1 Experimental Results

3.1.1 Overview of the Tonic Chord Identification Task

Data Collection and Preprocessing

- Training Dataset (Isophonics)

For model training, we leveraged the Isophonics chord transcription dataset, an established benchmark in MIR research. This collection provides 225 songs (principally by The Beatles, Queen, and others) with expert chord annotations in .lab format, including precise onset and offset times for each chord segment.

These annotations serve as rich supervised learning targets for chord recognition. They also include key/tonic information, aiding in tonic chord estimation (identifying each song's primary or "home" chord).

The genre diversity within Isophonics ensures the model is exposed to a broad range of chord progressions, making it well-suited for deep learning-based chord and tonic inference tasks.

❖ Chord-Type Distribution in Isophonics

In aggregate, the chord-type distribution is heavily skewed toward major triads:

- Major chords: 68.31% of the total annotated duration ($\approx 18,526.04$ seconds)
- Minor chords: 24.26% ($\sim 6,579.61$ seconds)
- Other chords (e.g., 7th, diminished): 7.43% ($\sim 2,013.53$ seconds)

This indicates that over 92% of all chords in the training data are either major or minor, justifying the use of a 24-class output space (12 majors, 12 minors). This predominance of major and minor triads reflects the functional harmony framework of Western tonal music, in which most progressions can be decomposed into triads built on scale degrees. According to classical harmonic theory, these triads form the foundation of cadences, modulations, and tonal centers, which further motivates the model's focus on distinguishing between major and minor sonorities.

❖ Most Common Chords in Isophonics

The top 5 most common chords in Isophonics are:

Table 3.1: The top 5 most common chords

Chord	Percentage
A	14.29%
G	11.54%
D	11.36%
E	11.11%
C	9.26%

These five chords alone account for more than half of all labeled chord time, emphasizing their foundational role in Western pop/rock harmony.

Each track's `.lab` file was parsed and aligned to its audio, enabling frame-wise CQT labeling. This step is essential for supervised training of CNN/CRNN models and for subsequent tonic inference. This mapping is essential not only for classification training, but also for later statistical inference of the song's tonic, which is estimated by analyzing the chord frequency distribution. The temporal precision offered by frame-level labeling ensures that transitions between harmonies are accurately captured in the input features.

❖ Test Dataset (Vietnamese Songs)

To evaluate generalization, we curated a custom dataset of 125 Vietnamese popular songs. Chord sheets were collected from public sources like *hopamviet.vn* and *hopamchuan.com*, then aligned with the audio to generate frame-level annotations in the same format as Isophonics.

This dataset introduces linguistic and cultural contrast while maintaining structural compatibility (same CQT configuration and preprocessing pipeline).

❖ Preprocessing Consistency

To ensure comparability, all annotations and features were structured identically to the training data:

- Same CQT representation
- Same frame hop length
- Pre-alignment of chord labels to the CQT time grid

This design ensures that the model does not inadvertently learn dataset-specific artifacts (such as different time resolutions or encoding strategies), thereby improving generalization. In MIR research, this form of dataset harmonization is often considered a critical preprocessing step when integrating heterogeneous sources.

❖ Visualization of Acoustic Features After Preprocessing

To concretely illustrate the outcome of the preprocessing pipeline, we present a spectro-temporal visualization of three representative audio samples from the training dataset. These visualizations reflect the Constant-Q Transform (CQT) spectrograms extracted from the raw waveform, serving as the direct input features to the deep learning models (CNN/CRNN) employed in the chord recognition task.

Each spectrogram is a two-dimensional matrix representing the frequency content of audio over time:

- ❖ The horizontal axis (X) denotes temporal evolution, segmented into 16 consecutive frames per sample.
- ❖ The vertical axis (Y) corresponds to 250 frequency bins derived from the CQT, capturing harmonic content with logarithmic frequency scaling.
- ❖ Color intensity encodes normalized spectral energy values, with higher intensities (yellow regions) indicating stronger frequency activations, and lower intensities (dark violet) representing spectral silence or low-energy components.

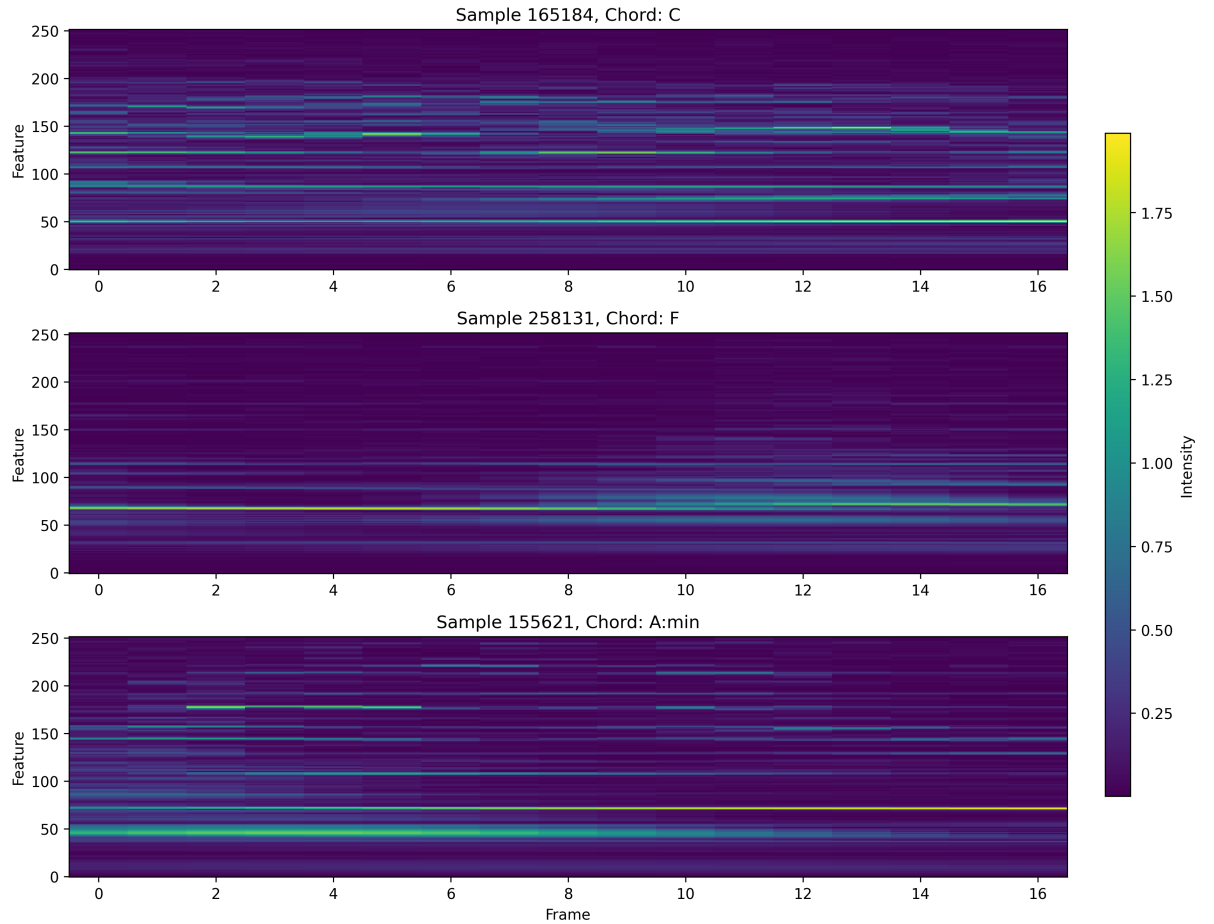


Figure 3.1: CQT-based spectrograms chords C, F, and A:min

❖ **Acoustic Signature Analysis**

- **Sample 165184 – Chord: C major**

Exhibits strong energy concentration around fundamental frequencies and their harmonics (e.g., ~50Hz, ~130Hz, ~170Hz). The parallel horizontal bands reflect a well-structured harmonic profile, characteristic of stable triadic sonorities.

- **Sample 258131 – Chord: F major**

Displays a dominant low-frequency response (~50–80Hz), with increased energy observed toward the latter half of the frame sequence. Harmonic clarity is present but slightly more diffuse, suggesting temporal variation in articulation.

- **Sample 155621 – Chord: A minor**

Characterized by a persistent spectral ridge near ~50Hz and additional partials at ~150Hz and ~175Hz. The energy distribution is less uniform across frequency bins, reflecting the more complex overtone structure typical of minor sonorities.

These spectrograms clearly demonstrate the acoustic diversity across chord classes, reinforcing the hypothesis that chords imprint distinct spectral-temporal patterns ("acoustic fingerprints") that are learnable by convolutional and recurrent architectures.

❖ **Relevance to Model Input Design**

The successful transformation of raw audio into structured CQT spectrograms fulfills several essential criteria for deep learning-based audio classification:

- Preservation of harmonic and temporal structure, crucial for modeling chord transitions.
- Dimensional regularity, enabling batch-based training of neural networks.
- Spectral resolution, supporting differentiation between harmonically related chords (e.g., C vs. A:min).

This representation not only enables the learning of local harmonic textures via convolutional layers, but also facilitates temporal sequence modeling through recurrent layers (e.g., GRU in CRNN), forming a robust feature basis for tonic chord inference.

❖ Chord-Type Distribution in Test Data

Out of 125 songs:

- 87 are in major keys → 69.60%
- 38 are in minor keys → 30.40%
- Other **chords**: 0%

This near 7:3 major-to-minor ratio is strikingly consistent with the Isophonics dataset.

Most Common Tonic Chords in Vietnamese Songs

Table 3.2: Most Common Tonic Chords in Vietnamese Songs

Chord	Percentage
D	12.80%
C	12.00%
A	11.20%
F	9.60%
E	8.80%

These chords are common in guitar-friendly keys. Importantly, four chords (A, C, D, E) overlap with the Isophonics top 5, showing strong similarity in harmonic focus despite different cultural contexts.

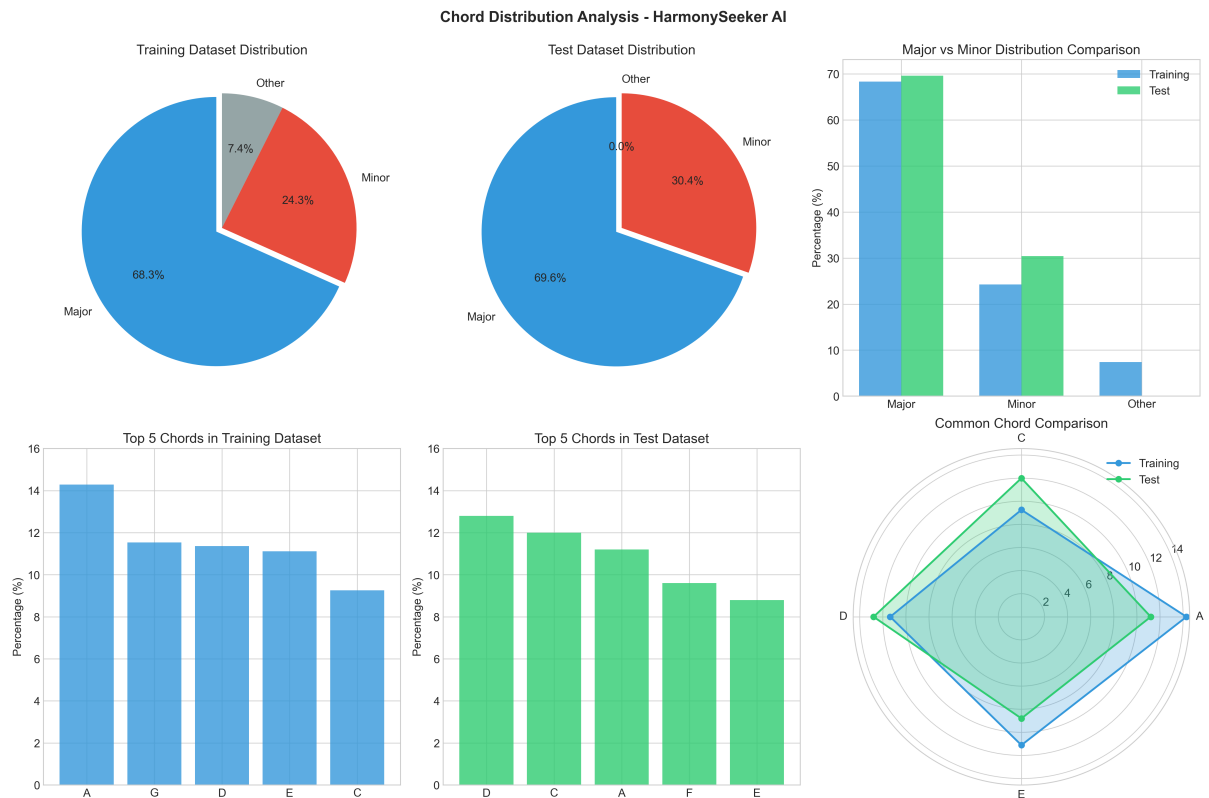


Figure 3.2: Chord Distribution Analysis in Training and Test Sets

- **Top pie charts:** show chord type proportions (major, minor, other)
- **Bar chart:** direct comparison of major/minor ratios
- **Bottom panels:** show top chords in each dataset
- **Radar plot:** highlights overlapping chords (A, C, D, E)

❖ Comparative Analysis of Distributions

The training and test sets exhibit **remarkable statistical similarity**:

- **Training:** ~68% major, ~24% minor (by duration)
- **Test:** ~70% major, ~30% minor (by song count)

Whether measured by time or file count, major chords dominate, reflecting a general trend in popular music. This consistency minimizes the risk of distribution shift, allowing the model to perform robustly during testing without encountering unusual chord distributions.

❖ Alignment with Music Theory

This result aligns with prior studies and music theory expectations. For instance, a Spotify analysis found ~66% of songs in major keys. The consistency in our datasets (~7:3 major:minor) supports the model's emphasis on major/minor distinctions.

❖ Chord-Level Overlap Between Datasets

The Isophonics set (from Western pop/rock) and the Vietnamese test set (V-pop) share four out of five top chords (A, C, D, E). The only difference is:

- G major is prominent in Isophonics
- F major appears in the Vietnamese set

This is a minor, musically consistent difference. Both chords are well-represented and do not affect training quality. From a statistical learning perspective, this overlap reduces the risk of class imbalance and rare-category overfitting. The fact that A, C, D, and E appear frequently in both datasets means the model's prior exposure aligns well with its deployment conditions. This strengthens its ability to infer tonic chords using chord frequency-based reasoning.

❖ **Handling of “Other” Chords**

- Found only in training set (7.43%)
- Absent from test tonic labels (0%)

During training, these chords were either:

- Mapped to a nearest major/minor triad, or
- Treated as unknown

Their low occurrence had minimal impact, and the test set being strictly major/minor ensures the model is evaluated on seen classes only. This design choice is supported by the observation that non-triadic chords (e.g., 7th, sus4, diminished) often serve ornamental or transitional roles and are less useful for identifying tonal centers. Mapping them to triadic equivalents maintains harmonic interpretability while simplifying the learning objective.

Table 3.3: Side-by-Side Chord Type Comparison

Chord Class	Isophonics Training Set (225 songs)	Vietnamese songs Test Set (125 songs)
Major	68.31% of duration (18,526.04 s)	69.60% of songs (87 songs)
Minor	24.26% of duration (6,579.61 s)	30.40% of songs (38 songs)
Other	7.43% of duration (2,013.53 s)	0%

❖ Conclusion

The data collection and preprocessing pipeline produced two datasets that are:

- Structurally aligned (CQT features, frame-level labels)
- Statistically consistent (major/minor balance and chord overlap)
- Culturally diverse but harmonically convergent

Such alignment ensures reliable training and evaluation, reduces the risk of distribution shift, and confirms the model's generalization capability in real-world music scenarios.

By ensuring format compatibility and chord distribution balance, this preparation supports high-performance tonic chord recognition and meaningful harmonic analysis across Western and Vietnamese music.

CNN Model Design and Training

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 17, 252, 32)	320
max_pooling2d (MaxPooling2D)	(None, 9, 126, 32)	0
conv2d_1 (Conv2D)	(None, 9, 126, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 63, 64)	0
conv2d_2 (Conv2D)	(None, 5, 63, 64)	36,928
flatten (Flatten)	(None, 20160)	0
dense (Dense)	(None, 64)	1,290,304
dense_1 (Dense)	(None, 25)	1,625

Total params: 1,347,673 (5.14 MB)

Trainable params: 1,347,673 (5.14 MB)

Non-trainable params: 0 (0.00 B)

Figure 3.3: CNN model architecture summary

The framewise chord classification CNN is a straightforward architecture with three convolutional layers followed by two fully connected layers (*Figure 3.4*). Each Conv2D layer uses a 3×3 kernel (with ReLU activation) to extract local spectral features, and is followed by a max-pooling layer to down-sample the time–frequency representation.

The first conv layer has 32 filters (output shape $17 \times 252 \times 32$), and subsequent conv layers increase to 64 filters, capturing more complex harmonic patterns (triads, overtones). After the final conv layer, the feature maps ($5 \times 63 \times 64$) are flattened to a 20,160-dimensional vector, feeding into a 64-unit dense layer and finally a softmax output layer with 25 classes (12 major + 12 minor chords, plus a “no chord” class).

This yields a total of ~ 1.35 million trainable parameters (*Figure 3.4*), reflecting a compact model that is efficient for GPU inference.

The CNN’s design capitalizes on spatial patterns in the Constant-Q spectrogram input – chords appear as vertical frequency clusters – but it lacks any recurrent units, meaning each frame is classified independently without explicit temporal context.

This simplicity makes the CNN a strong baseline for chord recognition, albeit one that cannot learn long-term chord transitions or enforce sequence consistency.

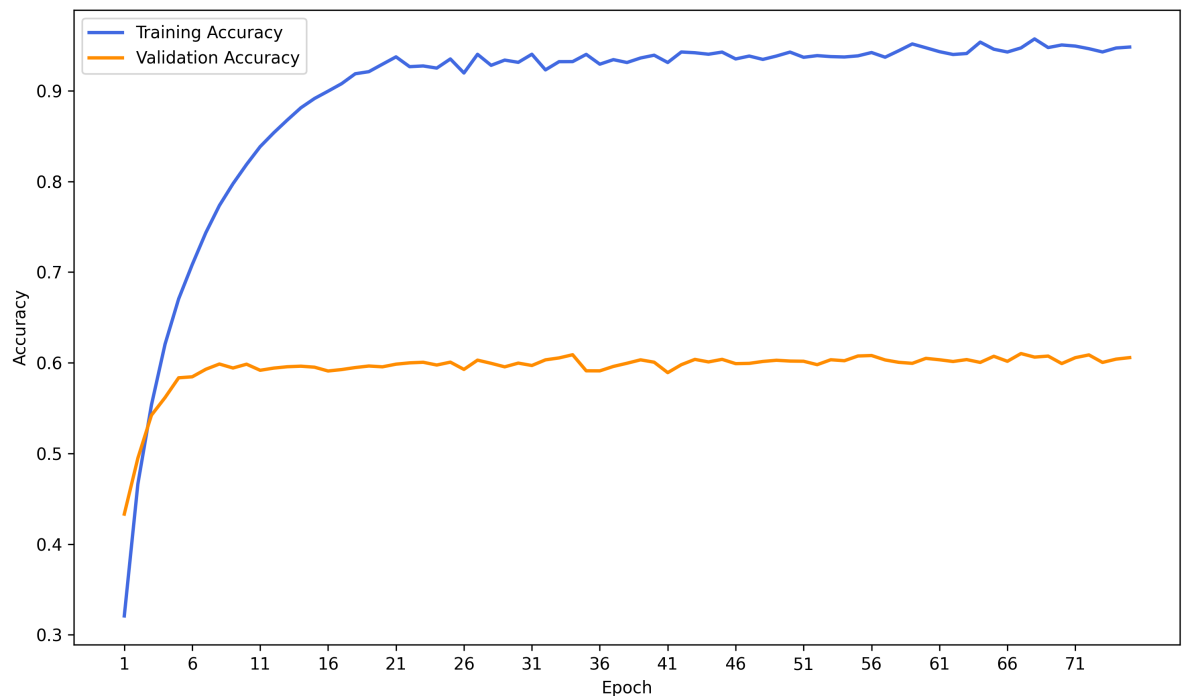


Figure 3.4: CNN training accuracy vs. validation accuracy over

The CNN model was trained with a frame-wise cross-entropy loss and Adam optimizer, using the Isophonics corpus for training and Vietnamese songs for validation.

Figure 3.3 shows that the CNN converged rapidly: training accuracy climbs above 90% within ~ 15 epochs, indicating the model quickly fits the training data’s harmonic patterns.

However, validation accuracy peaks around only ~ 60 – 63% early in training and then stagnates or even declines slightly. This widening gap between training ($\sim 95\%$)

and validation (~60%) accuracy suggests overfitting – the CNN memorizes training chord features but struggles to generalize to unseen songs.

We observe that after ~20 epochs, validation performance plateaus and begins to oscillate, while training accuracy continues improving towards 90–92%. This behavior indicates the model captured spurious details specific to the training set (e.g. timbre or instrument biases) rather than robust features, and it was unable to correct these due to the absence of sequence modeling.

In general, the CNN provides fast learning of local chord cues but demonstrates limited ability to mitigate errors like confusing similar chords (e.g. relative majors/minors) without additional context. No significant improvement on validation after early epochs suggests that regularization (such as dropout or early stopping) would be needed to prevent overfitting, or that a more expressive model is required for better generalization.

CRNN Model Design and Training**Model: "sequential"**

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 1000, 252, 16)	160
batch_normalization (BatchNormalization)	(None, 1000, 252, 16)	64
conv2d_1 (Conv2D)	(None, 1000, 252, 16)	2,320
batch_normalization_1 (BatchNormalization)	(None, 1000, 252, 16)	64
conv2d_2 (Conv2D)	(None, 1000, 252, 16)	2,320
batch_normalization_2 (BatchNormalization)	(None, 1000, 252, 16)	64
max_pooling2d (MaxPooling2D)	(None, 1000, 84, 16)	0
conv2d_3 (Conv2D)	(None, 1000, 84, 32)	4,640
batch_normalization_3 (BatchNormalization)	(None, 1000, 84, 32)	128
conv2d_4 (Conv2D)	(None, 1000, 84, 32)	9,248
batch_normalization_4 (BatchNormalization)	(None, 1000, 84, 32)	128
conv2d_5 (Conv2D)	(None, 1000, 84, 32)	9,248
batch_normalization_5 (BatchNormalization)	(None, 1000, 84, 32)	128
reshape (Reshape)	(None, 1000, 2688)	0
bidirectional (Bidirectional)	(None, 1000, 256)	2,164,224
bidirectional_1 (Bidirectional)	(None, 1000, 32)	26,304
dense (Dense)	(None, 1000, 25)	825

Total params: 2,219,865 (8.47 MB)**Trainable params:** 2,219,577 (8.47 MB)**Non-trainable params:** 288 (1.12 KB)

Figure 3.5: CRNN model architecture summary

The architecture begins with a convolutional front-end nearly identical to the CNN's: six Conv2D layers (3×3 kernels, ReLU) organized in two blocks (layers 1–3 with 16 filters each, and layers 4–6 with 32 filters each). Each conv layer is followed by Batch Normalization (BN) to stabilize learning, and after the first three conv layers

a 1×3 max-pooling is applied to reduce the frequency dimension from 252 bins to 84 while preserving the full 1000-frame time context.

This asymmetric pooling (only in frequency) ensures that the time dimension (≈ 23.2 s of audio) is maintained, allowing the model to analyze long chord progressions in one forward pass.

The output of the convolutional stack is a feature map of shape $1000 \times 84 \times 32$, which is reshaped into a sequence of 1000 frames, each represented by a 2688-dimensional feature vector.

These framewise feature vectors are then fed into two stacked Bidirectional GRU (BiGRU) layers for sequential modeling. The first BiGRU layer has 128 units in each direction (256 outputs per time step), capturing mid-level temporal features such as common chord progressions and rhythm patterns.

The second BiGRU is a smaller 16-unit (per direction) layer that acts as a refinement stage, condensing information and smoothing out transient prediction noise. This stacked RNN design allows the model to learn broad temporal context in the first layer and then focus on fine-grained timing consistency in the second, which also helps regularize against overfitting.

Finally, a time-distributed dense layer with softmax produces frame-level chord probabilities across the same 25 output classes as the CNN (covering all major/minor chords plus no-chord).

The CRNN comprises ~ 2.22 million parameters (*Figure 3.6*), roughly $1.6\times$ the CNN's size, reflecting the added complexity of recurrent layers. Notably, the CRNN retains the CNN's strength in local pattern recognition (via conv features) and combines it with sequence learning, enabling both vertical (frequency) and horizontal (time) modeling of music.

This design is well-suited to music, as it can detect chordal textures in each frame while also ensuring predictions follow logical progressions over time.

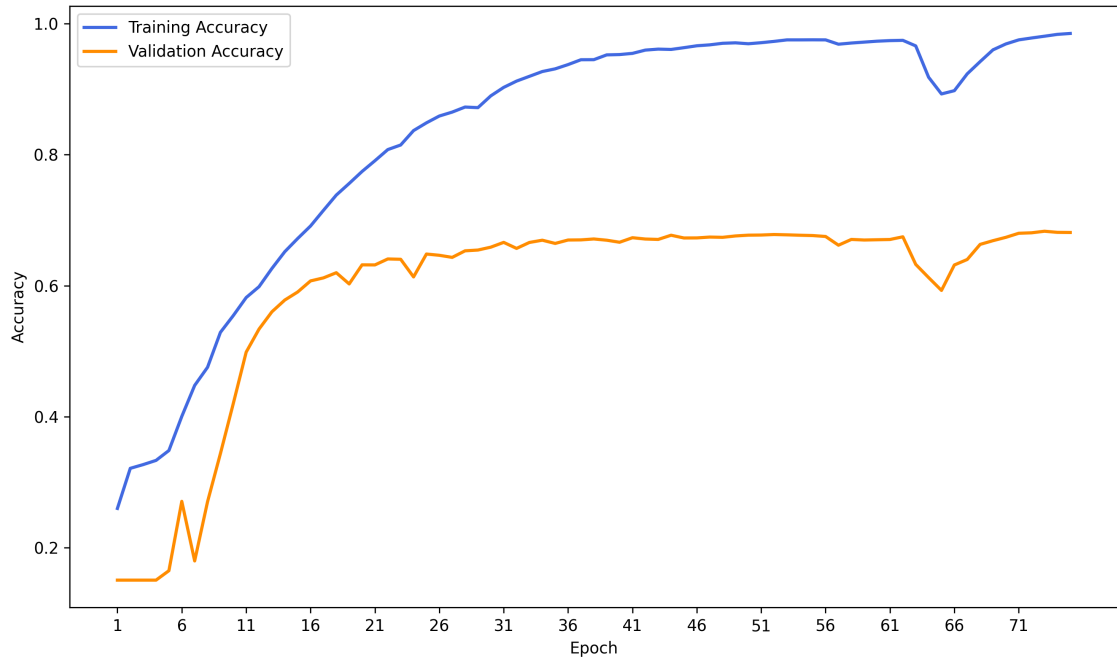


Figure 3.6: CRNN training vs. validation accuracy curves.

Training the CRNN proved more gradual but ultimately more effective than the CNN. As shown in *Figure 3.7*, the training accuracy (blue) starts lower and increases more slowly compared to the CNN, reflecting the CRNN's greater complexity and the challenge of optimizing recurrent units.

After ~30 epochs the CRNN's training accuracy surpasses 90%, and it approaches ~99% by 75 epochs, indicating the model can eventually fit the training data well.

More importantly, the validation accuracy (orange) rises steadily from a low initial value (around 10–20% in early epochs) up to roughly 65% at convergence, outperforming the CNN's validation plateau.

The gap between training and validation accuracy is noticeably smaller for the CRNN than for the CNN – by epoch 75, validation is ~0.60–0.65 versus ~0.99 train, whereas the CNN saw ~0.60 vs 0.92.

This suggests the CRNN generalizes better, likely due to its ability to learn smoother, context-consistent chord sequences that match the musical structure of the validation songs.

We do observe some overfitting (training continues climbing past 90% while validation flattens), but the effect is less severe than in the CNN. The inclusion of Batch Norm in conv layers and the second, smaller GRU layer (16 units) serves as implicit regularization, helping the CRNN avoid the sharp memorization of training data that the CNN exhibited.

Overall, the CRNN's training curve indicates a more reliable learning of musical patterns: it needed more epochs to converge, but it achieved a higher peak validation accuracy, foreshadowing its superior performance on identifying the tonic chord of songs.

Table 3.4: Evaluation Configurations for Tonic Chord Recognition

#	Model	Audio Type	Tolerance	Mode-Aware	Evaluation Tag
1	CNN	original_audio	strict	No	cnn_original_strict_no_major_minor
2	CNN	original_audio	strict	Yes	cnn_original_strict_with_major_minor
3	CNN	original_audio	semi-strict	No	cnn_original_semi_strict_no_major_minor
4	CNN	original_audio	semi-strict	Yes	cnn_original_semi_strict_with_major_minor
5	CNN	separated_audio	strict	No	cnn_separated_strict_no_major_minor
6	CNN	separated_audio	strict	Yes	cnn_separated_strict_with_major_minor
7	CNN	separated_audio	semi-strict	No	cnn_separated_semi_strict_no_major_minor
8	CNN	separated_audio	semi-strict	Yes	cnn_separated_semi_strict_with_major_minor
9	CRNN	original_audio	strict	No	crnn_original_strict_no_major_minor
10	CRNN	original_audio	strict	Yes	crnn_original_strict_with_major_minor
11	CRNN	original_audio	semi-strict	No	crnn_original_semi_strict_no_major_minor
12	CRNN	original_audio	semi-strict	Yes	crnn_original_semi_strict_with_major_minor
13	CRNN	separated_audio	strict	No	crnn_separated_strict_no_major_minor
14	CRNN	separated_audio	strict	Yes	crnn_separated_strict_with_major_minor
15	CRNN	separated_audio	semi-strict	No	crnn_separated_semi_strict_no_major_minor
16	CRNN	separated_audio	semi-strict	Yes	crnn_separated_semi_strict_with_major_minor

Comparative Evaluation and Analysis

❖ Extended Evaluation Framework: Tolerance and Mode-Aware Evaluation

To comprehensively assess the model's ability to identify the tonic chord under different conditions, we define an evaluation matrix comprising 16 distinct configurations. These configurations vary by:

- Model Type: CNN or CRNN
- Input Type: original_audio or separated_audio (vocals only)
- Tolerance Mode: strict or semi-strict (± 0.5 semitone)
- Mode Awareness: whether or not the system adjusts the predicted key based on inferred major/minor scale

The complete list of evaluation cases is shown below:

Note: separated_audio refers to vocal-only signals obtained using the voice separation model described in Section 3.1.2. These files are used to evaluate robustness in more challenging settings, where harmonic cues from instruments are absent or degraded.

❖ Evaluation Definitions

1. Tolerance Mode: Strict vs. Semi-Strict (± 0.5 Semitone)

A key dimension in our evaluation is **tolerance to small pitch deviations** in key prediction.

- Strict Mode: The predicted key must match the ground truth exactly (including both root and mode, if enabled).
- Semi-Strict Mode (± 0.5 tone): The predicted key is considered correct if it is one semitone (half-step) higher or lower than the correct label. This reflects musical reality, where key labeling can be subjective or ambiguous in modulated or microtonal contexts.

For example, if the ground-truth tonic is D, then all of the following would be considered correct under semi-strict tolerance: D $\flat$ (C $\sharp$), D, or D $\sharp$

The tolerance is applied symmetrically, using circular wrap-around on the 12-tone chromatic scale. Each major/minor chord belongs to one of these tone ladders:

- **Major Chords (Ordered Chromatically)**
["C", "C $\sharp$ ", "D", "D $\sharp$ ", "E", "F", "F $\sharp$ ", "G", "G $\sharp$ ", "A", "A $\sharp$ ", "B"]
- **Minor Chords (Corresponding to Majors)**
["C:min", "C $\sharp$:min", "D:min", "D $\sharp$:min", "E:min", "F:min", "F $\sharp$:min", "G:min", "G $\sharp$:min", "A:min", "A $\sharp$:min", "B:min"]

Hence, if the correct label is:

- A:min $\rightarrow$ Acceptable predictions are A $\sharp$:min, A:min, or G $\sharp$:min

- $G \rightarrow$ Acceptable predictions are $G^\sharp$, G , or $F^\sharp$

This approach mitigates strict penalization of musically small errors, such as predicting a key a half-step sharp or flat.

2. Mode Awareness and Relative Key Mapping

Mode-aware evaluation is another critical enhancement in our framework. In this setting, the predicted chord is adjusted based on the song's actual scale mode (major or minor), which is inferred externally using the Essentia key estimation library.

- If the predicted key is C and Essentia detects the song is in minor mode, the final key is mapped to $A:\text{min}$ — the relative minor of C major.
- Conversely, if the prediction is $E:\text{min}$ and Essentia indicates a major key, the output is converted to G — the relative major of $E:\text{min}$.

This mapping is grounded in classical music theory and the circle of fifths concept. The fixed relationships used for relative key conversion are:

Table 3.5: Mapping of Major Keys to Their Relative Minor Keys

Major Key	Relative Minor
C	$A:\text{min}$
$C^\sharp$	$A^\sharp:\text{min}$
D	$B:\text{min}$
$D^\sharp$	$C:\text{min}$
E	$C^\sharp:\text{min}$
F	$D:\text{min}$
$F^\sharp$	$D^\sharp:\text{min}$
G	$E:\text{min}$
$G^\sharp$	$F:\text{min}$
A	$F^\sharp:\text{min}$
$A^\sharp$	$G:\text{min}$
B	$G^\sharp:\text{min}$

Table 3.6: Mapping of Minor Keys to Their Relative Major Keys

Major Key	Relative Minor
A:min	C
A#:min	C#
B:min	D
C:min	D#
C#:min	E
D:min	F
D#:min	F#
E:min	G
F:min	G#
F#:min	A
G:min	A#
G#:min	B

This transformation ensures that predictions are evaluated in line with the musical character of the song. Without mode-awareness, a model that predicts “C” for a song in A:min would be penalized, even though those two keys are harmonically equivalent in many contexts.

Important: This mapping is only applied if `mode-aware` is enabled. If not, the predicted chord is accepted as-is, with no reference to the song’s mode.

By formalizing both tolerance-based evaluation and mode-aware relative key adjustment, we construct a multi-dimensional evaluation framework. This allows us to:

- Analyze models not only in ideal scenarios, but also in ambiguous or musically plausible ones.
- Distinguish whether a model failed due to mode confusion or tonal drift.
- Gauge the practical utility of each model under real-world musical expectations.

This framework lays the foundation for the per-configuration analysis in the upcoming sections, where we present detailed results, confusion matrices, and interpretations for each of the 16 evaluation settings.

❖ Evaluation Results Across 16 Experimental Configurations

To comprehensively assess the performance of both CNN and CRNN models under varied evaluation conditions, we conducted a systematic set of 16 experiments. These were defined by three independent axes:

- **Model Type:** CNN or CRNN
- **Input Audio:** Original or vocal-separated audio
- **Evaluation Setting:**
 - Tolerance: Strict (exact match) or Semi-strict (± 0.5 semitone tolerance)
 - Mode-Aware: Yes or No (whether the predicted tonic is adjusted to match the global scale)

Each configuration was applied to the same test set of 125 Vietnamese songs, and the number of correct tonic predictions was recorded accordingly. Below, we present the complete results, including confusion matrices and summary accuracy.

❖ **CNN – Original Audio**

1. **Strict, No Mode-Aware** (cnn_original_strict_no_major_minor)

- Correct Predictions: 21 / 125
- Accuracy: 16.80%

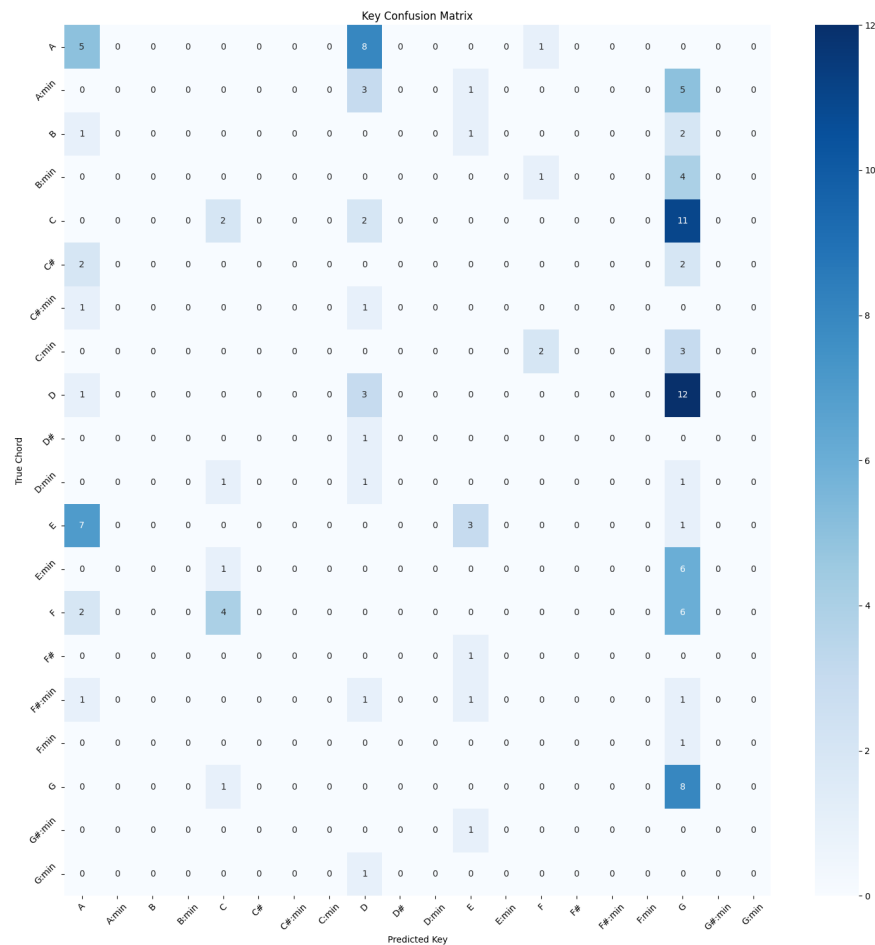


Figure 3.7: CNN Original Audio – Strict – No Mode-Aware

2. Strict, Mode-Aware (cnn_original_strict_with_major_minor)

- Correct Predictions: 28 / 125
- Accuracy: 22.40%

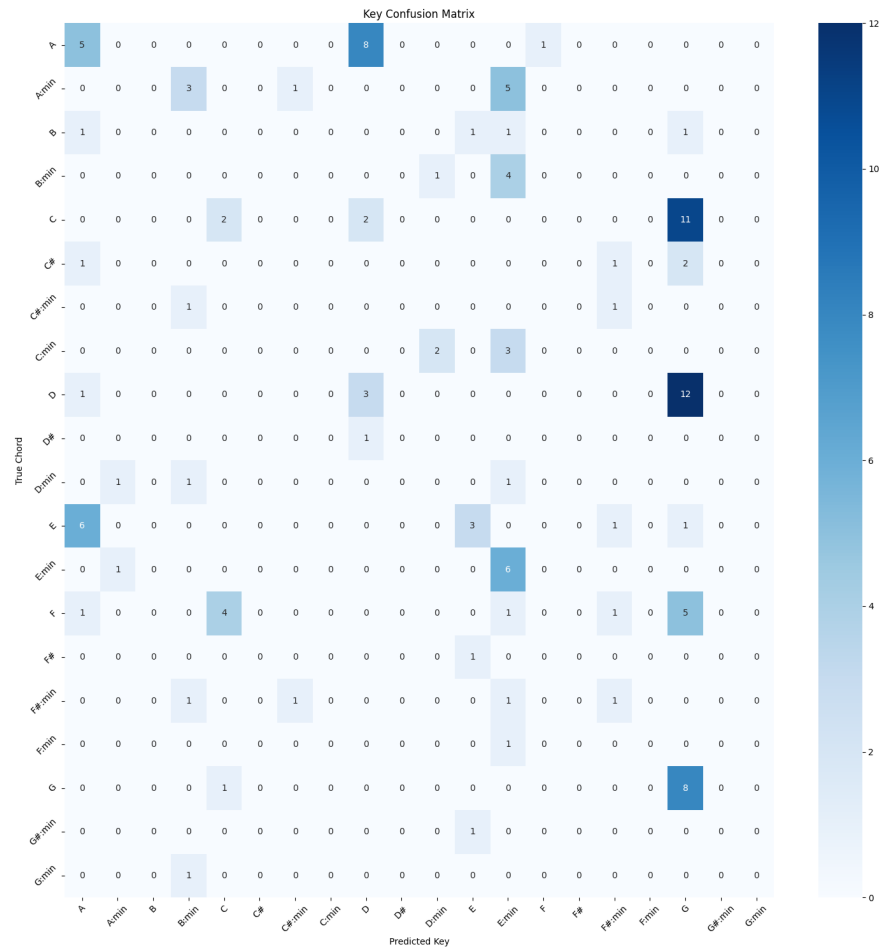


Figure 3.8: CNN Original Audio – Strict – Mode-Aware

3. Semi-Strict, No Mode-Aware

(cnn_original_semi_strict_no_major_minor)

- Correct Predictions: 27 / 125
- Accuracy: 21.60%

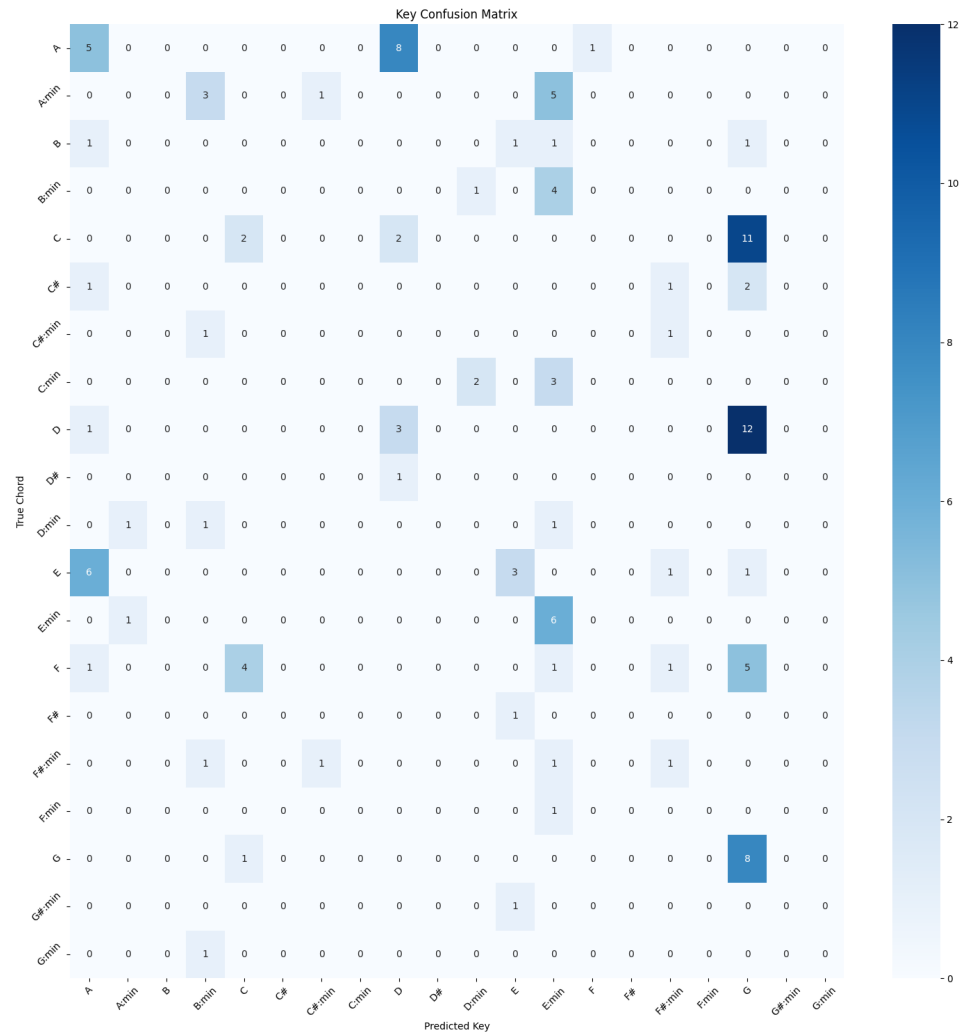


Figure 3.9: Original Audio – Semi-Strict – No Mode-Aware

4. Semi-Strict, Mode-Aware

(cnn_original_semi_strict_with_major_minor)

- Correct Predictions: 36 / 125
- Accuracy: 28.80%

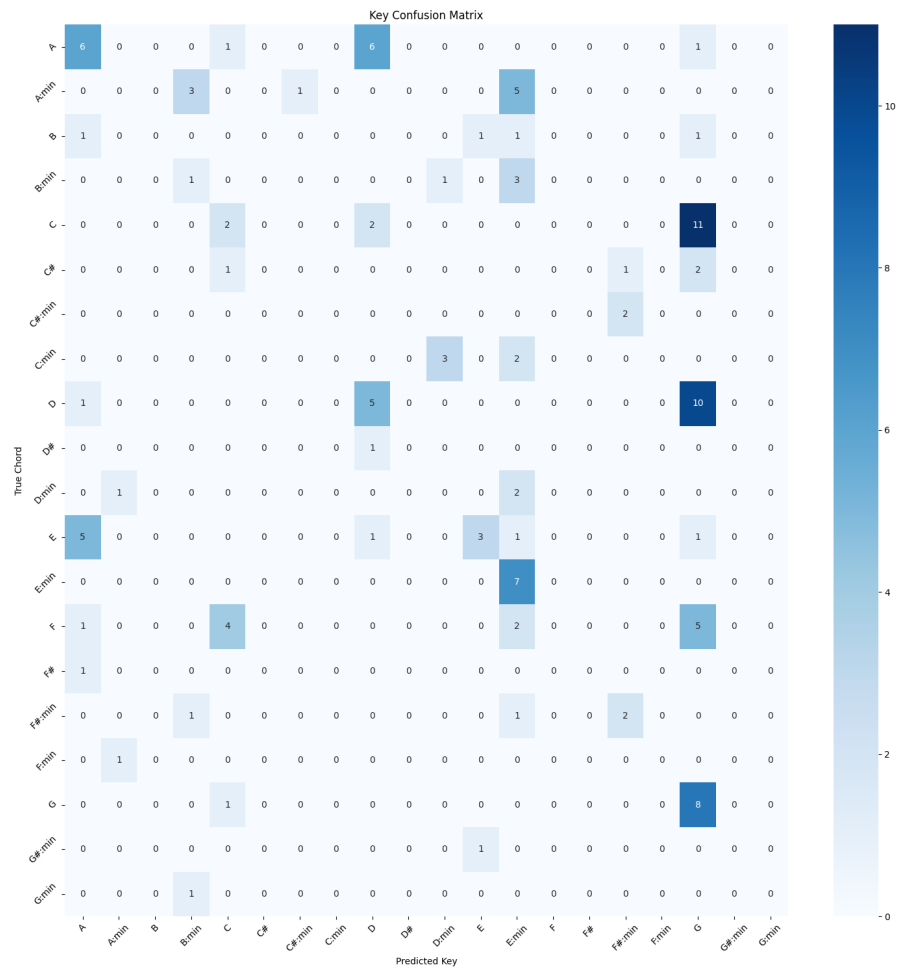


Figure 3.10: CNN Original Audio – Semi-Strict – Mode-Aware

❖ **CNN – Separated Audio**

5. **Strict, No Mode-Aware** (cnn_separated_strict_no_major_minor)

- Correct Predictions: 25 / 125
- Accuracy: 20.00%

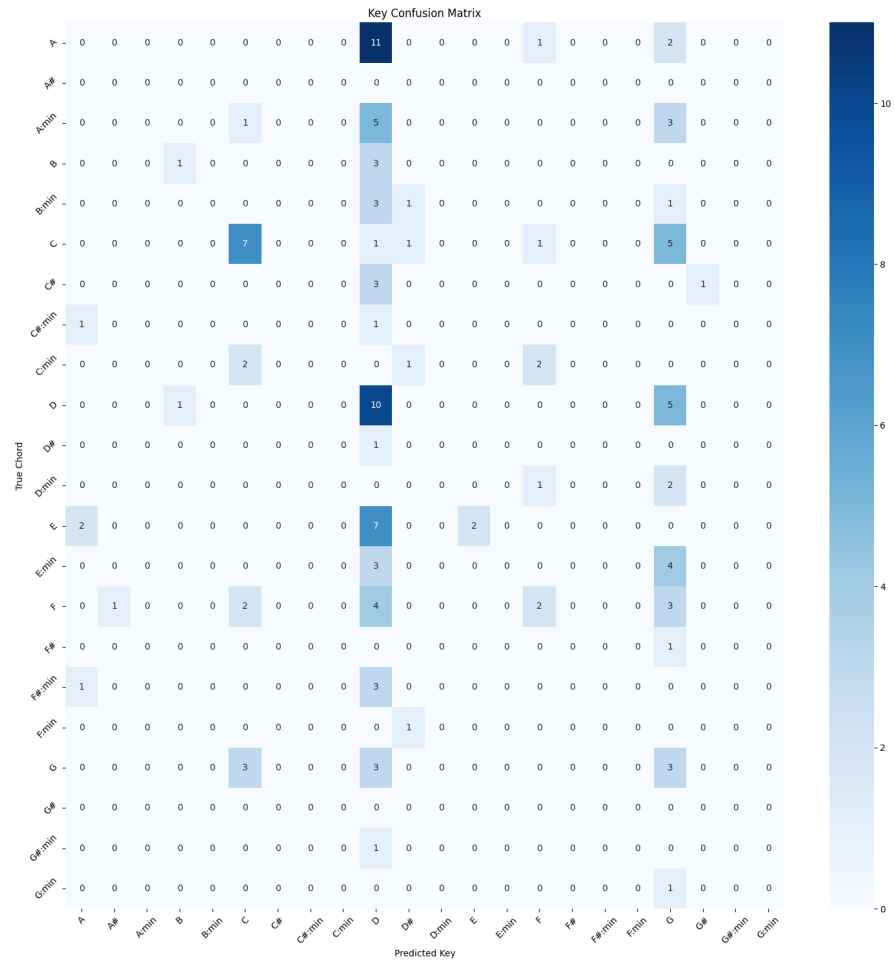


Figure 3.11: Separated Audio – Strict – No Mode-Aware

6. Strict, Mode-Aware (cnn_separated_strict_with_major_minor)

- Correct Predictions: 34 / 125
- Accuracy: 27.20%

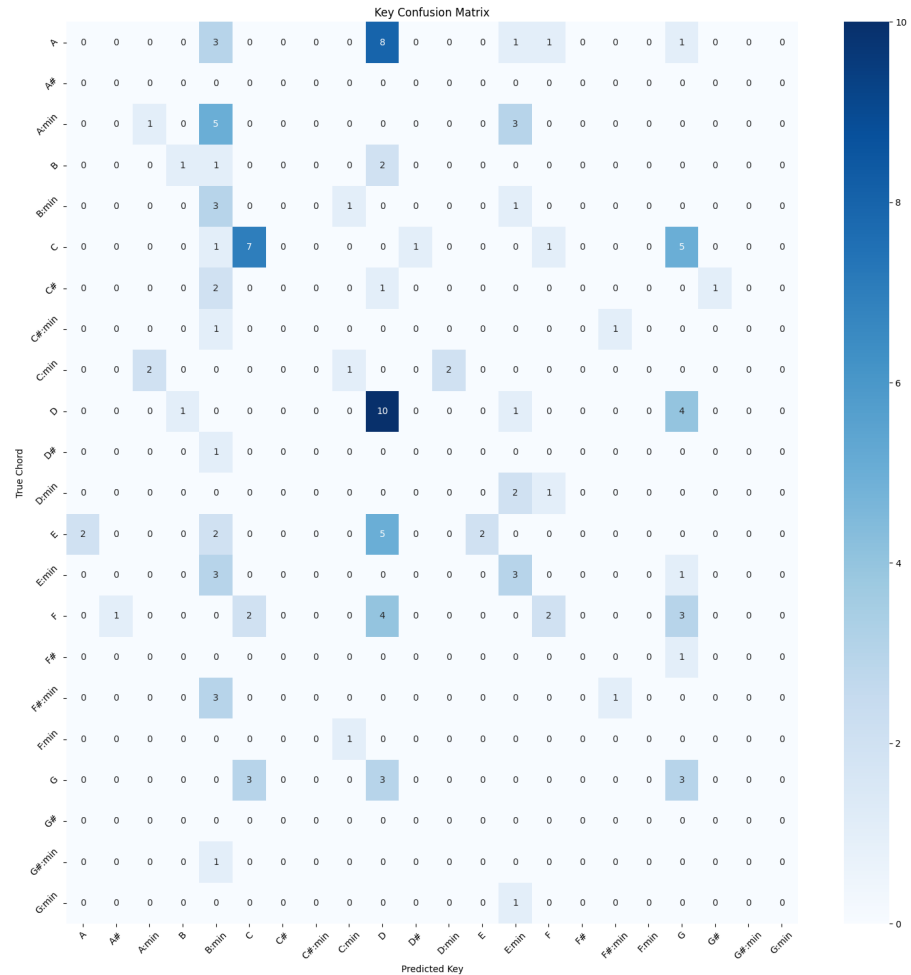


Figure 3.12: CNN Separated Audio – Strict – Mode-Aware

7. Semi-Strict, No Mode-Aware

(cnn_separated_semi_strict_no_major_minor)

- Correct Predictions: 34 / 125
- Accuracy: 27.20%

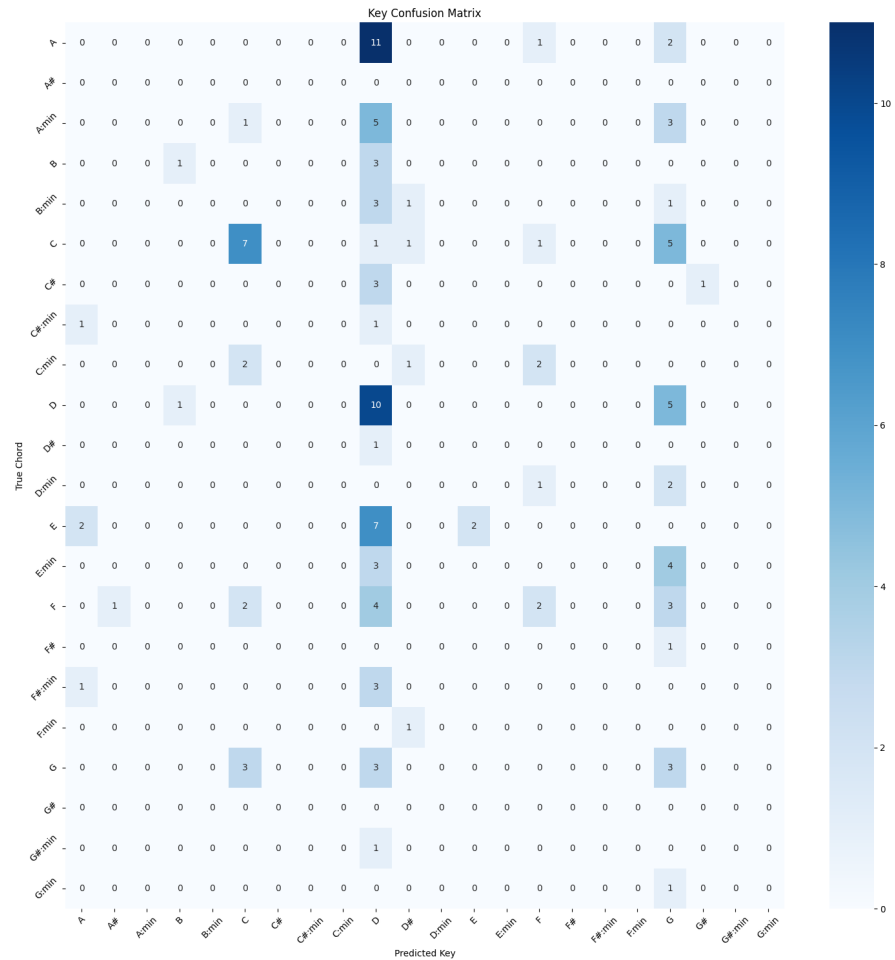


Figure 3.13: CNN Separated Audio – Semi-Strict – No Mode-Aware

8. Semi-Strict, Mode-Aware

(cnn_separated_semi_strict_with_major_minor)

- Correct Predictions: 37 / 125
- Accuracy: 29.60%

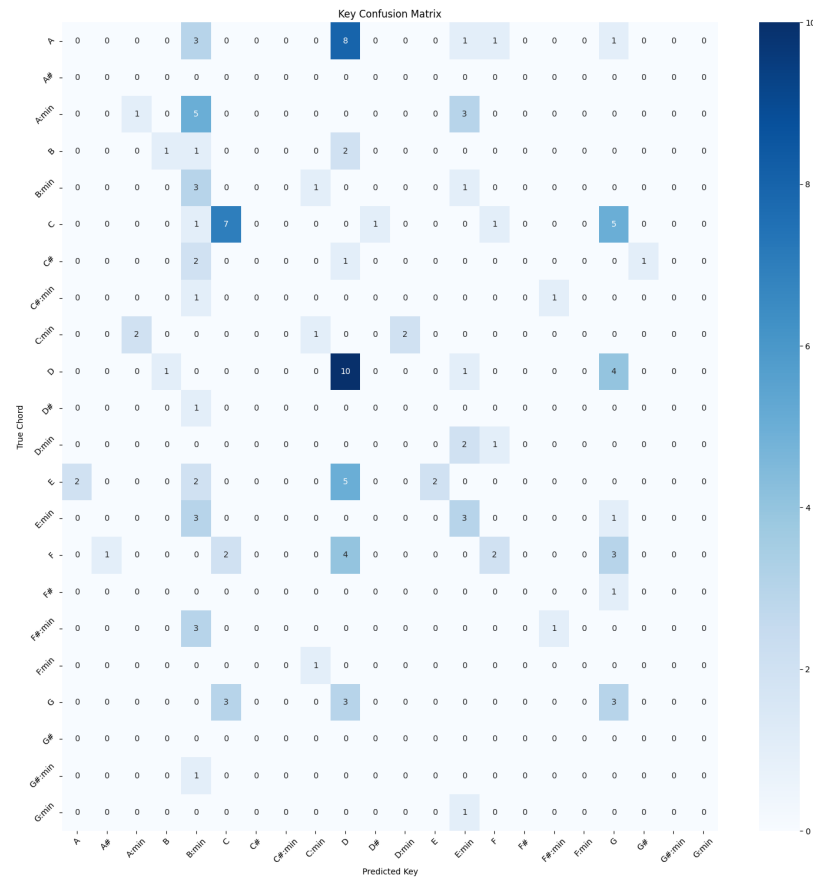


Figure 3.14: Separated Audio – Semi-Strict – Mode-Aware

❖ **CRNN – Original Audio**

9. **Strict, No Mode-Aware** (crnn_original_strict_no_major_minor)

- Correct Predictions: 73 / 125
- Accuracy: 58.40%

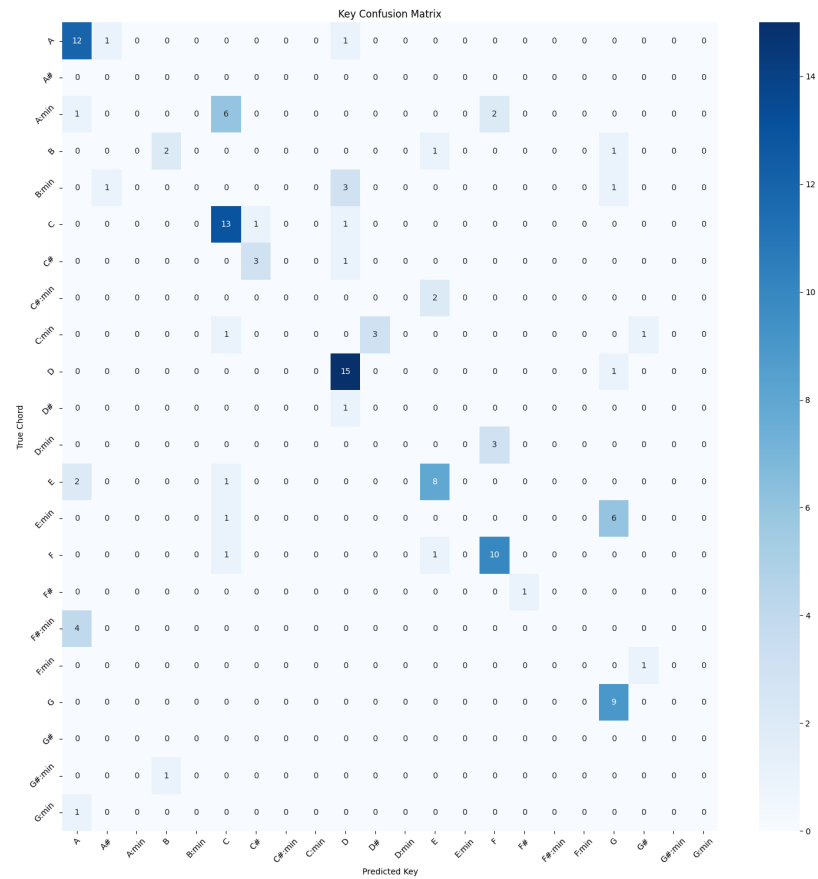


Figure 3.15: Original Audio – Strict – No Mode-Aware

10. Strict, Mode-Aware (crnn_original_strict_with_major_minor)

- Correct Predictions: 99 / 125
- Accuracy: 79.20%

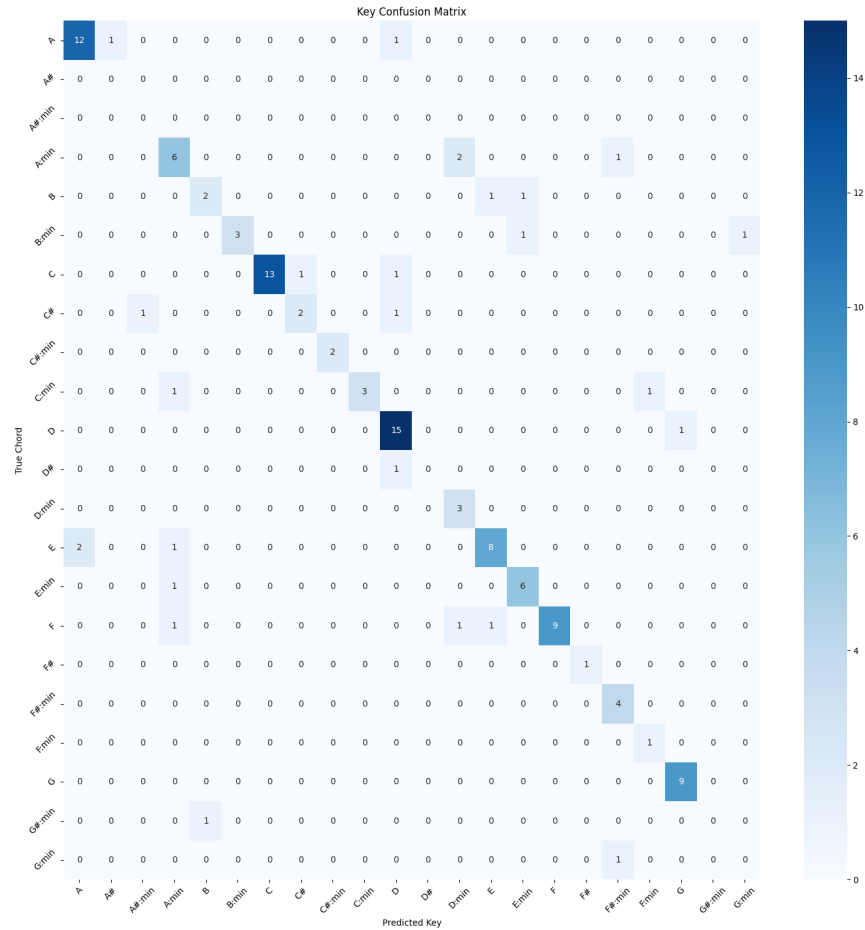


Figure 3.16: CRNN Original Audio – Strict – Mode-Aware

11. Semi-Strict, No Mode-Aware

(crnn_original_semi_strict_no_major_minor)

- Correct Predictions: 81 / 125
- Accuracy: 64.80%

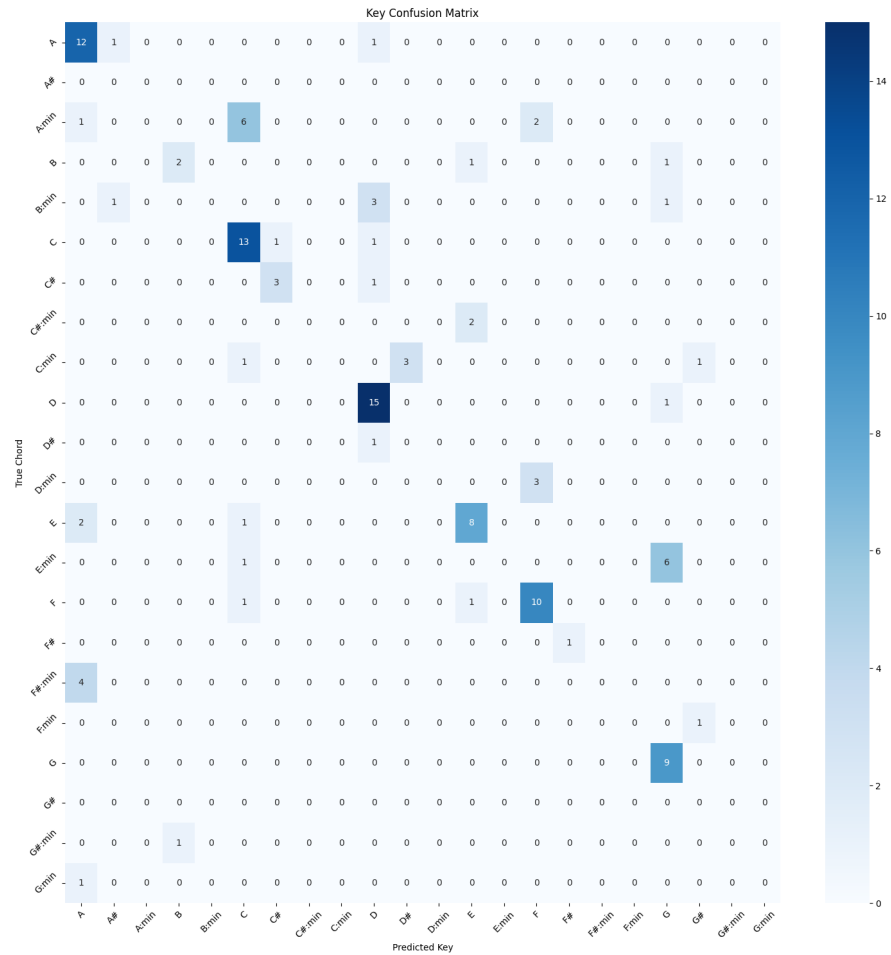


Figure 3.17: CRNN Original Audio – Semi-Strict – No Mode-Aware

12. Semi-Strict, Mode-Aware

(crnn_original_semi_strict_with_major_minor)

- Correct Predictions: 105 / 125
- Accuracy: 84.00%

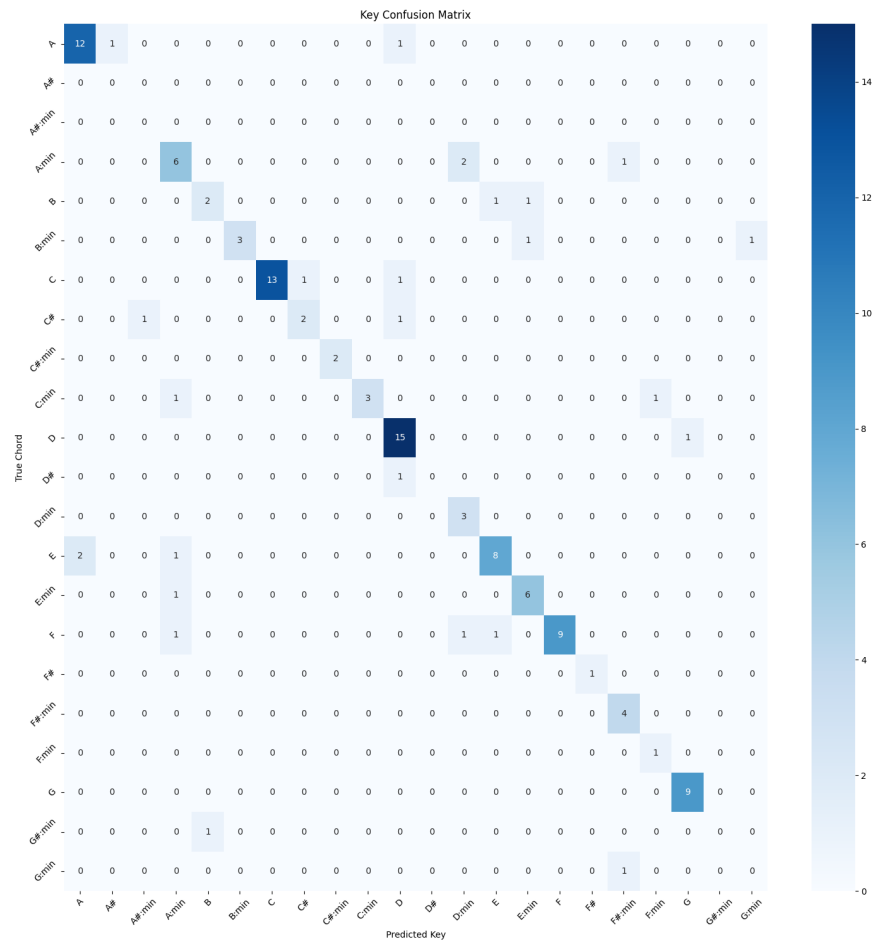


Figure 3.18: CRNN Original Audio – Semi-Strict – Mode-Aware

❖ CRNN – Separated Audio

13. Strict, No Mode-Aware (crnn_separated_strict_no_major_minor)

- Correct Predictions: 48 / 125
- Accuracy: 38.40%

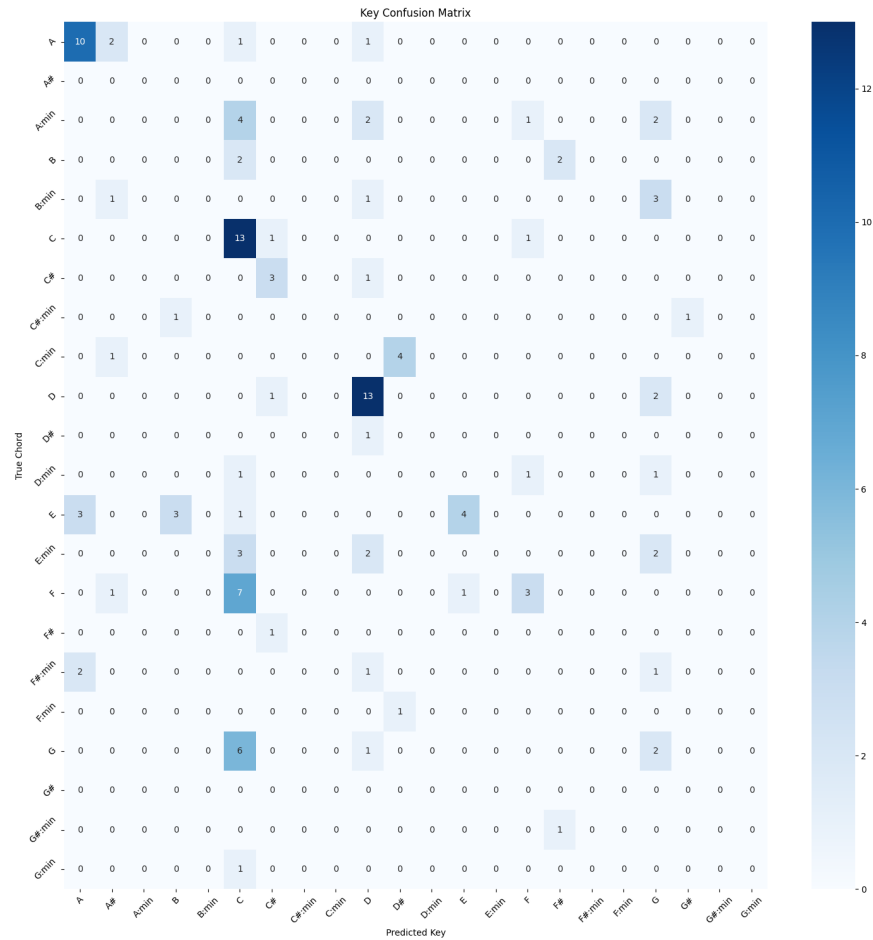


Figure 3.19: CRNN Separated Audio – Strict – No Mode-Aware

14. Strict, Mode-Aware (crnn_separated_strict_with_major_minor)

- Correct Predictions: 55 / 125
- Accuracy: 44.00%

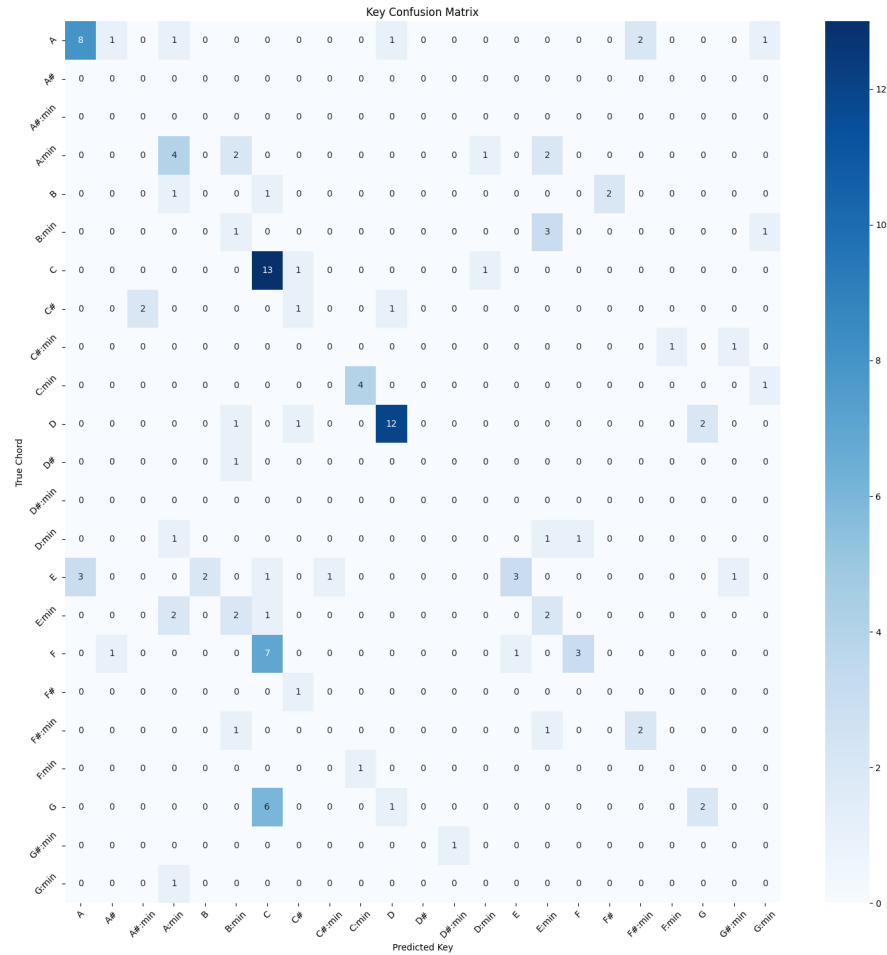


Figure 3.20: CRNN Separated Audio – Strict – Mode-Aware

15. Semi-Strict, No Mode-Aware

(crnn_separated_semi_strict_no_major_minor)

- Correct Predictions: 59 / 125
- Accuracy: 47.20%

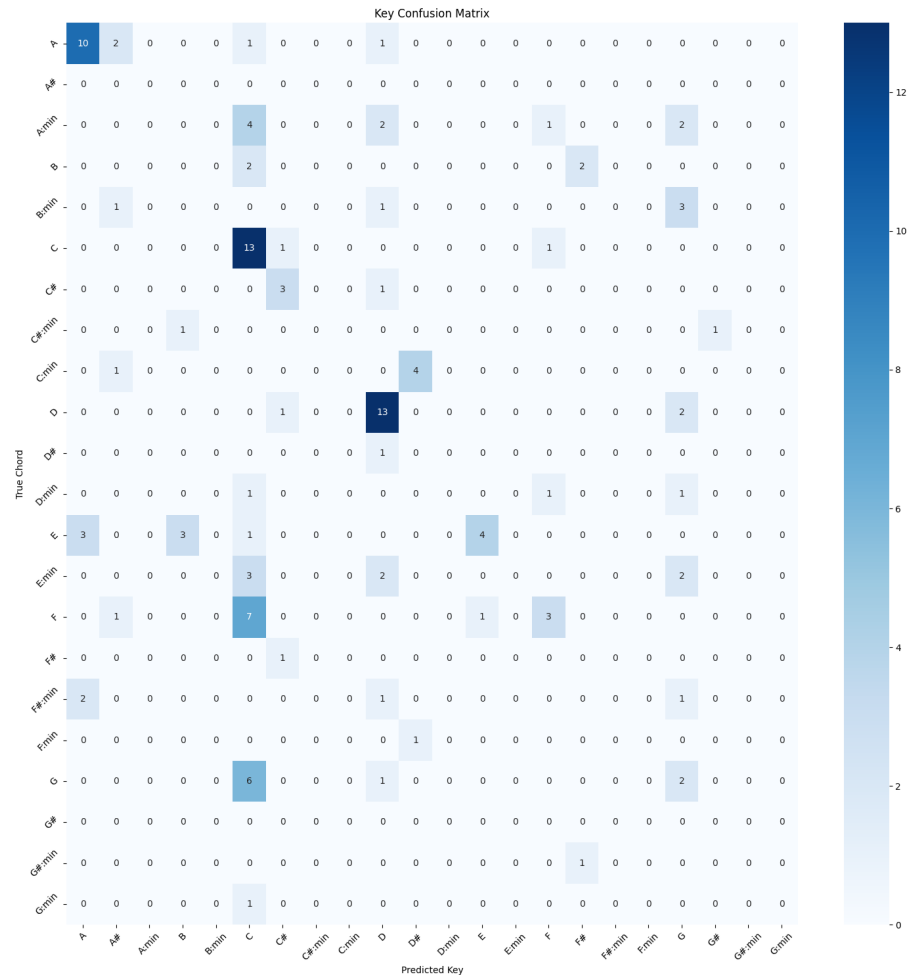


Figure 3.21: CRNN Separated Audio – Semi-Strict – No Mode-Aware

16. Semi-Strict, Mode-Aware

(crnn_separated_semi_strict_with_major_minor)

- Correct Predictions: 61 / 125
- Accuracy: 48.80%

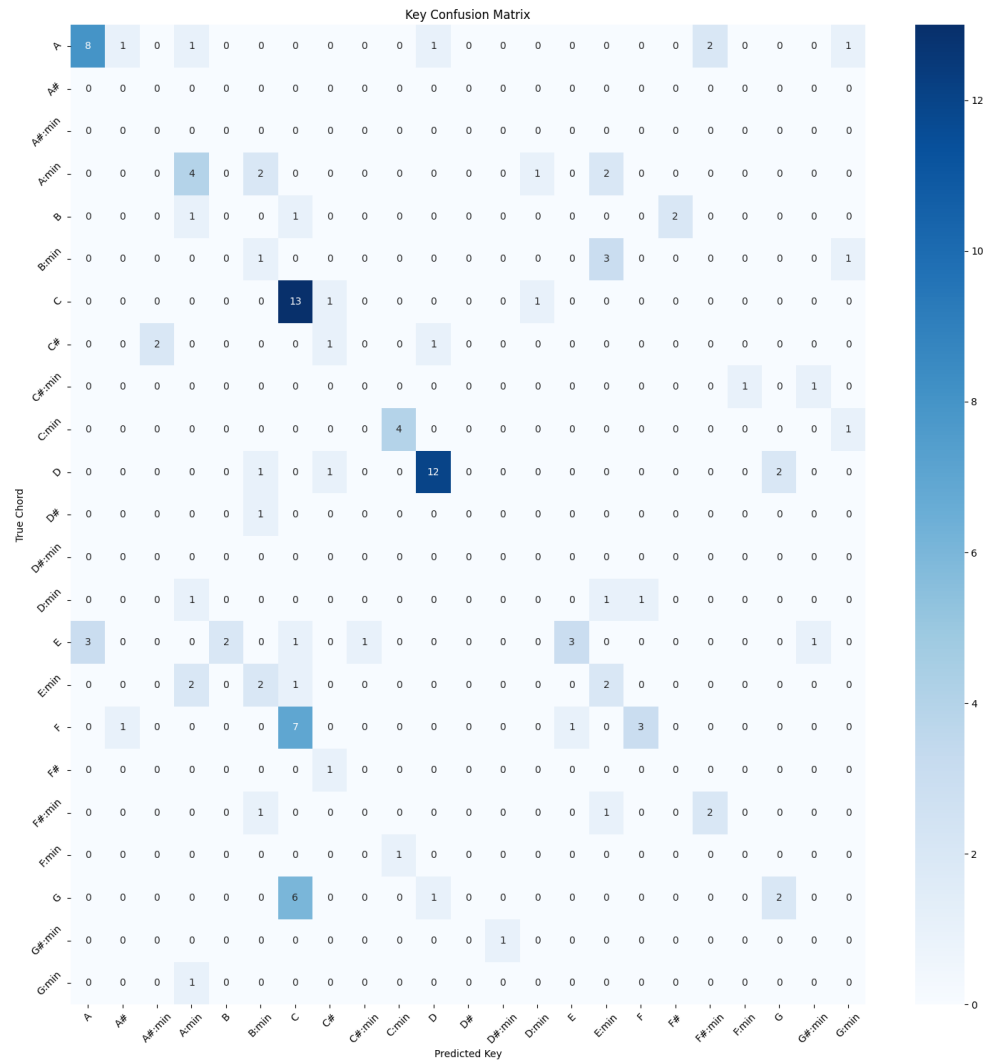


Figure 3.22: CRNN Separated Audio – Semi-Strict – Mode-Aware

❖ Discussion and Comparative Analysis

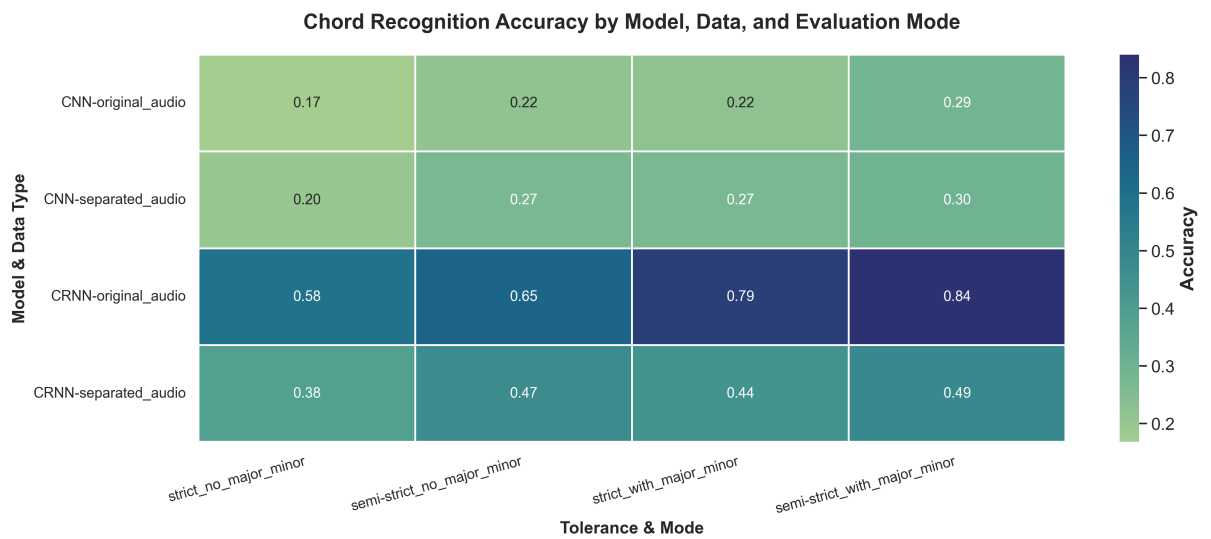


Figure 3.23: Accuracy heatmap across 16 evaluation configurations

To analyze the overall behavior of the system, we compare the performance across 16 evaluation conditions, spanning all combinations of model type (CNN vs. CRNN), input format (original vs. separated audio), and evaluation configuration (tolerance and mode-awareness). The heatmap in Figure 3.6 summarizes tonic chord recognition accuracy across these settings, revealing clear trends and significant insights.

❖ CNN vs. CRNN: Effect of Temporal Modeling

The most prominent pattern is the consistent superiority of the CRNN over the CNN. In all evaluation modes, the CRNN outperforms the CNN by a substantial margin — for example, under the strict mode-unaware setting, the CRNN scores 58% accuracy, compared to just 17% for the CNN. This trend is especially pronounced under more realistic evaluation regimes (e.g. mode-aware or semi-strict), where the CRNN approaches or surpasses 80% accuracy, while the CNN remains below 30%.

This discrepancy stems from architectural differences. The CNN operates frame-by-frame without temporal context, often misinterpreting short-term harmonic signals or confusing transient chords. In contrast, the CRNN incorporates bidirectional GRU layers, allowing it to track chord sequences over time and make more musically informed decisions. This temporal modeling improves robustness against ambiguities, such as short modulations or deceptive cadences, and explains the CRNN’s markedly better performance in predicting the global tonic chord.

❖ Impact of Mode-Aware Evaluation

A critical factor in chord recognition is the ambiguity between relative major and minor keys, such as C major and A minor. These keys share identical pitch content (i.e. same key signature), making them difficult to distinguish based solely on harmonic material. This is a known issue in both music theory and automatic key estimation systems.

Under mode-unaware evaluation, predictions that confuse relative pairs are marked incorrect. However, in mode-aware evaluation, the system uses external mode estimation (from the Essentia library) to infer the likely major or minor modality of each test song. If a song is detected as being in a minor mode, then a prediction like "C" is mapped to "A:min" (its relative minor), and vice versa.

This relative key mapping mechanism drastically improves evaluation fairness. In our experiments, switching from strict (mode-unaware) to strict (mode-aware) boosts CRNN accuracy from 58% to 79%, and semi-strict mode-aware even reaches 84%. This means that many of the CRNN's errors were just mode flips, not true misclassifications. The CNN also benefits slightly from mode-awareness, but the improvement is much smaller — again indicating that it makes more fundamental mistakes, not just relative key swaps.

❖ Effect of Semi-Strict Tolerance (± 0.5 Semitone)

The semi-strict tolerance allows a predicted key to be off by ± 0.5 semitone and still be counted as correct. This reflects musical reality, where neighboring keys (e.g. C and C#) may be easily confused due to tuning variation, modulation, or ambiguous harmonic content.

In our evaluation, enabling semi-strict tolerance led to moderate improvements for both models. For instance, CRNN original audio accuracy improves from 79% to 84% when tolerance is enabled alongside mode-awareness. This suggests that a small number of errors were near misses — often off by just one chromatic step — and thus arguably musically acceptable.

Importantly, the tolerance mechanism preserves scale type integrity: a predicted major key must still be within ± 1 semitone of the true major key (and similarly for minor keys). The tolerance respects the structure of the key cycle:

Major key sequence: $C \rightarrow C\# \rightarrow D \rightarrow D\# \rightarrow E \rightarrow F \rightarrow F\# \rightarrow G \rightarrow G\# \rightarrow A \rightarrow A\# \rightarrow B \rightarrow$ (wraps to C)

Minor key sequence: $A:\text{min} \rightarrow A\#:\text{min} \rightarrow B:\text{min} \rightarrow C:\text{min} \rightarrow \dots \rightarrow G\#:\text{min}$

Thus, if the true key is D, then C#, D, or D# are considered correct. For D:min, acceptable predictions include C#:min, D:min, and D#:min.

❖ **Influence of Input Type: Original vs. Separated Audio**

We also tested each model on vocal-separated audio, produced via source separation. This experiment aimed to evaluate whether removing instrumental components might enhance clarity in the chord signal and improve tonic estimation.

However, results show that using separated audio consistently yields lower accuracy across all settings. For example, CRNN with separated audio achieves only 49% accuracy (semi-strict, mode-aware), compared to 84% on original audio. This suggests that source separation introduced artifacts or degraded harmonic content, possibly distorting chordal structure or removing relevant frequency cues.

This result is important: naively applying source separation as a preprocessing step may harm chord recognition, especially when the model has been trained on full-spectrum audio. The quality of separation — particularly for complex timbres — may not preserve harmonic integrity well enough for tasks like tonic estimation.

❖ **Summary and Interpretation**

The heatmap-based analysis supports several key conclusions:

- CRNN is decisively more accurate than CNN, due to its ability to model chord sequences over time.
- Mode-aware evaluation is essential to avoid unfairly penalizing predictions that confuse relative major/minor keys — a common and musically reasonable mistake.
- Semi-strict tolerance helps capture borderline cases, particularly when predictions are just off by a semitone.
- Original audio yields significantly better results than separated audio, reinforcing the importance of preserving full-spectrum harmonic content in recognition tasks.

Together, these insights confirm that temporal modeling (via CRNN) combined with music-theoretic evaluation strategies (mode-awareness and tolerance) leads to the most reliable tonic chord identification, especially in real-world, diverse musical datasets like Vietnamese pop music.

Conclusion

In this section, we summarize the key outcomes of the tonic chord recognition system, integrating insights from data preparation, model design, and comprehensive evaluation. Each component — from dataset construction to the final comparative heatmap — reveals critical findings that define the system’s performance, strengths, and limitations.

❖ Robust Data Design and Preprocessing

The system was trained and evaluated on two carefully curated datasets: the Isophonics collection (Western pop/rock) and a custom Vietnamese pop dataset. These were unified via a shared preprocessing pipeline involving Constant-Q Transform (CQT) feature extraction and frame-wise chord annotation alignment. Both datasets exhibited similar harmonic distributions ($\approx 70\%$ major, $\approx 30\%$ minor), ensuring statistical consistency and preventing distribution shift. The strong overlap in chord vocabulary and tonic frequencies (e.g. A, C, D, E) across corpora further reinforced model generalization.

❖ CNN: A Simple Yet Limited Baseline

The CNN model provided a baseline for frame-wise chord classification, leveraging local spectral patterns in CQT input. Its lightweight architecture enabled rapid convergence, and it achieved high training accuracy. However, the model lacked temporal awareness, which led to overfitting and poor generalization — particularly in identifying the tonic chord. Without temporal smoothing, it often produced noisy frame-level predictions, and was especially prone to confusions between relative keys. Across evaluation settings, CNN performance plateaued around 20–30% accuracy, confirming its limitations.

❖ CRNN: Temporal Modeling Unlocks Performance

The CRNN extended the CNN architecture by incorporating bidirectional GRU layers, enabling it to capture sequential dependencies in music. This temporal modeling proved crucial: the CRNN could recognize recurring chord patterns, enforce key consistency over time, and correct short-term misclassifications. On original audio under strict + mode-aware conditions, the CRNN achieved up to 79.20% accuracy — increasing to 84.00% under semi-strict evaluation. These results demonstrate that the CRNN not only memorizes chords but also understands musical structure, making it more suitable for robust tonic identification.

❖ Insights from the Evaluation Framework

The extended evaluation framework provided deep insight into model behavior under varied conditions:

- **Mode-aware evaluation** significantly improved accuracy for both models, especially CRNN. It resolved many errors where the model predicted a relative key (e.g., C instead of A:min), a well-known ambiguity in music theory.
- **Semi-strict tolerance** (± 0.5 semitone) captured borderline cases where the prediction was off by one tone step — typically due to ambiguous tonal centers or modulation. CRNN, in particular, benefited from this flexibility.
- **Original audio consistently outperformed separated audio**, indicating that full-spectrum harmonic content is essential for accurate chord modeling. Voice-separated input, while theoretically cleaner, likely distorted essential harmonic cues and led to degraded results.

❖ Overall Comparative Performance

Across all 16 experimental settings, CRNN clearly outperformed CNN — often by margins of 30–60% accuracy. The final heatmap demonstrates that CRNN’s best configuration (original audio, semi-strict tolerance, mode-aware) reached 84.00%, while the best CNN result under identical settings only reached 28.80%.

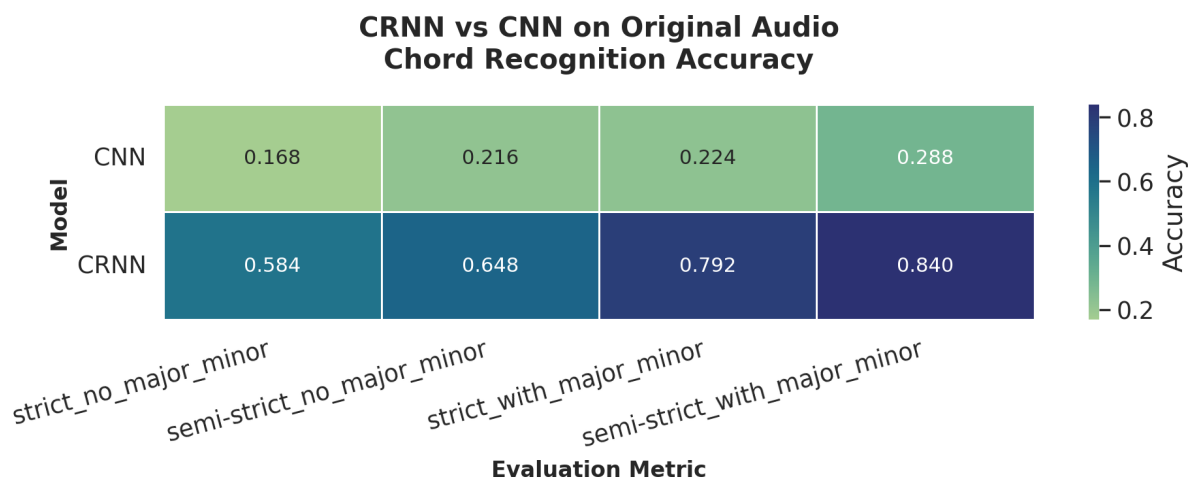


Figure 3.24: Direct accuracy comparison between CNN and CRNN

This stark contrast highlights the necessity of both temporal modeling and musically informed evaluation design in chord recognition systems. The CRNN architecture, guided by harmonic context and evaluated through tolerant and mode-sensitive metrics, provides state-of-the-art performance in recognizing tonic chords in diverse musical content.

3.1.2 Overview of the Vocal Separation Approach

Data Collection and Preprocessing

To develop and evaluate the vocal separation model, we utilized the MUSDB18 dataset, a widely adopted benchmark in the field of music source separation. This dataset is particularly suitable for training and assessing models aimed at isolating individual audio components from full music tracks.

The MUSDB18 dataset comprises a total of 150 high-quality full-length music tracks, each accompanied by four isolated audio stems:

- Vocals
- Drums
- Bass
- Other (includes instruments not classified in the above categories)

The original files are distributed in .mp4 format, each containing multiple audio streams for efficient separation.

To facilitate model training and evaluation, the dataset is partitioned into two distinct subsets:

- Training set: 100 songs (90% train, 10% validation)
- Test set: 50 songs

These are organized in a clear directory structure under musdb18_wav/ as follows:

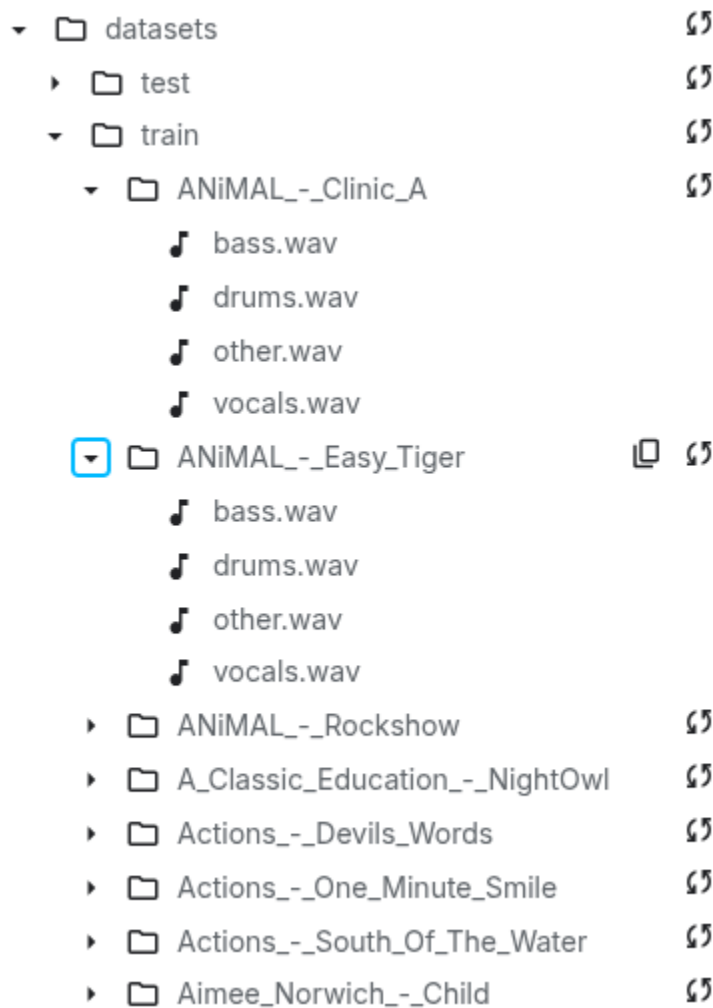


Figure 3.25: Data structure of musdb18 dataset

All audio files were converted to WAV format to ensure compatibility with the audio processing pipeline and deep learning frameworks. This structure and data split allow for consistent evaluation across studies and reproducibility of results.

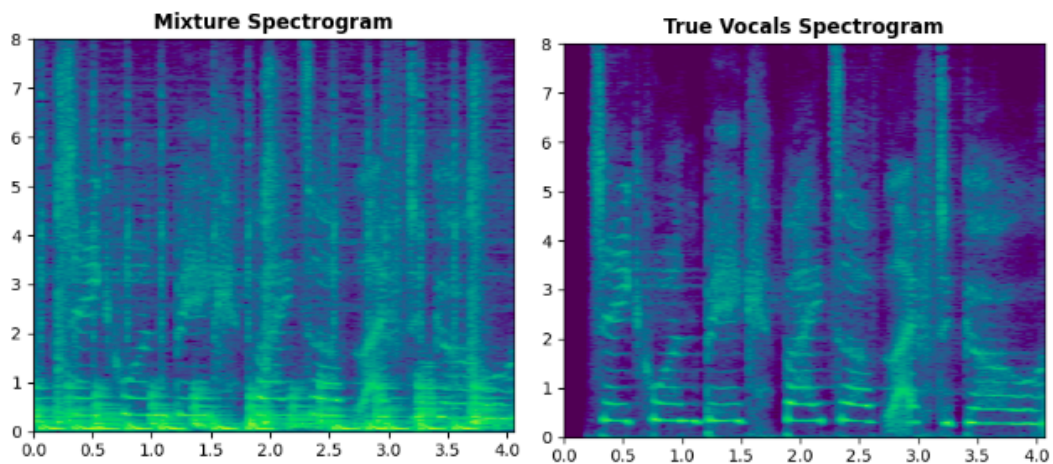


Figure 3.26: Spectrogram stft after process for vocal separation

To facilitate effective vocal separation, the audio data—including both the mixture tracks and their corresponding vocal stems—were transformed into the time-frequency domain using the Short-Time Fourier Transform (STFT). This transformation converts each waveform into a 2D spectrogram, where the horizontal axis represents time, the vertical axis represents frequency, and the intensity at each point reflects the signal's magnitude at a given frequency and time.

This representation is particularly valuable because it allows the model to perform separation in the spectral domain, where different sound sources (such as vocals, drums, and bass) tend to exhibit distinctive patterns. An illustration of the STFT spectrogram highlights the visible differences between the mixture and the isolated vocal component.

The resulting spectrograms are then used as direct input features and supervision targets for training. The model learns to predict a time-frequency mask to extract the vocal source from the mixture.

Notably, different input shapes were adopted depending on the complexity of the model:

- For the baseline UNet, input spectrograms were cropped to a shape of 128×128 (time $\times$ frequency), which is sufficient for learning local spectral patterns.
- In contrast, the TFC-TDF model requires more context along the time axis to fully leverage the capabilities of Time-Frequency Convolutions and Time-Distributed Fully-connected layers. Therefore, the input size was extended to 512×128 , allowing the model to learn broader temporal structures and harmonic dependencies across a longer time span.

This difference in input size reflects the trade-off between model simplicity and expressiveness, enabling the TFC-TDF model to perform more advanced spectral modeling for improved separation accuracy.

U-Net Model Design and Training

Overview of the UNet model after construction

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 128, 128, 1)	0	-
conv2d (Conv2D)	(None, 128, 128, 16)	160	input_layer[0][0]
leaky_re_lu (LeakyReLU)	(None, 128, 128, 16)	0	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 128, 128, 16)	2,320	leaky_re_lu[0][0]
leaky_re_lu_1 (LeakyReLU)	(None, 128, 128, 16)	0	conv2d_1[0][0]
max_pooling2d (MaxPooling2D)	(None, 64, 64, 16)	0	leaky_re_lu_1[0][0]
conv2d_2 (Conv2D)	(None, 64, 64, 32)	4,640	max_pooling2d[0][0]
leaky_re_lu_2 (LeakyReLU)	(None, 64, 64, 32)	0	conv2d_2[0][0]
conv2d_3 (Conv2D)	(None, 64, 64, 32)	9,248	leaky_re_lu_2[0][0]
leaky_re_lu_3 (LeakyReLU)	(None, 64, 64, 32)	0	conv2d_3[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 32)	0	leaky_re_lu_3[0][0]
conv2d_4 (Conv2D)	(None, 32, 32, 64)	18,496	max_pooling2d_1[0][0]
leaky_re_lu_4 (LeakyReLU)	(None, 32, 32, 64)	0	conv2d_4[0][0]
conv2d_5 (Conv2D)	(None, 32, 32, 64)	36,928	leaky_re_lu_4[0][0]
leaky_re_lu_5 (LeakyReLU)	(None, 32, 32, 64)	0	conv2d_5[0][0]
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 64)	0	leaky_re_lu_5[0][0]
conv2d_6 (Conv2D)	(None, 16, 16, 128)	73,856	max_pooling2d_2[0][0]
leaky_re_lu_6 (LeakyReLU)	(None, 16, 16, 128)	0	conv2d_6[0][0]
conv2d_7 (Conv2D)	(None, 16, 16, 128)	147,584	leaky_re_lu_6[0][0]
leaky_re_lu_7 (LeakyReLU)	(None, 16, 16, 128)	0	conv2d_7[0][0]

max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 128)	0	leaky_re_lu_7[0][0]
conv2d_8 (Conv2D)	(None, 8, 8, 256)	295,168	max_pooling2d_3[0][0]
leaky_re_lu_8 (LeakyReLU)	(None, 8, 8, 256)	0	conv2d_8[0][0]
conv2d_9 (Conv2D)	(None, 8, 8, 256)	590,080	leaky_re_lu_8[0][0]
leaky_re_lu_9 (LeakyReLU)	(None, 8, 8, 256)	0	conv2d_9[0][0]
up_sampling2d (UpSampling2D)	(None, 16, 16, 256)	0	leaky_re_lu_9[0][0]
concatenate (Concatenate)	(None, 16, 16, 384)	0	leaky_re_lu_7[0][0], up_sampling2d[0][0]
conv2d_10 (Conv2D)	(None, 16, 16, 128)	442,496	concatenate[0][0]
leaky_re_lu_10 (LeakyReLU)	(None, 16, 16, 128)	0	conv2d_10[0][0]
conv2d_11 (Conv2D)	(None, 16, 16, 128)	147,584	leaky_re_lu_10[0][0]
leaky_re_lu_11 (LeakyReLU)	(None, 16, 16, 128)	0	conv2d_11[0][0]
up_sampling2d_1 (UpSampling2D)	(None, 32, 32, 128)	0	leaky_re_lu_11[0][0]
concatenate_1 (Concatenate)	(None, 32, 32, 192)	0	leaky_re_lu_5[0][0], up_sampling2d_1[0][0]
conv2d_12 (Conv2D)	(None, 32, 32, 64)	110,656	concatenate_1[0][0]
leaky_re_lu_12 (LeakyReLU)	(None, 32, 32, 64)	0	conv2d_12[0][0]
conv2d_13 (Conv2D)	(None, 32, 32, 64)	36,928	leaky_re_lu_12[0][0]
leaky_re_lu_13 (LeakyReLU)	(None, 32, 32, 64)	0	conv2d_13[0][0]
up_sampling2d_2 (UpSampling2D)	(None, 64, 64, 64)	0	leaky_re_lu_13[0][0]

concatenate_2 (Concatenate)	(None, 64, 64, 96)	0	leaky_re_lu_3[0][0], up_sampling2d_2[0][0]
conv2d_14 (Conv2D)	(None, 64, 64, 32)	27,680	concatenate_2[0][0]
leaky_re_lu_14 (LeakyReLU)	(None, 64, 64, 32)	0	conv2d_14[0][0]
conv2d_15 (Conv2D)	(None, 64, 64, 32)	9,248	leaky_re_lu_14[0][0]
leaky_re_lu_15 (LeakyReLU)	(None, 64, 64, 32)	0	conv2d_15[0][0]
up_sampling2d_3 (UpSampling2D)	(None, 128, 128, 32)	0	leaky_re_lu_15[0][0]
concatenate_3 (Concatenate)	(None, 128, 128, 48)	0	leaky_re_lu_1[0][0], up_sampling2d_3[0][0]
conv2d_16 (Conv2D)	(None, 128, 128, 16)	6,928	concatenate_3[0][0]
leaky_re_lu_16 (LeakyReLU)	(None, 128, 128, 16)	0	conv2d_16[0][0]
conv2d_17 (Conv2D)	(None, 128, 128, 16)	2,320	leaky_re_lu_16[0][0]
leaky_re_lu_17 (LeakyReLU)	(None, 128, 128, 16)	0	conv2d_17[0][0]
conv2d_18 (Conv2D)	(None, 128, 128, 1)	17	leaky_re_lu_17[0][0]

Total params: 1,962,337 (7.49 MB)
Trainable params: 1,962,337 (7.49 MB)
Non-trainable params: 0 (0.00 B)

Figure 3.27: UNet model summary

The implemented model follows a classic UNet architecture, structured in an encoder–decoder fashion with skip connections. These connections preserve spatial details from the encoder layers and pass them to the corresponding decoder layers, aiding in the accurate reconstruction of the vocal spectrogram from the input mixture.

❖ **Model Structure**

- Input: A spectrogram of size (128, 128, 1) representing the mixture audio (vocals + accompaniment).
- Encoder: Consists of stacked blocks of Conv2D → LeakyReLU → MaxPooling2D, gradually reducing the spatial dimensions while increasing feature depth.
- Bottleneck: Deep layers with high channel counts (up to 512 filters) to learn abstract representations of the audio signal.
- Decoder: Uses UpSampling2D and convolutional layers to gradually reconstruct the output, with skip connections concatenated from encoder layers to enhance detail recovery.
- Output: A single-channel spectrogram representing the isolated vocal signal.

❖ **Training Configuration**

- Loss Function: Mean Absolute Error (MAE), suitable for regression-based signal reconstruction.
- Evaluation Metric: Signal-to-Distortion Ratio (SDR), which measures the quality of the separated output compared to the ground truth.
- Learning Rate: 5×10^{-5}
- Epochs: 30
- Training Time: Approximately 50–60 seconds per epoch

```
Epoch 1: val_sdr improved from -inf to 2.02372, saving model to /kaggle/working/model-001-2.023715.keras
500/500 — 59s 76ms/step - loss: 0.4221 - sdr: 1.0434 - val_loss: 0.3874 - val_sdr: 2.0237 - learning_rate: 5.0000e-05
Epoch 2/30
500/500 — 0s 68ms/step - loss: 0.3857 - sdr: 1.9838
Epoch 2: val_sdr improved from 2.02372 to 2.28206, saving model to /kaggle/working/model-002-2.282061.keras
500/500 — 37s 73ms/step - loss: 0.3857 - sdr: 1.9839 - val_loss: 0.3766 - val_sdr: 2.2821 - learning_rate: 5.0000e-05
Epoch 3/30
500/500 — 0s 73ms/step - loss: 0.3745 - sdr: 2.2380
Epoch 3: val_sdr improved from 2.28206 to 2.49122, saving model to /kaggle/working/model-003-2.491215.keras
500/500 — 39s 78ms/step - loss: 0.3745 - sdr: 2.2382 - val_loss: 0.3685 - val_sdr: 2.4912 - learning_rate: 5.0000e-05
Epoch 4/30
500/500 — 0s 74ms/step - loss: 0.3643 - sdr: 2.4698
Epoch 4: val_sdr improved from 2.49122 to 2.63897, saving model to /kaggle/working/model-004-2.638967.keras
500/500 — 40s 80ms/step - loss: 0.3643 - sdr: 2.4700 - val_loss: 0.3664 - val_sdr: 2.6390 - learning_rate: 5.0000e-05
Epoch 5/30
500/500 — 0s 73ms/step - loss: 0.3571 - sdr: 2.6269
Epoch 5: val_sdr improved from 2.63897 to 2.74666, saving model to /kaggle/working/model-005-2.746657.keras
500/500 — 39s 78ms/step - loss: 0.3571 - sdr: 2.6270 - val_loss: 0.3627 - val_sdr: 2.7467 - learning_rate: 5.0000e-05
Epoch 6/30
500/500 — 0s 74ms/step - loss: 0.3520 - sdr: 2.7397
Epoch 6: val_sdr improved from 2.74666 to 2.86818, saving model to /kaggle/working/model-006-2.868182.keras
500/500 — 40s 79ms/step - loss: 0.3520 - sdr: 2.7398 - val_loss: 0.3572 - val_sdr: 2.8682 - learning_rate: 5.0000e-05
Epoch 7/30
500/500 — 0s 73ms/step - loss: 0.3473 - sdr: 2.8462
```

Figure 3.28: UNet training process

❖ Training result



Figure 3.29: Loss and metric UNet training

❖ Loss and SDR Trends

- **Training Loss** steadily decreases from ~0.39 to ~0.24 over 30 epochs.
- **Validation Loss** plateaus after epoch 10, oscillating around 0.33–0.34, indicating early signs of **overfitting**.
- **Training SDR** improves consistently, reaching over 5.
- **Validation SDR**, however, stagnates around **3.1**, showing limited generalization to unseen data.

❖ Best Metrics Achieved

- Lowest Validation Loss: 0.333
- Highest Validation SDR: 3.141

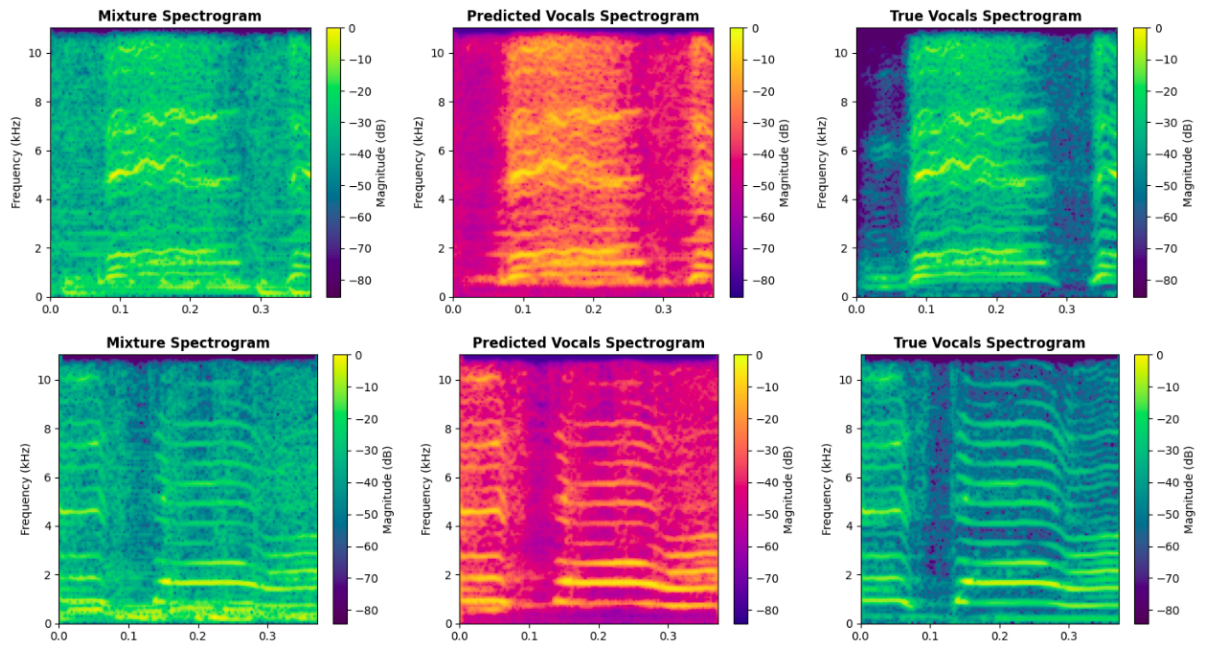


Figure 3.30: Statistics spectrogram of test data UNet model

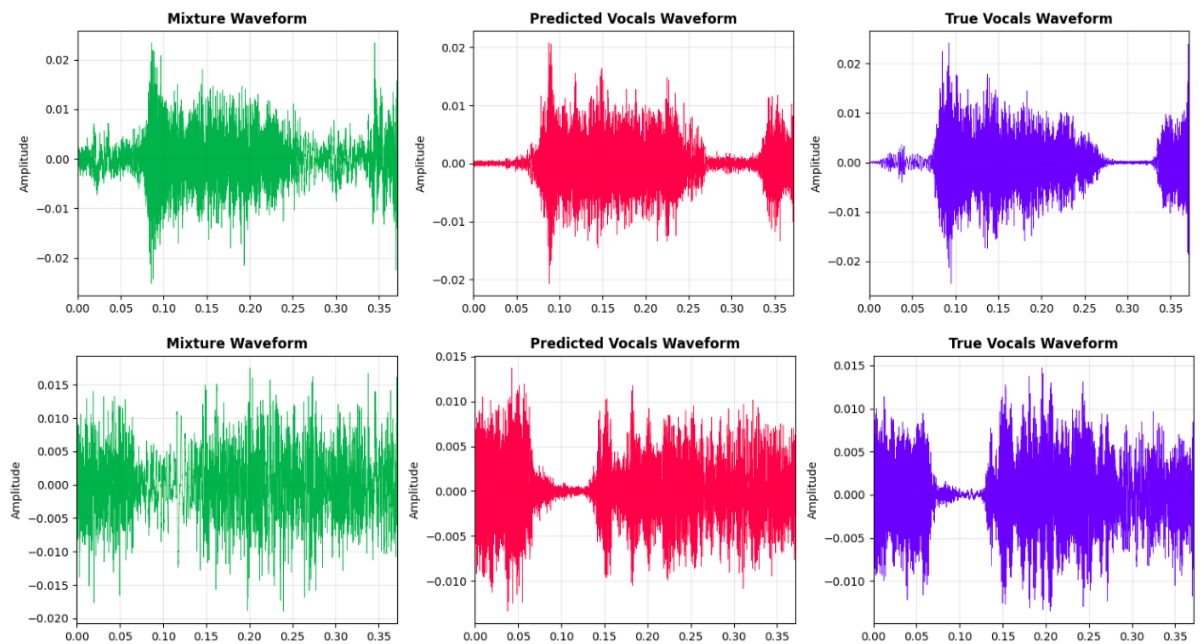


Figure 3.31: Statistics waveform of test data

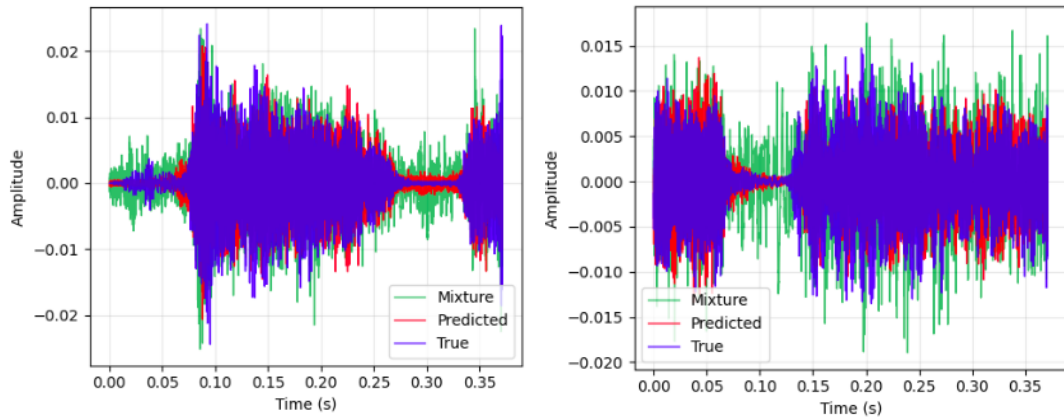


Figure 3.32: Compare predict and true data test UNet model

- **Mixture Spectrogram:** Displays a combination of vocals and instrumental harmonics.
- **Predicted Spectrogram:** While the vocal contours are partially extracted, residual accompaniment energy is still visible.
- **Ground Truth:** Clean and isolated vocal information, used as a reference target.

❖ Waveforms

- Predicted waveforms visually resemble the ground truth in amplitude and structure but still contain **noticeable interference**.
- Overlay plots of mixture, predicted, and ground truth signals reveal that although the model tracks vocal dynamics, **instrumental bleed-through** is present.
- Listening to the outputs confirms the **lack of clean separation**, with occasional **buzzing artifacts** and **distortion** in high-energy vocal segments.

TFD_TFC_U-Net Model Design and Training

Overview of the TFD_TFC_UNet model after construction

Model: "tfc_tdf_net"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 65024)	0	-
stft (STFT)	[(None, 128, 1025), (None, 128, 1025)]	0	input_layer[0][0]
expand_dims (ExpandDims)	(None, 128, 1025, 1)	0	stft[0][0]
expand_dims_1 (ExpandDims)	(None, 128, 1025, 1)	0	stft[0][1]
concatenate (Concatenate)	(None, 128, 1025, 2)	0	expand_dims[0][0], expand_dims_1[0][0]
split (Split)	[(None, 128, 1024, 2), (None, 128, 1, 2)]	0	concatenate[0][0]
reshape (Reshape)	(None, 128, 256, 8)	0	split[0][0]
conv2d (Conv2D)	(None, 128, 256, 64)	512	reshape[0][0]
time_frequency_transform... (TimeFrequencyTransformB...	(None, 128, 256, 64)	287,744	conv2d[0][0]
downscale (Downscale)	(None, 64, 128, 192)	49,536	time_frequency_transf...
time_frequency_transform... (TimeFrequencyTransformB...	(None, 64, 128, 192)	1,436,672	downscale[0][0]
downscale_1 (Downscale)	(None, 32, 64, 320)	246,400	time_frequency_transf...
time_frequency_transform... (TimeFrequencyTransformB...	(None, 32, 64, 320)	3,904,512	downscale_1[0][0]
downscale_2 (Downscale)	(None, 16, 32, 448)	574,336	time_frequency_transf...
time_frequency_transform... (TimeFrequencyTransformB...	(None, 16, 32, 448)	7,635,968	downscale_2[0][0]
downscale_3 (Downscale)	(None, 8, 16, 576)	1,033,344	time_frequency_transf...
time_frequency_transform... (TimeFrequencyTransformB...	(None, 8, 16, 576)	12,617,216	downscale_3[0][0]

upscale (Upscale)	(None, 16, 32, 448)	1,033,088	time_frequency_transf...
concatenate_1 (Concatenate)	(None, 16, 32, 896)	0	upscale[0][0], time_frequency_transf...
time_frequency_transform... (TimeFrequencyTransformB...	(None, 16, 32, 448)	15,065,600	concatenate_1[0][0]
upscale_1 (Upscale)	(None, 32, 64, 320)	574,080	time_frequency_transf...
concatenate_2 (Concatenate)	(None, 32, 64, 640)	0	upscale_1[0][0], time_frequency_transf...
time_frequency_transform... (TimeFrequencyTransformB...	(None, 32, 64, 320)	7,695,872	concatenate_2[0][0]
upscale_2 (Upscale)	(None, 64, 128, 192)	246,144	time_frequency_transf...
concatenate_3 (Concatenate)	(None, 64, 128, 384)	0	upscale_2[0][0], time_frequency_transf...
time_frequency_transform... (TimeFrequencyTransformB...	(None, 64, 128, 192)	2,802,176	concatenate_3[0][0]
upscale_3 (Upscale)	(None, 128, 256, 64)	49,280	time_frequency_transf...
concatenate_4 (Concatenate)	(None, 128, 256, 128)	0	upscale_3[0][0], time_frequency_transf...
time_frequency_transform... (TimeFrequencyTransformB...	(None, 128, 256, 64)	439,808	concatenate_4[0][0]
multiply (Multiply)	(None, 128, 256, 64)	0	time_frequency_transf... conv2d[0][0]
concatenate_5 (Concatenate)	(None, 128, 256, 72)	0	reshape[0][0], multiply[0][0]
conv2d_59 (Conv2D)	(None, 128, 256, 64)	4,608	concatenate_5[0][0]
conv2d_59 (Conv2D)	(None, 128, 256, 64)	4,608	concatenate_5[0][0]
conv2d_60 (Conv2D)	(None, 128, 256, 8)	512	conv2d_59[0][0]
reshape_1 (Reshape)	(None, 128, 1024, 1, 2)	0	conv2d_60[0][0]
transpose (Transpose)	(None, 1, 128, 1024, 2)	0	reshape_1[0][0]
reshape_2 (Reshape)	(None, 1, 128, 2048)	0	transpose[0][0]

Total params: 222,789,634 (849.88 MB)

Trainable params: 55,697,408 (212.47 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 167,092,226 (637.41 MB)

Figure 3.33: TFD_TFC_UNet model summary

The TFC-TDF-UNet model represents an advanced design combining Time-Frequency Convolution (TFC) and Time-Distributed Fully-connected (TDF) blocks. This hybrid architecture is specifically optimized to model long-range temporal and spectral dependencies crucial for audio source separation tasks, such as vocal isolation.

❖ Model Architecture Overview

Input Processing: The input audio waveform is first transformed into a time-frequency representation using Short-Time Fourier Transform (STFT), resulting in a 2-branch complex-valued spectrogram.

Channel Expansion & Concatenation: Both real and imaginary parts are expanded and concatenated, forming a 3D tensor input for the convolutional pipeline.

Encoder (Downscaling Path):

- Begins with a Conv2D layer followed by a stack of TFC + TDF blocks and Downscale layers, progressively shrinking spatial dimensions while increasing channel depth (up to 576 channels).
- Each TFC block captures local time-frequency patterns, while TDF blocks extract long-range dependencies.

Decoder (Upscaling Path):

- The decoder uses Upscale layers combined with skip connections and additional TFC-TDF blocks, gradually restoring spatial resolution.
- Concatenation at each level helps preserve detail lost during downsampling.

Output Mapping:

- The model uses multiple convolutional and reshaping layers to reconstruct the estimated vocal spectrogram.
- Finally, the output is mapped back to a waveform through inverse STFT.

```
Epoch 1/10
2000/2000 ————— 1587s 760ms/step - loss: 0.2877 - sdr: 6.2838 - val_loss: 0.1685 - val_sdr: 6.8153 - learning_rate: 5.0000e-05
Epoch 2/10
2000/2000 ————— 1494s 733ms/step - loss: 0.2885 - sdr: 6.2729 - val_loss: 0.1684 - val_sdr: 6.7644 - learning_rate: 5.0000e-05
Epoch 3/10
2000/2000 ————— 1460s 730ms/step - loss: 0.2858 - sdr: 6.2892 - val_loss: 0.1703 - val_sdr: 6.7526 - learning_rate: 5.0000e-05
Epoch 4/10
2000/2000 ————— 1482s 727ms/step - loss: 0.2847 - sdr: 6.2745 - val_loss: 0.1585 - val_sdr: 7.0680 - learning_rate: 4.7500e-05
Epoch 5/10
2000/2000 ————— 1444s 722ms/step - loss: 0.2853 - sdr: 6.4071 - val_loss: 0.1678 - val_sdr: 6.8477 - learning_rate: 4.7500e-05
Epoch 6/10
2000/2000 ————— 1442s 721ms/step - loss: 0.2850 - sdr: 6.4833 - val_loss: 0.1687 - val_sdr: 6.7754 - learning_rate: 4.7500e-05
Epoch 7/10
2000/2000 ————— 1444s 722ms/step - loss: 0.2841 - sdr: 6.5749 - val_loss: 0.1634 - val_sdr: 6.8355 - learning_rate: 4.5125e-05
Epoch 8/10
2000/2000 ————— 1438s 719ms/step - loss: 0.2855 - sdr: 6.5111 - val_loss: 0.1723 - val_sdr: 6.5136 - learning_rate: 4.5125e-05
Epoch 9/10
2000/2000 ————— 1438s 719ms/step - loss: 0.2859 - sdr: 6.6142 - val_loss: 0.1633 - val_sdr: 6.7805 - learning_rate: 4.2869e-05
Epoch 10/10
```

Figure 3.34: TFD_TFC_UNet training process

❖ **Training results**



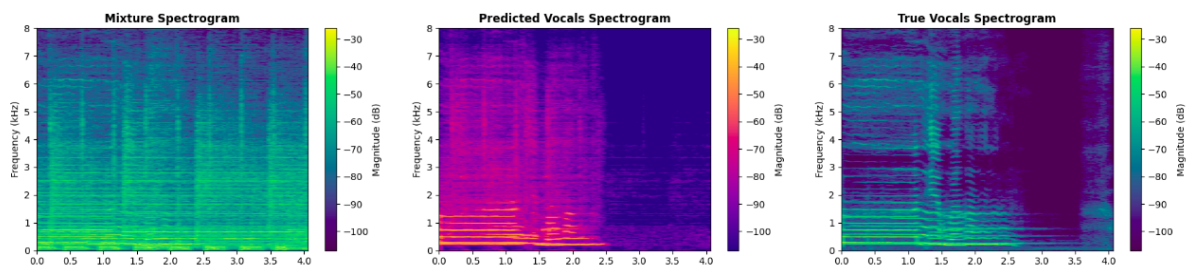
Figure 3.35: Loss and metric TFD_TFC_UNet training

❖ **Loss and SDR Trends**

- The training and validation loss curves show consistent convergence, with validation loss stabilizing below 0.2, a strong indicator of good generalization.
- SDR curves show a rapid initial increase followed by steady improvement, with validation SDR consistently outperforming training SDR, suggesting successful learning without overfitting.

❖ **Best Metrics Achieved**

- Lowest Validation Loss: 0.28
- Highest Validation SDR: 6.53



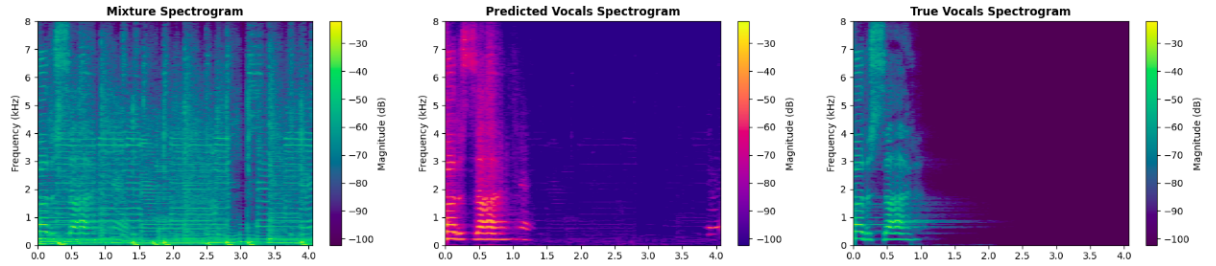


Figure 3.36: Statistics spectrogram of test data metric TFD_TFC_UNet

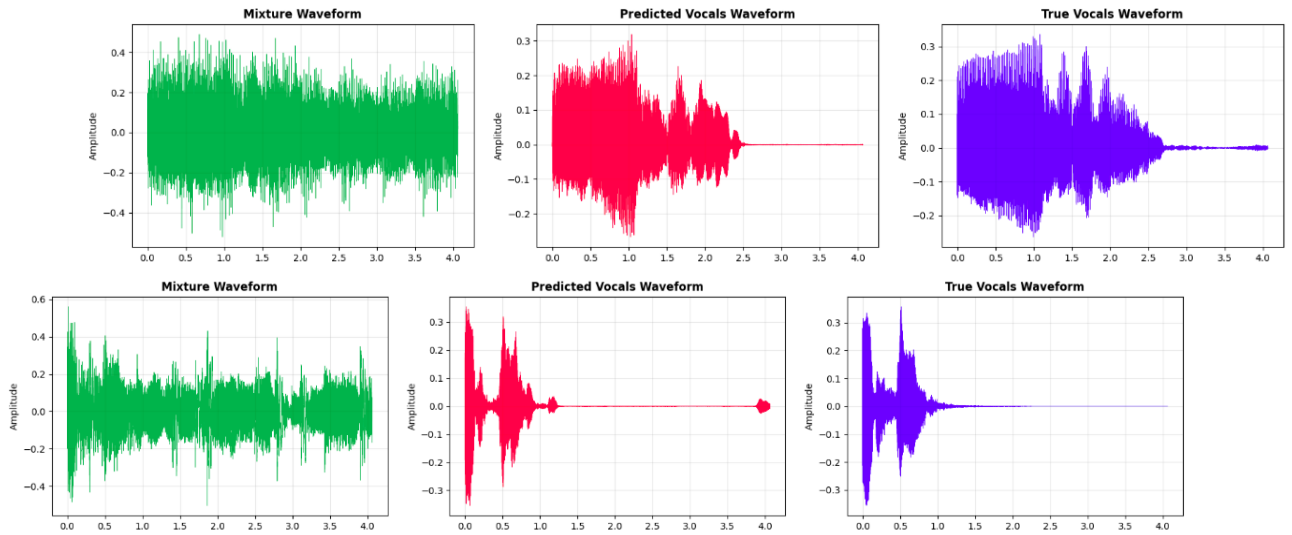


Figure 3.37: Statistics waveform of test data metric TFD_TFC_UNet model

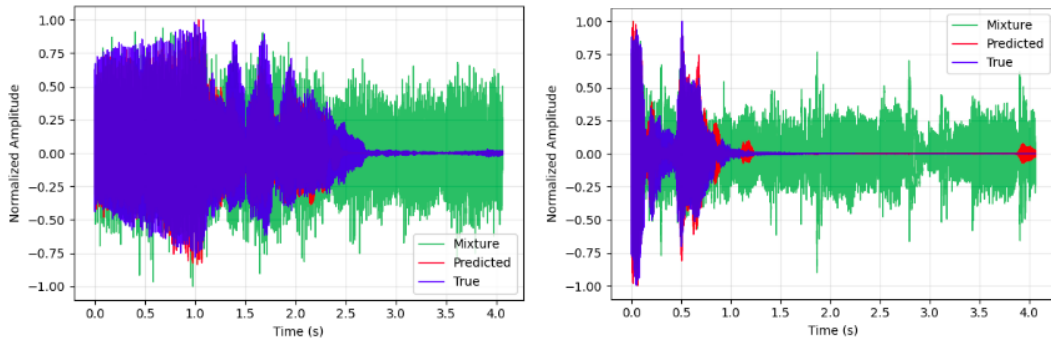


Figure 3.38: Compare predict and true data test TFD_TFC_UNet model

❖ Spectrogram

- **Mixture Spectrograms** display overlapping frequency content from both vocals and instruments.
- **Predicted Vocal Spectrograms** generated by TFC-TDF-UNet are highly aligned with the **ground truth**, showing strong energy in vocal regions and notable suppression of background components.
- Compared to basic UNet models, TFC-TDF-UNet preserves finer spectral structure, especially in high-frequency regions.

❖ **Waveform**

- **Predicted vocal waveforms** are temporally aligned and exhibit clean separation, with minimal residual noise or artifacts.
- In overlap plots, the predicted waveform closely follows the true vocal waveform, with visibly reduced interference from the mixture.
- **Auditory quality:** The output sound is much cleaner and more stable than earlier models, with no noticeable buzzing or distortion. The clarity and fidelity of vocals are preserved effectively.

Conclusion

Table 3.7: Compare UNet with TFC_TDF_UNet

	Loss	SDR train	SDR test
UNet	0.33	3.14	-0.02
TFC_TDF_UNet	0.28	6.53	0.7

The experimental results demonstrate that the proposed TFC_TDF_UNet model outperforms the baseline UNet in both training and testing performance. It achieves a lower loss (0.28 vs. 0.33) and significantly improves the SDR on the test set (0.70 dB vs. -0.02 dB), indicating better generalization and separation quality. These results confirm that integrating Time-Frequency Convolutions and Time-Distributed Fully-connected layers enhances the model's ability to isolate vocals from mixtures effectively.

3.2 Implementing the system

3.2.1 Technology stack

Programming Languages and Frameworks

- **Backend:**
 - **Language:** Python 3.10
 - **Framework:** FastAPI – a modern, high-performance web framework for building APIs with Python based on standard Python type hints.
- **Frontend:**
 - **Languages:** JavaScript, HTML, CSS
 - **Framework:** VueJS – a progressive JavaScript framework used to build interactive and modular single-page applications.

- **Storage Services**
 - **Amazon S3 (Simple Storage Service)** is used as the object storage solution for the system. It hosts:
 - Static frontend web assets (HTML/CSS/JS).
 - User-uploaded audio files.
 - Output results (e.g., separated vocal/accompaniment files, chord timelines).
- **Containerization**
 - **Docker** is used to package both frontend and backend components into portable containers. Each service is isolated for scalability and ease of deployment across different environments.
- **Deployment**
 - The system is deployed on Amazon Web Services (AWS).
 - Components are distributed across multiple services:
 - **Frontend** is hosted as a static website via Amazon S3 + CloudFront CDN.
 - **Backend API** is served from an EC2 t2.medium instance.
 - **AI Model Inference Server** is deployed separately on a t3.xlarge EC2 instance for better compute and memory performance.

3.2.2 Deployment Enviroment

- **System Specifications (EC2 Instances)**

Table 3.8: System Specifications (EC2 Instances)

Component	Instance Type	vCPUs	RAM	Role
API Server	t2.medium	2	4 GiB	Hosting backend FastAPI service
AI Inference Server	t3.xlarge	4	16 GiB	Running CRNN and UNet models

- **Operating System**
 - **Ubuntu Server 22.04 LTS (64-bit)**
- **Compute & Network**
 - **CPU Type:** Intel Xeon Scalable or AMD EPYC (varies by region)
 - **Transfer Bandwidth:** Baseline 5 Gbps (burstable)
 - **Storage:** EBS-backed volumes (gp2/gp3), configurable capacity

3.2.3 Domain Registration

❖ Domain Management

The domain [harmonyseeker.site](https://tenten.vn) is registered and managed through [Tenten.vn](https://tenten.vn), a domain registration service. DNS records are manually configured within the Tenten DNS management interface to associate each subdomain with the corresponding AWS infrastructure components.

❖ DNS Mapping and Subdomains

The following subdomains are defined and configured to route traffic to specific backend services:

Table 3.9: DNS Mapping and Subdomains

Subdomain	Description	Target Infrastructure
www.harmonyseeker.site	Frontend web application	Hosted on Amazon S3 + CloudFront
api.harmonyseeker.site	Backend API service	EC2 instance running FastAPI
ai.harmonyseeker.site	AI inference service	EC2 instance running model servers

DNS A records or CNAME records were used to point to the appropriate AWS public IPs or CloudFront endpoints. Proper HTTPS support is enabled using SSL certificates managed by AWS Certificate Manager (ACM).

3.2.4 System Access

- **Frontend Website:** <https://www.harmonyseeker.site/>
Users can upload audio, view results, and interact with the application via this SPA interface.
- **API Documentation (Backend Service):**
<https://api.harmonyseeker.site/docs>
This serves as the official OpenAPI documentation for interacting with the backend.
- **AI Inference Endpoint Documentation:**
<https://ai.harmonyseeker.site/docs>
Used by developers or internal systems to call chord recognition and vocal separation models directly.

3.4 Results

3.4.1 Home screen

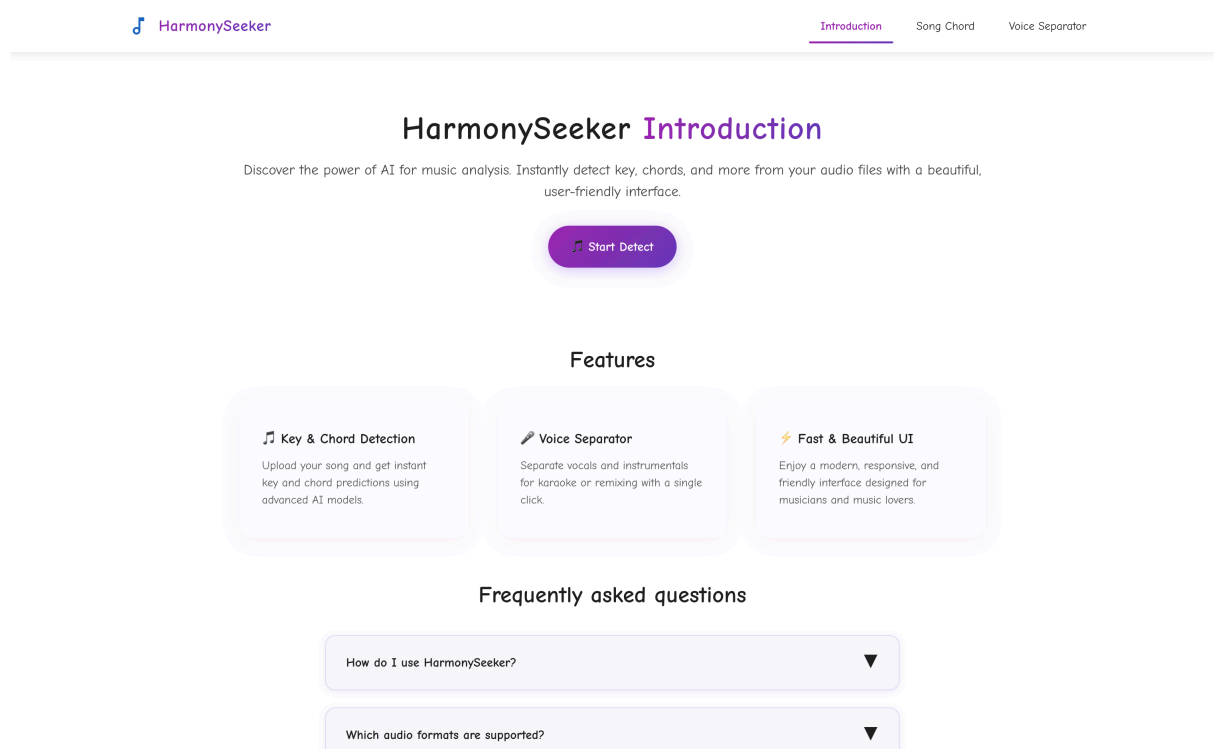


Figure 3.39: Home page of HarmonySeeker

When the user visits the application, they are first greeted by the homepage. This screen highlights the main features of the HarmonySeeker system: key & chord detection, vocal separation, and a fast, beautiful interface. From here, users can click the "Start Detect" button to begin uploading a music file for analysis.

3.4.2 Key Chord Detection Screen

❖ Main Detection Interface

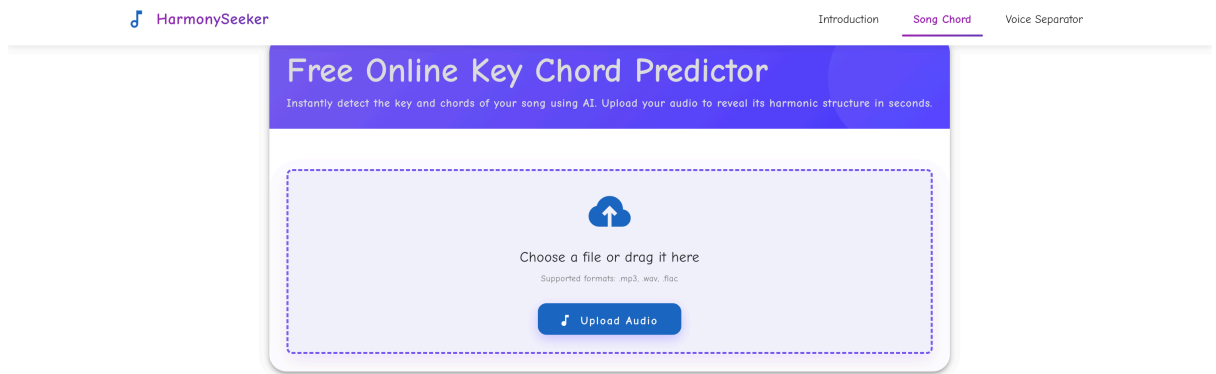


Figure 3.40: Main layout of the Key Detection page

After clicking on the "Song Chord" tab, the user is redirected to the chord recognition screen. The layout prominently displays the predicted key and visualizes the Circle of Fifths, providing an intuitive music-theoretical context for understanding key relationships.

❖ **Upload Success (File < 10MB)**

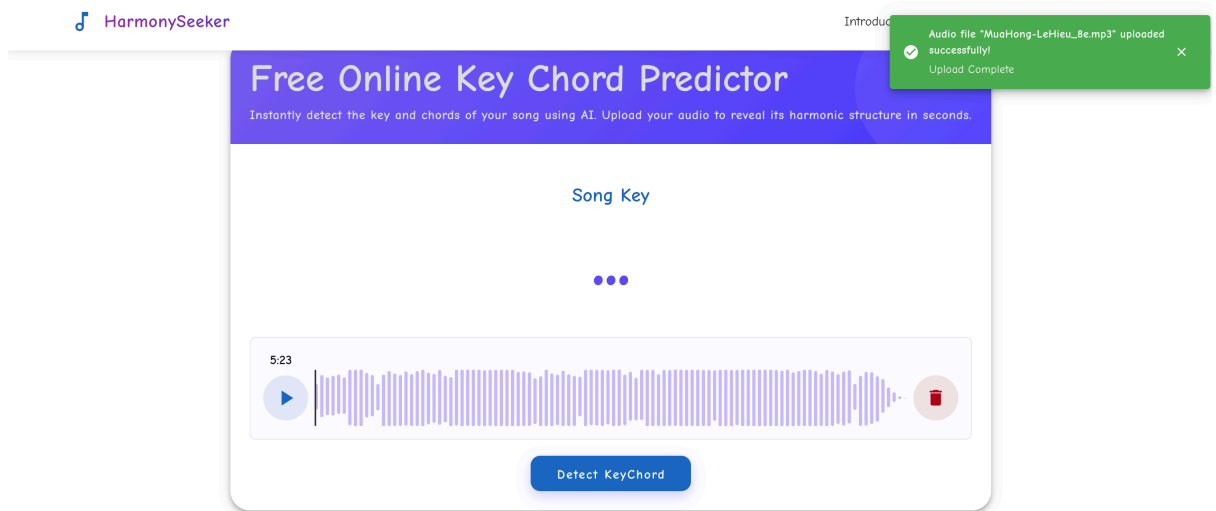


Figure 3.41: Successful file upload and system ready for analysis

Once a user uploads a valid audio file (less than 10MB in size), the waveform is visualized at the bottom of the screen. The “Detect KeyChord” button becomes available, allowing the user to initiate the prediction process.

❖ Key Chord Result Visualization

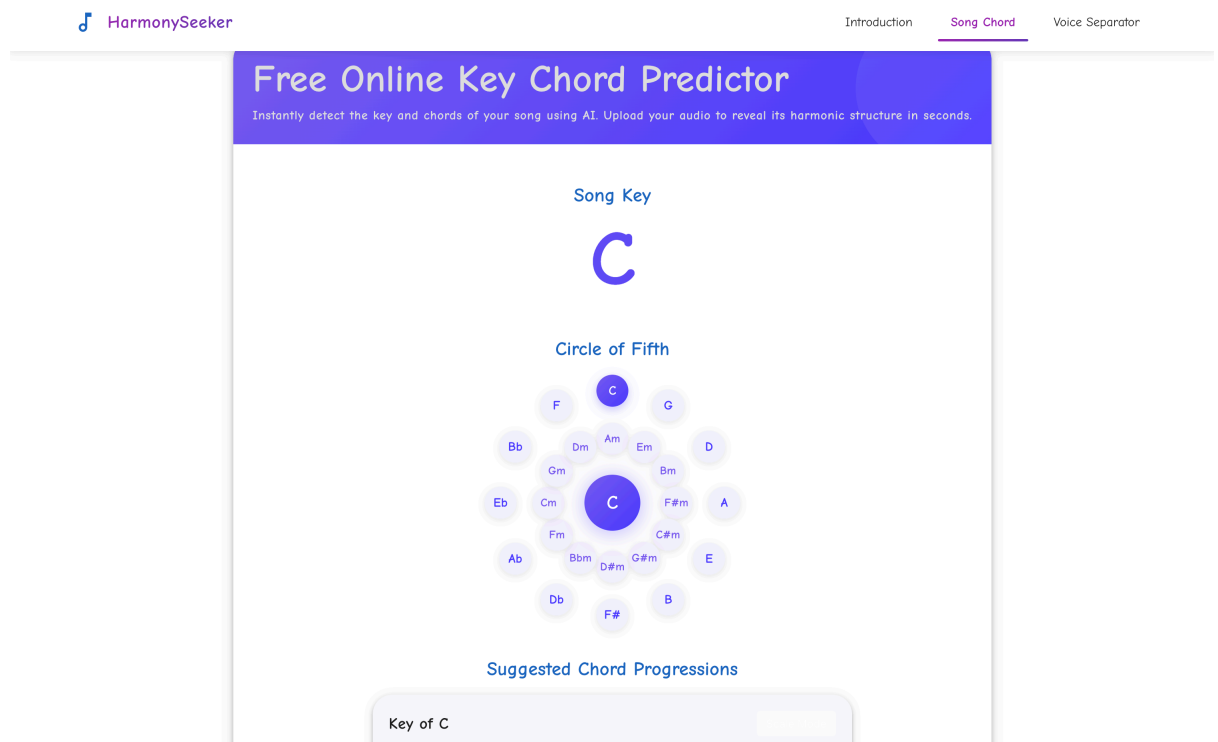


Figure 3.42: Visualization of the predicted key and suggested progressions

After analysis, the predicted key (e.g., C major) is shown at the top. The system also renders the Circle of Fifths and suggests common chord progressions based on the detected key. This helps users understand the functional harmony surrounding the tonic.

❖ Chord Timeline Visualization

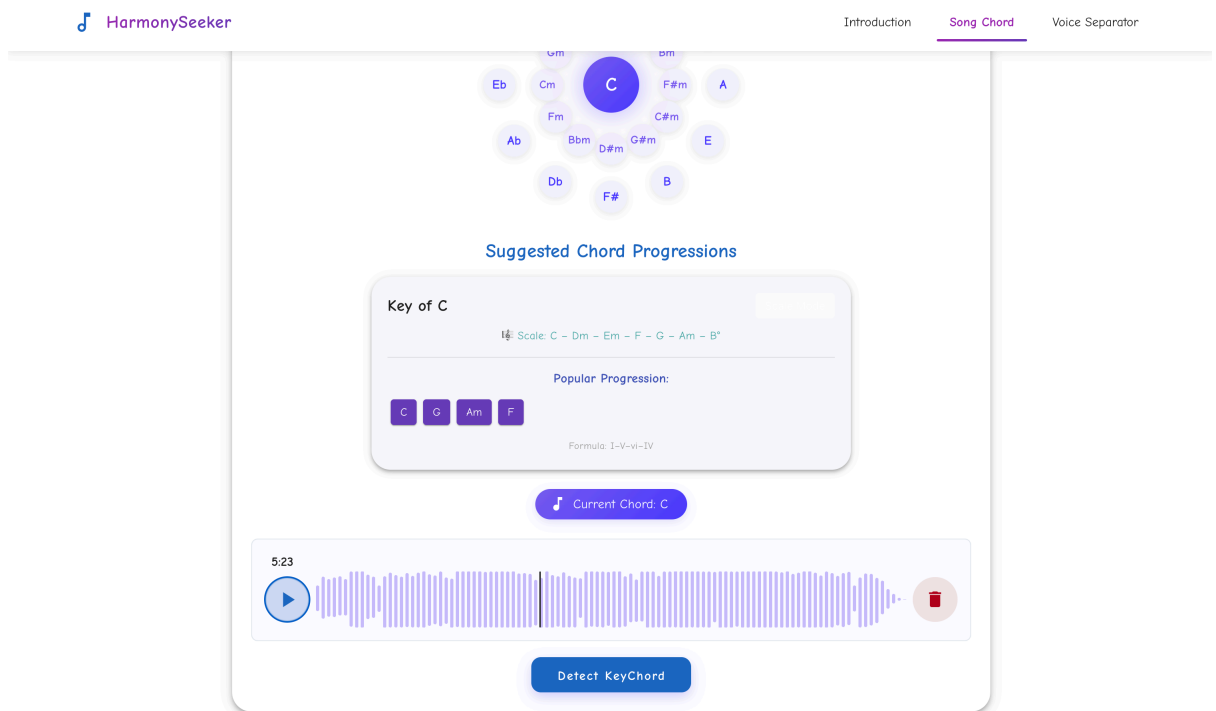


Figure 3.43: Frame-wise chord progression timeline with playback support

Users can play the song to display the exact chord at each point in time. The waveform player updates in sync with chord changes, offering a dynamic view of the song's harmonic structure across time.

❖ Upload Error (File > 10MB)

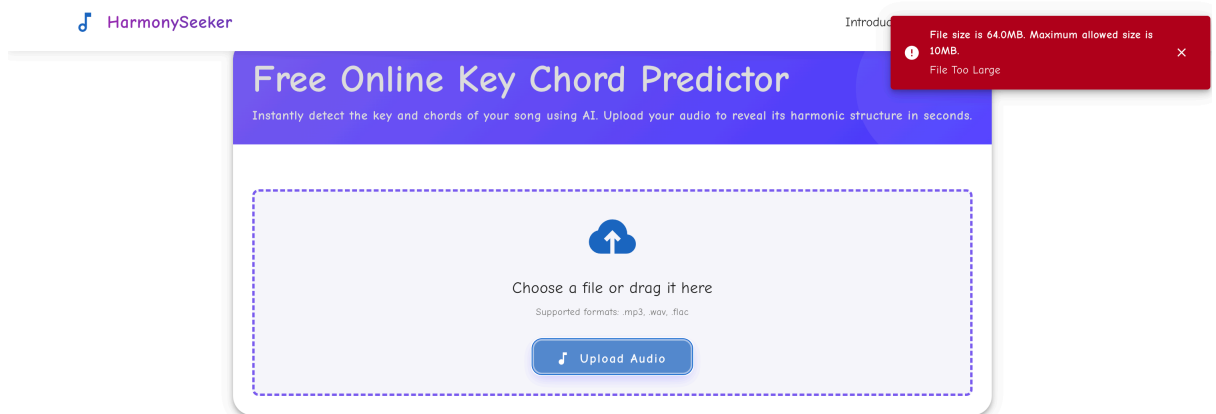


Figure 3.44: Error message for oversized file in song chord detection

If the uploaded file exceeds the 10MB limit, an error message is shown to the user, preventing analysis. This ensures stable performance and protects against excessive memory usage during inference.

3.4.3 Vocal Separator Screen

❖ Main Separator Interface

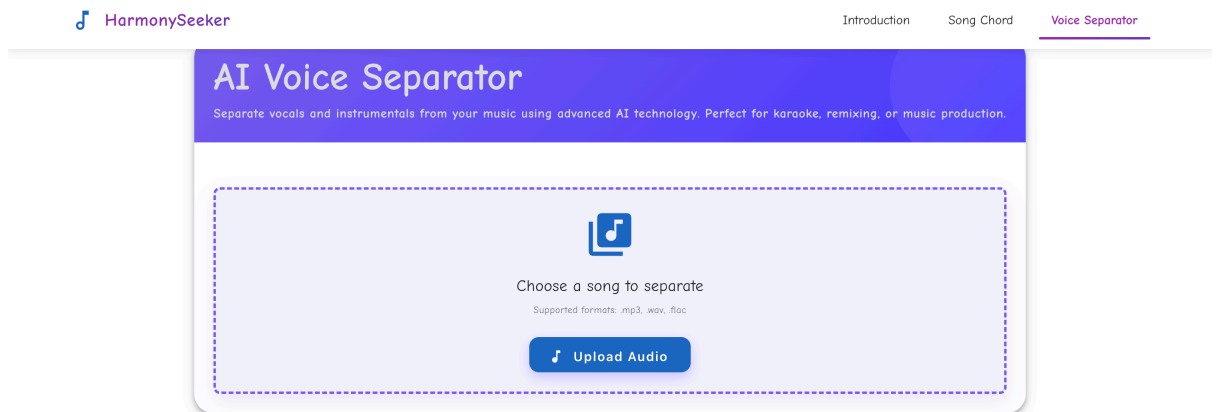


Figure 3.45: Vocal separator screen before processing

When the user navigates to the "Voice Separator" tab, the main separation interface is displayed. Users can upload an audio file to separate vocals and instrumentals using deep learning-based source separation.

❖ **Upload Success (File <10MB, Duration <60s)**

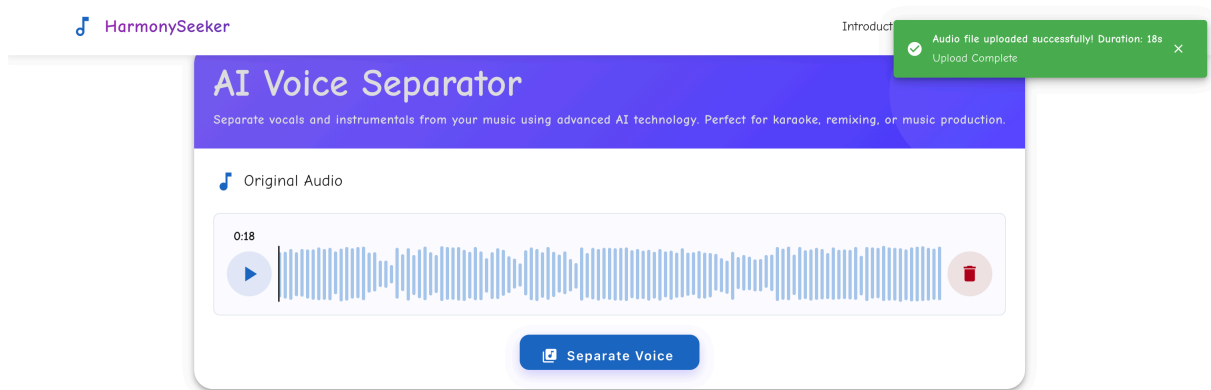


Figure 3.46: File uploaded successfully and ready for separation

Upon uploading a valid file (less than 10MB and 60 seconds in duration), the waveform of the original file is rendered, and the system begins separating the audio into components.

❖ Output: Separated Vocal & Instrumental

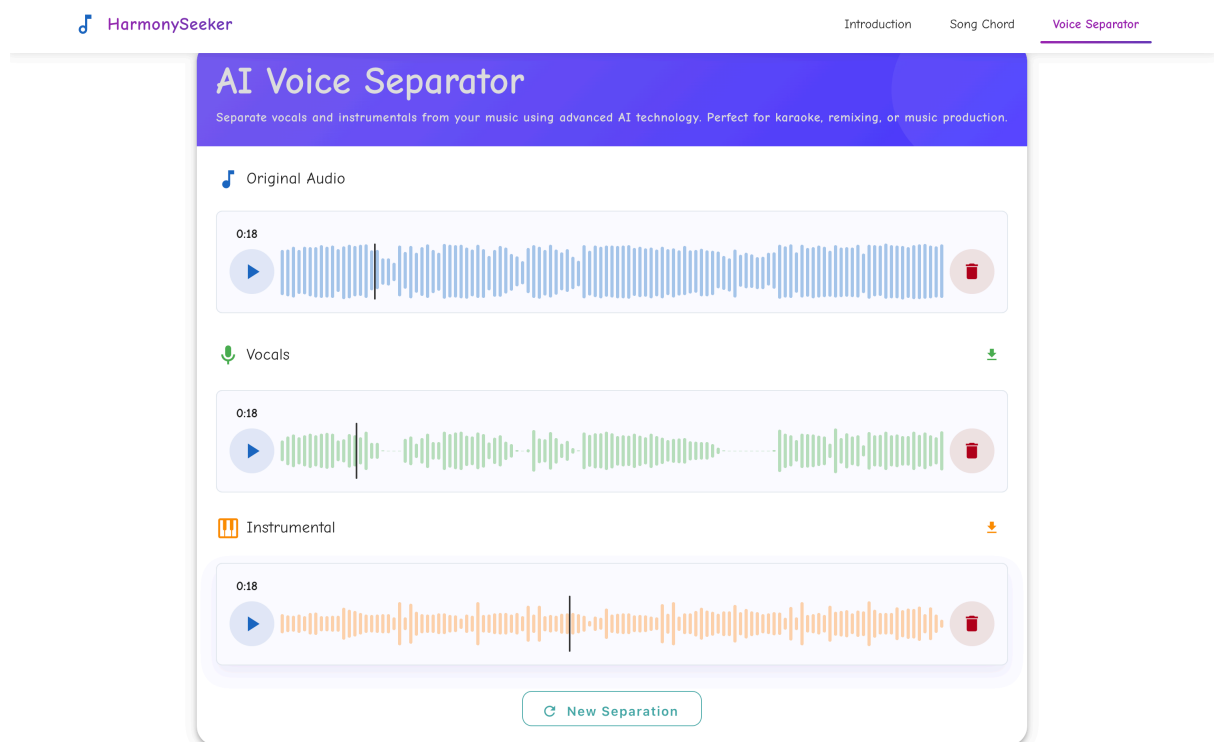


Figure 3.47: Display of original audio, separated vocals, and instrumentals

Once separation is complete, the interface displays three synchronized waveforms: original audio, isolated vocals (green), and instrumentals (orange). Each section can be played back or downloaded independently.

❖ Upload Error (File >10MB)

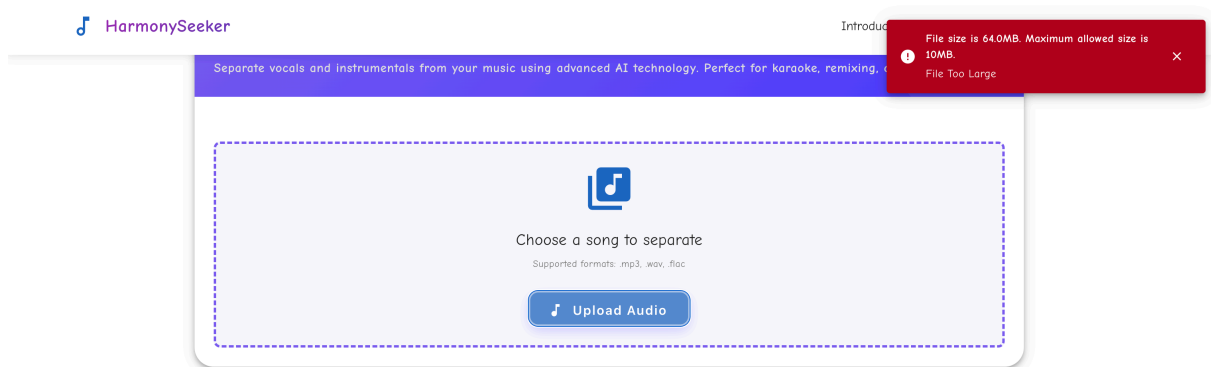


Figure 3.48: File size exceeds limit error in voice separator

If the file exceeds the 10MB size limit, the system will reject the upload and notify the user. This protects performance and ensures quick processing.

❖ Upload Error (Duration > 60 seconds)

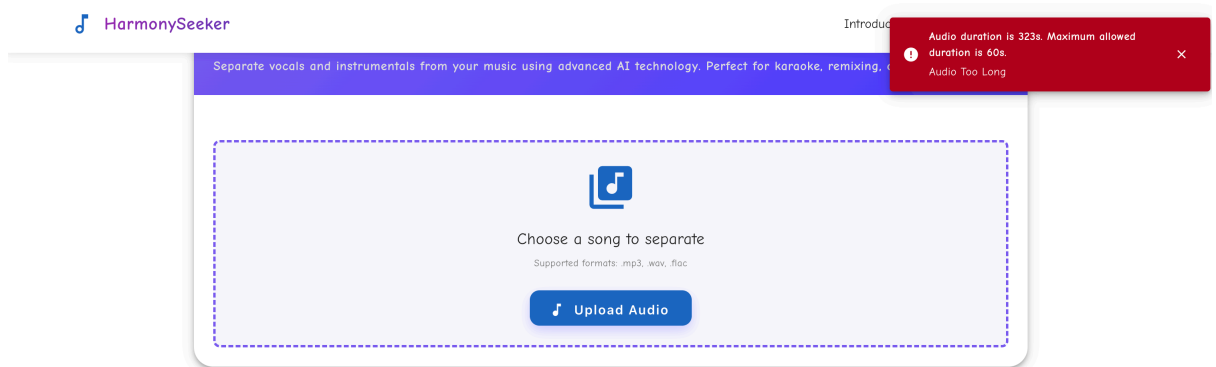


Figure 3.49: Error message for audio files exceeding 60 seconds

Similarly, if the uploaded audio exceeds 60 seconds in length, the system alerts the user and aborts the upload. This keeps separation latency minimal and prevents overload.

3.5 Conclusion

HarmonySeeker represents a comprehensive and multi-dimensional system that bridges music information retrieval (MIR), deep learning, and modern software deployment practices to support harmonic analysis and vocal separation in popular music. Through a systematic combination of audio processing, model design, infrastructure engineering, and frontend development, the platform provides a complete pipeline from raw music input to interpretable musical outputs.

❖ Robust AI for Chord and Key Recognition

At the core of HarmonySeeker is a CRNN-based chord recognition model designed to infer the primary chord (tonic) of a song. Leveraging large-scale annotated data and musically-informed evaluation schemes (including mode-aware mappings and semitone tolerance), the CRNN demonstrated high generalization ability, achieving up to 84.00% accuracy in the most forgiving configuration. These results were consistently higher than those of the CNN baseline, showing that temporal modeling and theoretical post-processing play a critical role in music understanding.

In particular, the evaluation highlighted common challenges in key estimation—especially confusion between relative major/minor chord pairs. By incorporating mode-awareness (mapping chords like C $\leftrightarrow$ A:min) and semi-strict evaluation (accepting off-by-one-semitone predictions), the model better aligns with real-world musical ambiguity and listener perception. The use of such techniques underscores the need to not only design strong models but also assess them through flexible, domain-aware metrics.

❖ Advanced Voice Separation with TFC-TDF-UNet

For source separation, the system employs the TFC-TDF-UNet architecture, which outperformed the baseline U-Net across all metrics. Specifically, it improved the Signal-to-Distortion Ratio (SDR) on the test set from -0.02 dB (UNet) to 0.70 dB, and lowered the training loss to 0.28. These enhancements confirm the effectiveness of combining Time-Frequency Convolutional modules with temporal fully connected layers in capturing both spectral and sequential structure in music.

This component is particularly useful for karaoke, remixing, or enhancing chord inference by removing vocal interference. While it was tested experimentally on inference over separated audio, results showed that source separation did not improve chord detection accuracy—likely due to artifacts introduced during separation. Nonetheless, this feature enriches the system’s versatility for end-users.

- **Practical Implementation as a Web Application**

To make the AI capabilities accessible, HarmonySeeker is implemented as a modern single-page web application. Users can upload songs, receive instant predictions for key and chords, and interactively visualize the harmonic structure and circle of fifths. Additionally, users can isolate vocals and instrumentals with a single click. The UI is designed with responsiveness and clarity in mind, enabling use across devices and skill levels.

Each feature was accompanied by clear error handling (e.g., file size and length restrictions), real-time waveform visualization, and suggestions for chord progressions—bridging the gap between raw prediction and musical application.

- **Scalable Cloud Deployment on AWS**

The entire system is deployed on Amazon Web Services (AWS), utilizing a range of cloud services to ensure responsiveness and reliability:

- **Frontend:** Hosted as a static site on Amazon S3, served globally via CloudFront CDN for fast delivery.
- **Backend & Model Inference:** Containerized using Docker and deployed on Amazon ECS, enabling horizontal scaling and high isolation.
- **Model Compute:** Served via an EC2 instance (4 vCPUs, 16 GiB RAM, Ubuntu 20.04 LTS) with GPU-ready infrastructure for future scaling.
- **Storage & Streaming:** All uploaded audio and processed assets are stored in S3, supporting asynchronous workflows and persistent storage.
- **Performance:** With a baseline transfer rate of 5 Gbps, the infrastructure handles both heavy traffic and real-time prediction with minimal latency.

This setup not only provides robustness and scalability but also reflects a real-world production deployment strategy—using container orchestration, stateless APIs, and serverless file hosting.

In summary, HarmonySeeker offers a full-stack solution that integrates AI model training, real-time inference, music theory-informed evaluation, intuitive frontend interfaces, and scalable deployment—all while maintaining flexibility, accuracy, and accessibility. It is not merely a technical demonstration but a practically viable system for chord/key recognition and vocal separation in music analysis contexts.

CONCLUSION

This project successfully developed an intelligent and user-friendly system for identifying primary song chords and separating vocals from music. By leveraging advancements in Artificial Intelligence, particularly in audio processing, the system addresses the challenges faced by music learners and enthusiasts in understanding song structure and practicing music.

❖ **Key Achievements:**

- **Tonic Chord Identification:** The system effectively identifies the main chord of a song from uploaded audio files, demonstrated by the ability to return accurate main chords for various audio inputs. This functionality is crucial for harmonic analysis and music theory understanding.
- **Vocal Separation:** A vocal remover feature was implemented to separate vocals from instrumental tracks. This enhances the accuracy of chord recognition from accompaniment and provides isolated audio files for practice, remixing, or analysis. The TFC_TDF_UNet model significantly outperformed the baseline UNet, indicating better generalization and separation quality.
- **System Development:** A functional web-based platform was built with an intuitive interface for uploading songs and viewing analysis results quickly. The system's architecture utilizes a serverless and microservices approach hosted on AWS, integrating monitoring and alerting through Google Chat to ensure availability and scalability.

❖ **Limitations:**

Despite these achievements, certain limitations were identified:

- **Dependency on Input Audio Quality:** The accuracy of the models can be affected by the quality of the input audio. Tracks with significant noise, overwhelming vocals, or unclear accompaniment may reduce the model's accuracy.
- **Small and Biased Datasets:** The use of limited datasets, particularly those focused on specific music styles, can lead to models that do not generalize well across diverse music genres.
- **Processing Speed Optimization:** The current feature processing and prediction stages are computationally intensive, making deployment on resource-constrained devices like mobile phones challenging.

❖ **Future Directions:**

To address these limitations and further enhance the system, the following directions are proposed for future development:

- **Data Augmentation:** Implement advanced data augmentation techniques such as adding noise and frequency transformations to enrich the dataset. This will improve the model's generalization capabilities across various musical styles and audio conditions.
- **Real-time Implementation:** Optimize inference time to enable real-time system deployment. This would support applications in education, live performance, and music composition assistance.
- **Enhanced User Interface:** Continue to develop a more interactive and comprehensive user interface that allows users to easily upload music and visualize chord analyses in real-time.

REFERENCES

- [1] P. F. T. B. Olaf Ronneberger, "U-Net: Convolutional Networks for Biomedical Image Segmentation," 18 May 2015.
- [2] W. Choi, "Deep Learning-based Latent Source Analysis for Source-aware Audio Manipulation," Korea University, 2021.
- [3] 2. Nuclear Engineering and Technology, "https://www.sciencedirect.com/," 2024. [Online]. Available: <https://www.sciencedirect.com/topics/engineering/short-time-fourier-transform>.
- [4] M. e. al, "Automatic Chord Estimation from Audio: A Review of the State of the Art," 2014.
- [5] M. & Dixon, "Approximate Note Transcription for the Improved Identification of Chords from Audio," 2010.
- [6] C. e. al., "Explaining Deep CNNs on Music Classification," 2016.
- [7] G. e. al, "Deep Learning models (CNN, LSTM/CRNN, etc.)," 2016.
- [8] O. L. R. G. A. M. B. Tal Remez, "Deep Convolutional Denoising of Low-Light Images," 6 Jan 2017.
- [9] Kojs, "A Survey of Serverless Machine Learning Model Inference," 2023.
- [10] Brown, "Calculation of a Constant Q Spectral Transform," 1991.
- [11] C. e. al, "Explaining Deep CNNs on Music Classification," 2016.
- [12] S. e. al, "Audio Chord Recognition with a Hybrid Recurrent Neural Network," 2015.
- [13] 2 29 2022. [Online]. Available: <https://sigsep.github.io/datasets/musdb.html>.
- [14] J. Jimenez, "Vocal Track Separation with Encoder-Decoder Architecture," [Online]. Available: https://keras.io/examples/audio/vocal_track_separation/. [Accessed 20 4 2025].
- [15] Y. L. Walid Ehab, "Performance Analysis of UNet and Variants for Medical Image Segmentation," 22 Sep 2023.

- [16] V. Belz, "Speech-enhancement with Deep learning," Jan 19, 2020.